Laboratory Record

CSE3006 Computer Networks

B11+B12+B13, BL2026270100338,

Fall Semester :2026-2027

Name of the student: Shashi Singh

Register Number: 24BAI10686

B.Tech Computer Science and Engineering

Programme Output (POs):

1. Engineering knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
2. Problem analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
3. Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
4. Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
5. Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
6. The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
7. Environment and sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
8. Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
9. Individual and teamwork: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
10. Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
11. Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own

work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

12. Life-long learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

Index

Exp. No.	Date	Name of the Experiment	Page Number
1.	25.07.2026	Study of Network Hardware & Functionalities and Create a LAN Configuration using Hub and Switch	1-12
2.	01.08.2026	Configure Mesh, Ring, Bus, Star and Hybrid Topologies using Cisco Packet Tracer	13-26
3.	08.08.2026	Inter-LAN Communication using Router Configuration and Working of Repeaters and Bridges	27-34
4.	22.08.2026	Error Correcting Techniques using Hamming Code and Cyclic Redundancy Check	35-41
5.	24.08.2026	Implementation of Data Link Layer (DLL) Protocols - Stop and Wait ARQ, Go-Back-N ARQ and Selective Repeat ARQ	42-51
6.	05.09.2026	IPV6 Addressing using Cisco Packet Tracer.	52-57
7.	05.09.2026	ARP using Cisco Packet Tracer.	57-67
8.	09.09.2026	A. To configure a wireless network using a wireless router and end devices in Cisco Packet Tracer. B. To implement an IoT setup in Cisco Packet Tracer.	68-75
9.	18.09.2026	Develop a TCP client-server program	76-79
10.	19.09.2026	Develop a client–server network communication program	80-84

Exp No: 1	Study of Network Hardware & Functionalities and Create a LAN Configuration using Hub and Switch
25.07.2026	

Aim:

a. To study various hardware components involved in the design and performance of networks, and to study various useful network commands.
b. To design and configure a LAN (Local Area Network) using a Switch and Hub.

Objective:

1. Installing, and familiarising oneself with, Cisco Packet Tracer.
2. Using different components of Cisco Packet Tracer, such as network devices, end devices, connectors, and so on.
3. Building a simple network by connecting two End Devices with wires.
4. Trying out various basic network commands, such as 'color', 'ipconfig', 'ipconfig all', 'cls', 'nslookup', 'control c', and 'ping', and learning their function(s).
5. Creation of a Local Area Network while making use of a Switch and Hub.

Simulation Tool:

CISCO Packet Tracer

Theory:

Packet Tracer is a cross-platform visual simulation tool designed by Cisco Systems that allows users to create network topologies and imitate modern computer networks. The software allows users to simulate the configuration of Cisco routers and switches using a simulated command line interface. Packet Tracer makes use of a drag and drop user interface, allowing users to add and remove simulated network devices as they see fit.

Some important network devices are routers, switches, and hubs.

A router is a device that connects two or more packet-switched networks or subnetworks. It serves two primary functions: managing traffic between these networks by forwarding data packets to their intended IP addresses, and allowing multiple devices to use the same Internet connection.

A switch connects devices on a computer network by using packet switching to receive and forward data to the destination device. It learns the identities of connected devices and then only forwards data to the port connected to the device to which it is addressed.

A hub is a node that broadcasts data to every computer or Ethernet-based device connected to it. It is best suited for small, simple local area network (LAN) environments.

Color is an inbuilt command found inside the Windows Command Processor (cmd.exe), that is used for changing the colors for the console's foreground and background.

IPCONFIG stands for Internet Protocol Configuration. This is a command-line application which displays all the current TCP/IP (Transmission Control Protocol/Internet Protocol) network configuration, and refreshes the DHCP (Dynamic Host Configuration Protocol) and DNS (Domain Name Server). It also displays IP address, subnet mask, and default gateway for all adapters. 'ipconfig all' shows even more detailed information about the same.

Cls is a command used to clear the screen, or console window, of commands executed prior, and their outputs.

Nslookup lets an Internet server administrator or any computer user enter a host name and find out the corresponding IP address or domain name system (DNS) record. The user can also enter a command for it to do a reverse DNS lookup and find the host name for an IP address that is specified.

Control C aborts the current task and allows the user to regain control. Ping's most basic use is to confirm network connectivity between two hosts. It sends out an ICMP echo request to which it expects an ICMP echo reply response (ICMP stands for Internet Control Message Protocol).

LAN (Local Area Network):

LAN stands for local area network. A network is a group of two or more connected computers, and a LAN is a network contained within a small geographic area, usually within the same building. Home WiFi networks and small business networks are common examples of LANs.

The simplest Internet-connected LANs require only a router and a way for computing devices to connect to the router, such as via Ethernet cables or a WiFi hotspot. LANs without an Internet connection need a switch for exchanging data. Large LANs, such as those in a large office building, may need additional routers or switches to more efficiently forward data to the right devices.

A switch is a networking device that connects devices in a network and uses packet switching to send, receive or forward data packets or data frames over the network. A switch has many ports, to which computers are plugged in. When a data frame arrives at any port of a network switch, it examines the destination address, performs necessary checks and sends the frame to the corresponding device(s). It supports unicast, multicast as well as broadcast communications.

A network hub is a node that broadcasts data to every computer or Ethernet-based device connected to it. A hub is less sophisticated than a switch, the latter of which can isolate data transmissions to specific devices.

Network hubs are best suited for small, simple local area network (LAN) environments. Hubs cannot provide routing capabilities or other advanced network services. Because they operate by forwarding packets across all ports indiscriminately, network hubs are sometimes referred to as "dumb switches."

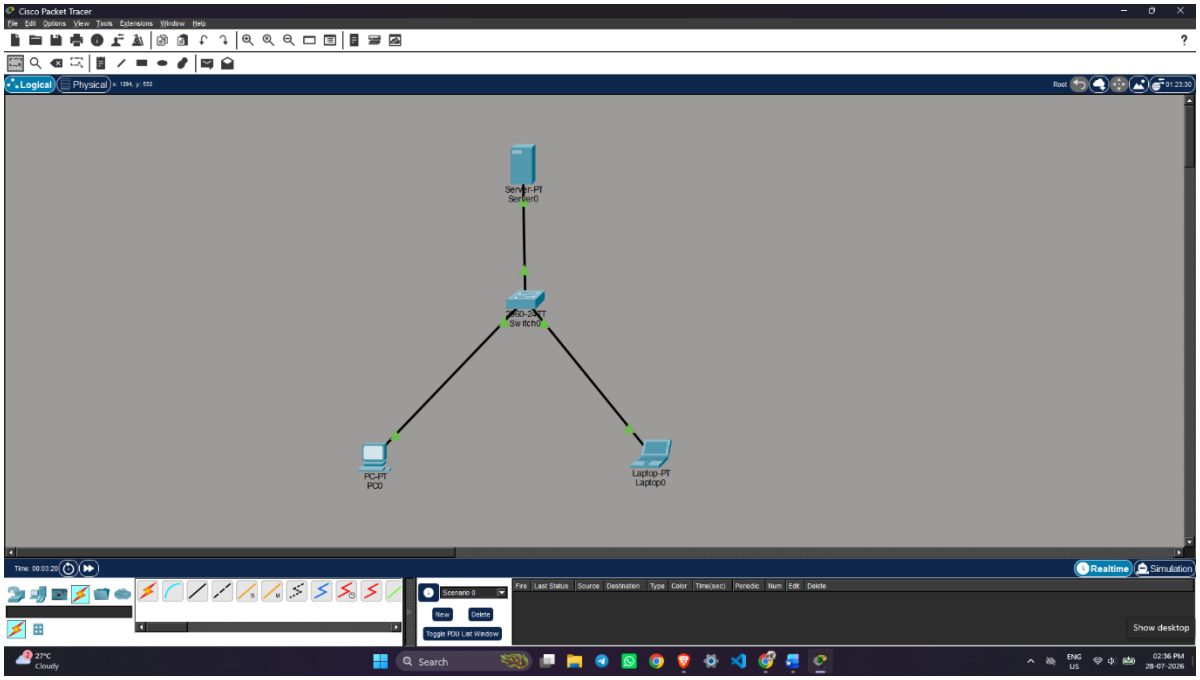
Procedure:

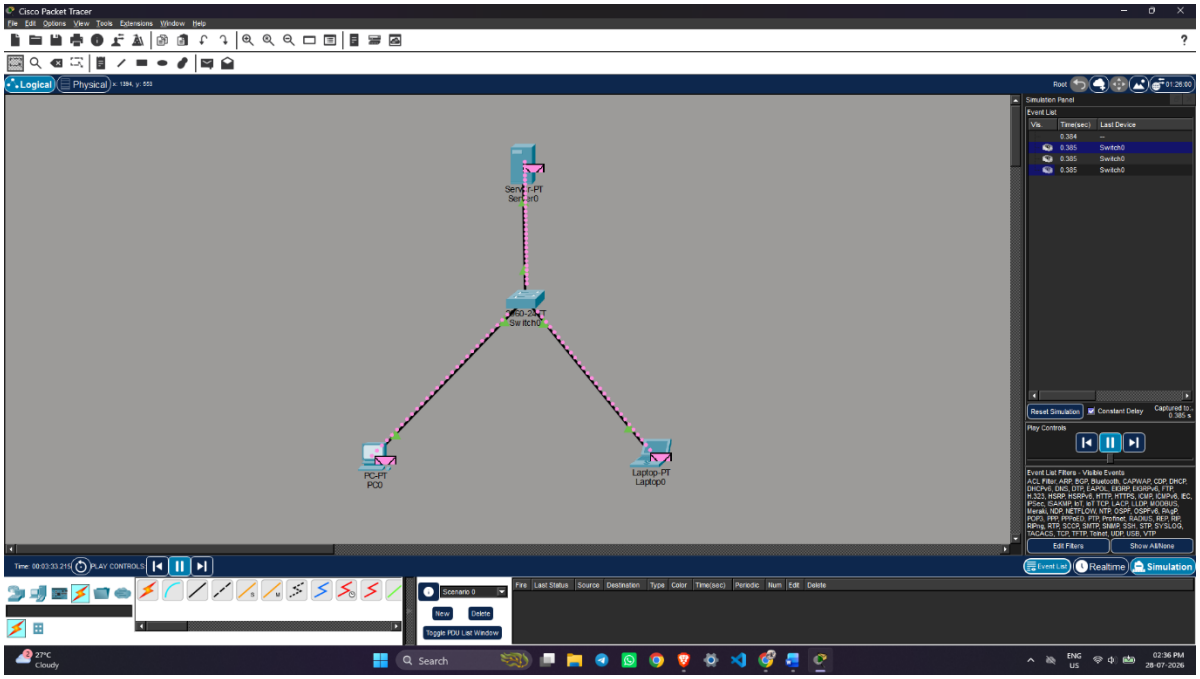
a. **Installation of CISCO Packet Tracer and study exercise:**
   - First, look up “Cisco Packet Tracer” or similar, on your search engine of choice.
   - You should find a webpage on the Cisco Networking Academy site. Click on that, then click on “View Courses”.
   - You should now have been redirected to the Skills For All website. Find the “Getting Started with Cisco Packet Tracer” course, and click on “Get Started”.
   - Log in with any account of your choice.
   - You should now be able to access the course material. On one of the first few pages, you should be able to find a link that redirects you to a Resources page.
   - From here, you can download (and, subsequently, install) the right version.
   - After looking through all of the various menus and options, trying out a few devices, testing how connections work etc. Select any two End Devices of your choice, and move them onto the main area.
   - Select a switch, and a server, and move them to the main area as well.
   - Connect them with wires as appropriate (there is an option titled “Automatically Choose Connection type” which picks the right one for you).
   - A network with two end devices has now been created!
   - For the next task, open up the Command Prompt.
   - Type in the aforementioned commands one by one, and take screenshots of their outputs, if any.

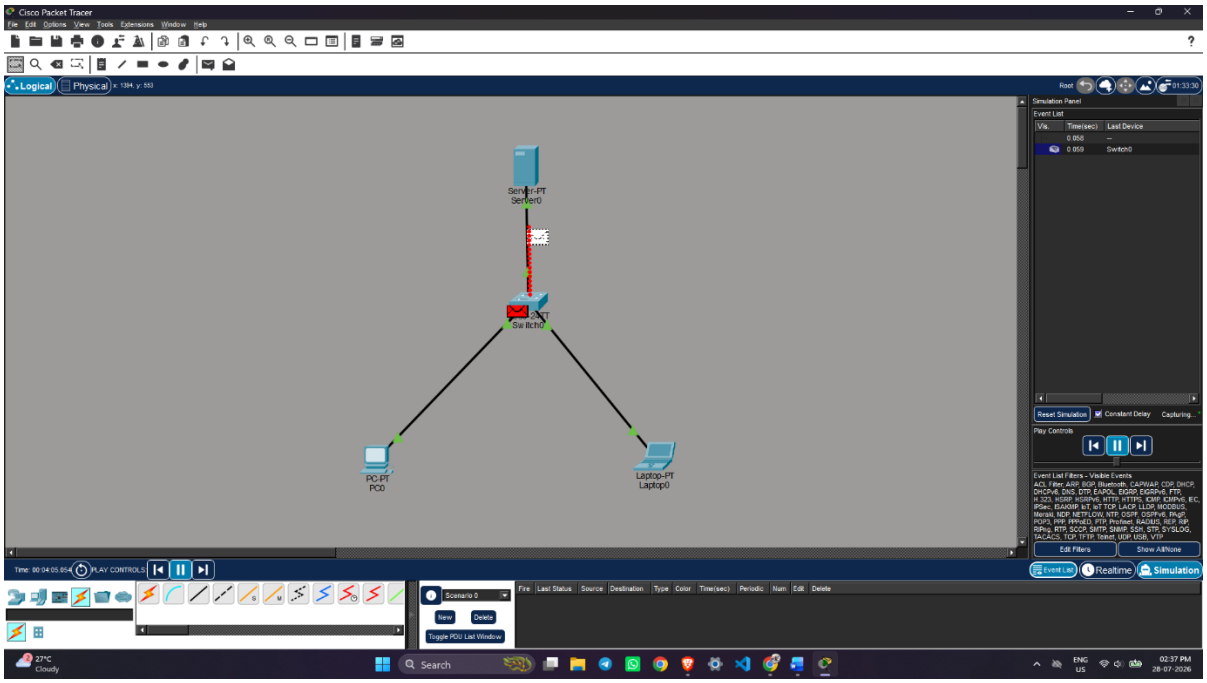
b. **Create a LAN Configuration using a Switch and Hub**
   - Startup Cisco Packet Tracer, and place any 4 end devices of your choice.
   - Connect them all using a switch/ Hub.
   - For each of them: double-click the device, go to the Desktop tab, click on IP Configuration and set an IP.
   - Make sure to give them all similar IPs – for eg, if an address is p.q.r.s, keep the ‘p’, ‘q’, and ‘r’ values (1 to 254) equal for all 4 end devices.
   - In one of the toolbars, on the top left, click on ‘Add Simple PDU’.
   - Select one of the devices to be the sender of the message, and another to be the recipient.

- Click on ‘Simulation’ in the bottom right, and start the simulation of the Scenario.
- Take screenshots as and when necessary, throughout the duration of the simulation.

Output: *(Simulation Screenshots)*

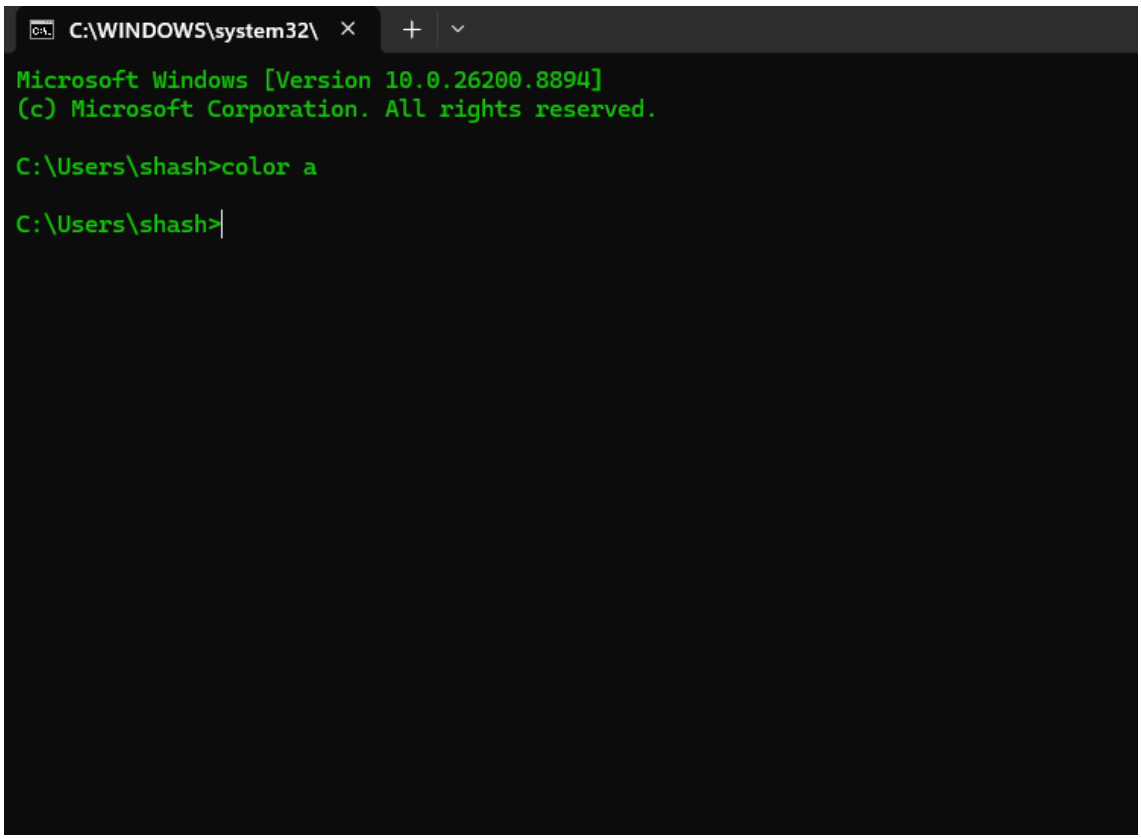






a. **Terminal command outputs**

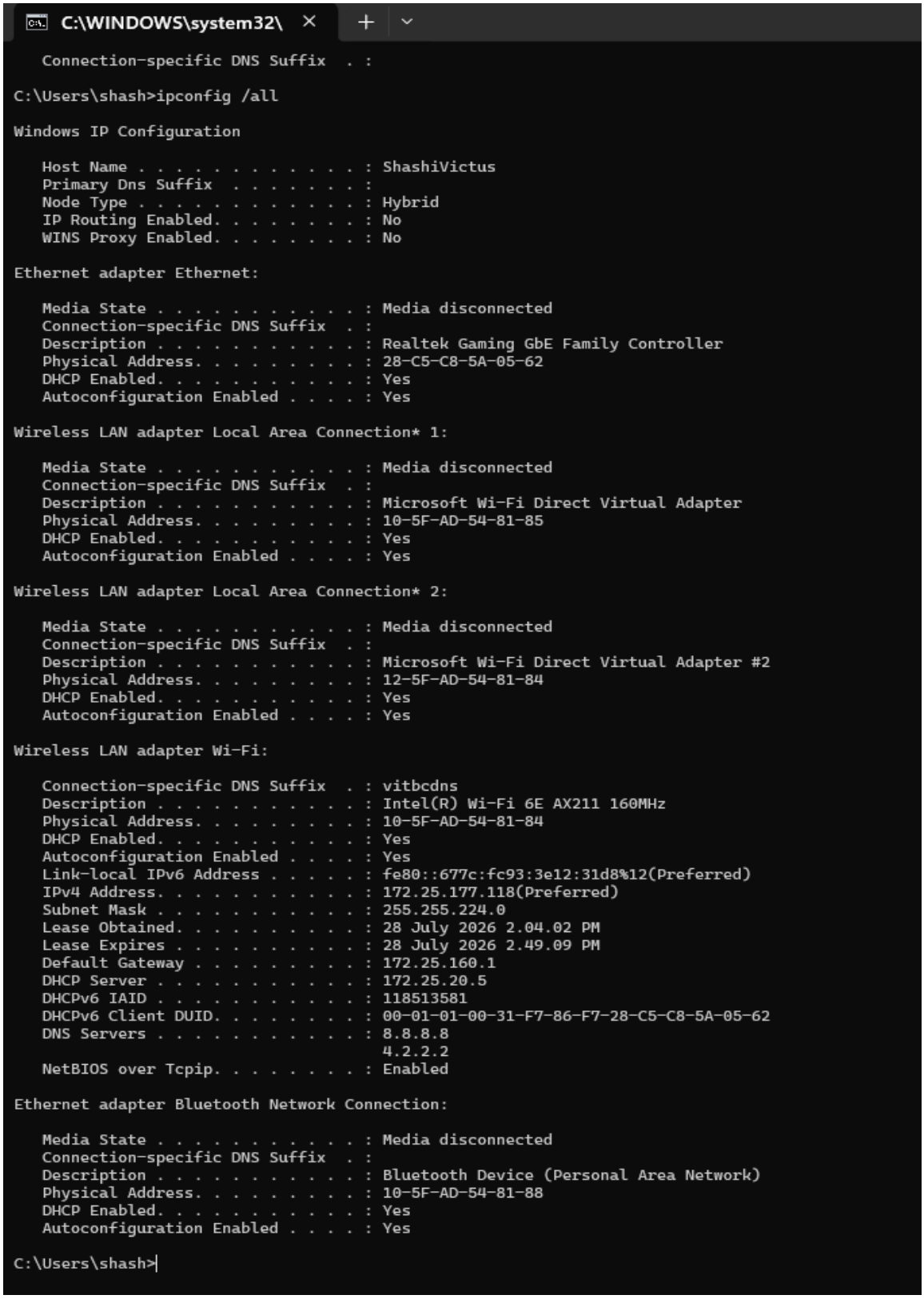
- **color**



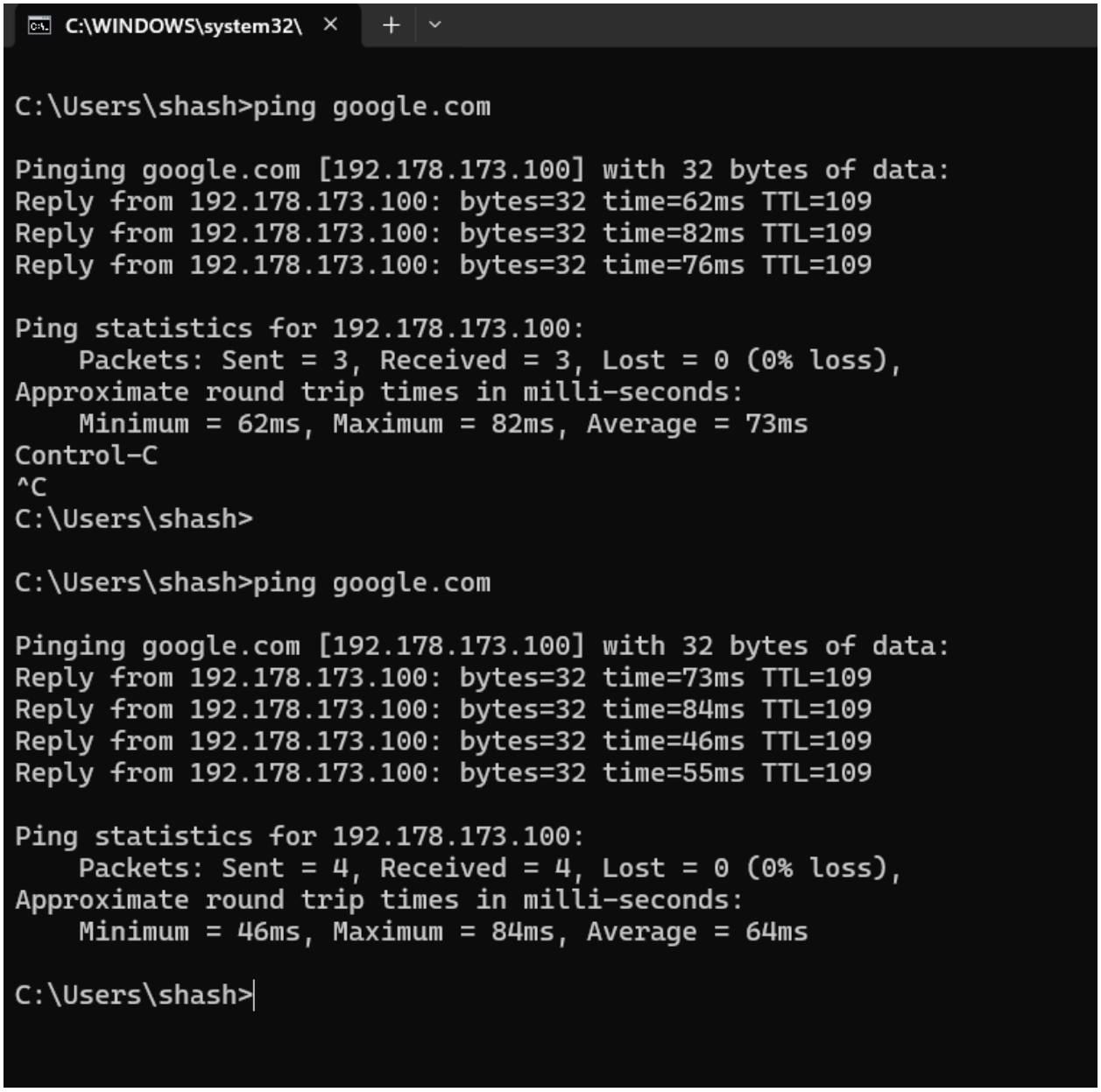
- **ipconfig**


C:\WINDOWS\system32\
Microsoft Windows [Version 10.0.26200.8894]
(c) Microsoft Corporation. All rights reserved.
C:\Users\shash>color a
C:\Users\shash>color
C:\Users\shash>ipconfig
Windows IP Configuration
Ethernet adapter Ethernet:
Media State . . . . . : Media disconnected
Connection-specific DNS Suffix . :
Wireless LAN adapter Local Area Connection* 1:
Media State . . . . . : Media disconnected
Connection-specific DNS Suffix . :
Wireless LAN adapter Local Area Connection* 2:
Media State . . . . . : Media disconnected
Connection-specific DNS Suffix . :
Wireless LAN adapter Wi-Fi:
Connection-specific DNS Suffix . : vitbcdns
Link-local IPv6 Address . . . . . : fe80::677c:fc93:3e12:31d8%12
IPv4 Address. . . . . : 172.25.177.118
Subnet Mask . . . . . : 255.255.224.0
Default Gateway . . . . . : 172.25.160.1
Ethernet adapter Bluetooth Network Connection:
Media State . . . . . : Media disconnected
Connection-specific DNS Suffix . :
C:\Users\shash>

- **ipconfig /all**



- **ping google.com**



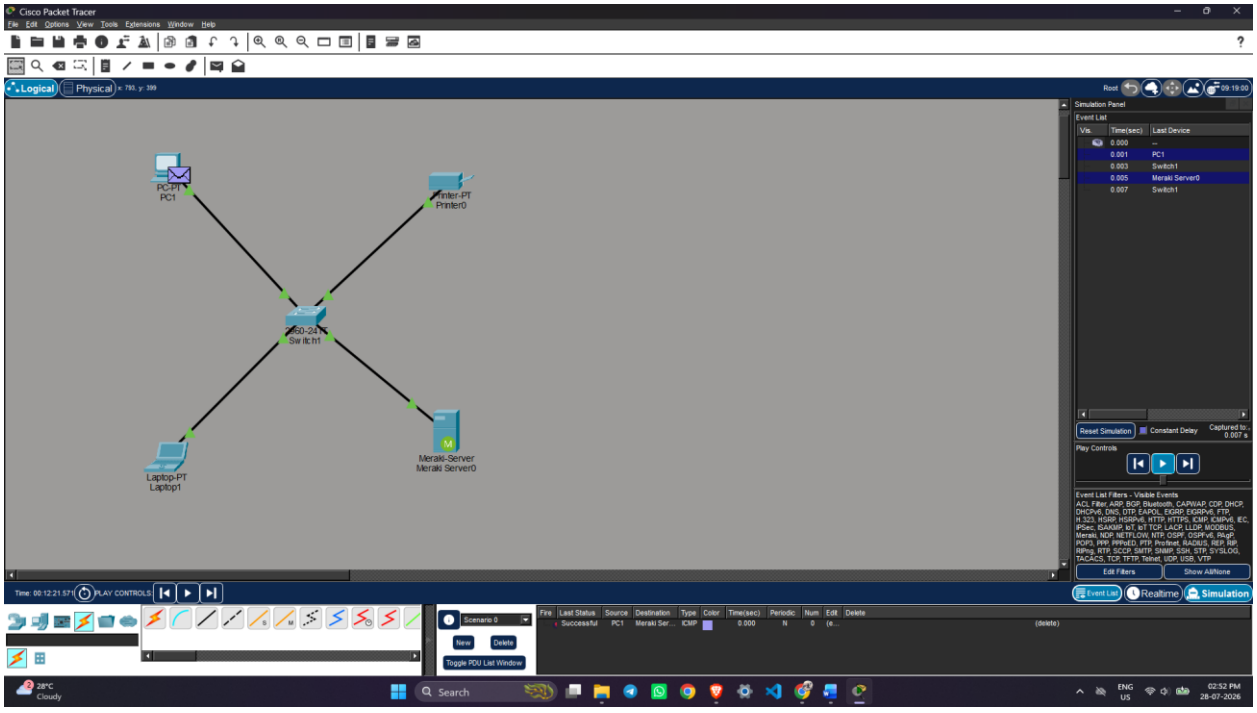
- **nslookup google.com**

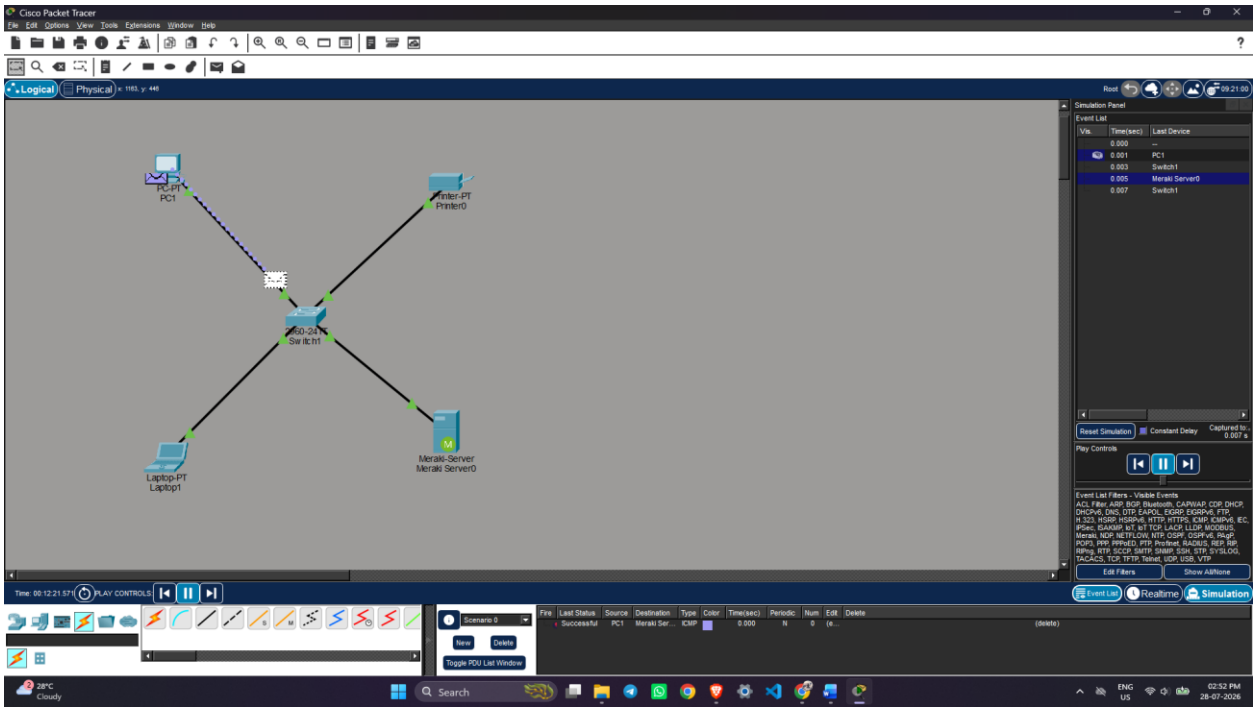
```
C:\Users\shash>nslookup google.com
Server:  dns.google
Address:  8.8.8.8

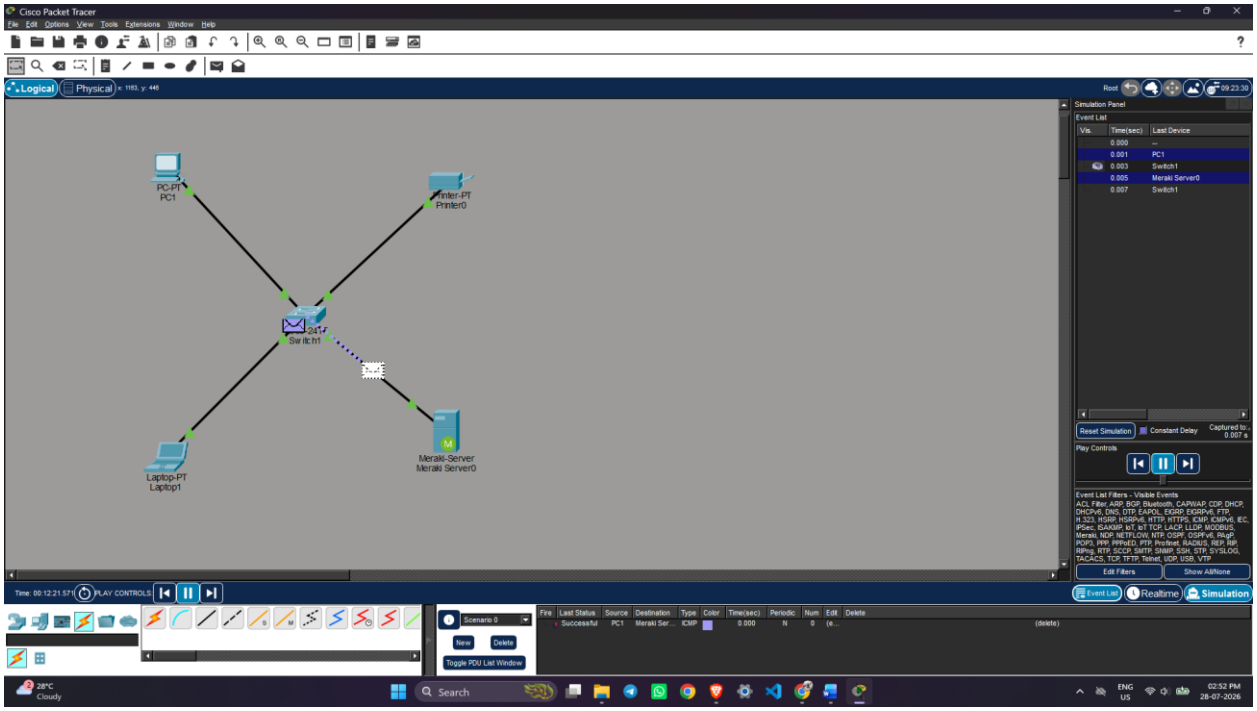
Non-authoritative answer:
Name:    google.com
Addresses:  2404:6800:4000:1017::64
          2404:6800:4000:1017::65
          2404:6800:4000:1017::8b
          2404:6800:4000:1017::66
          142.250.146.113
          142.250.146.101
          142.250.146.139
          142.250.146.100
          142.250.146.102
          142.250.146.138

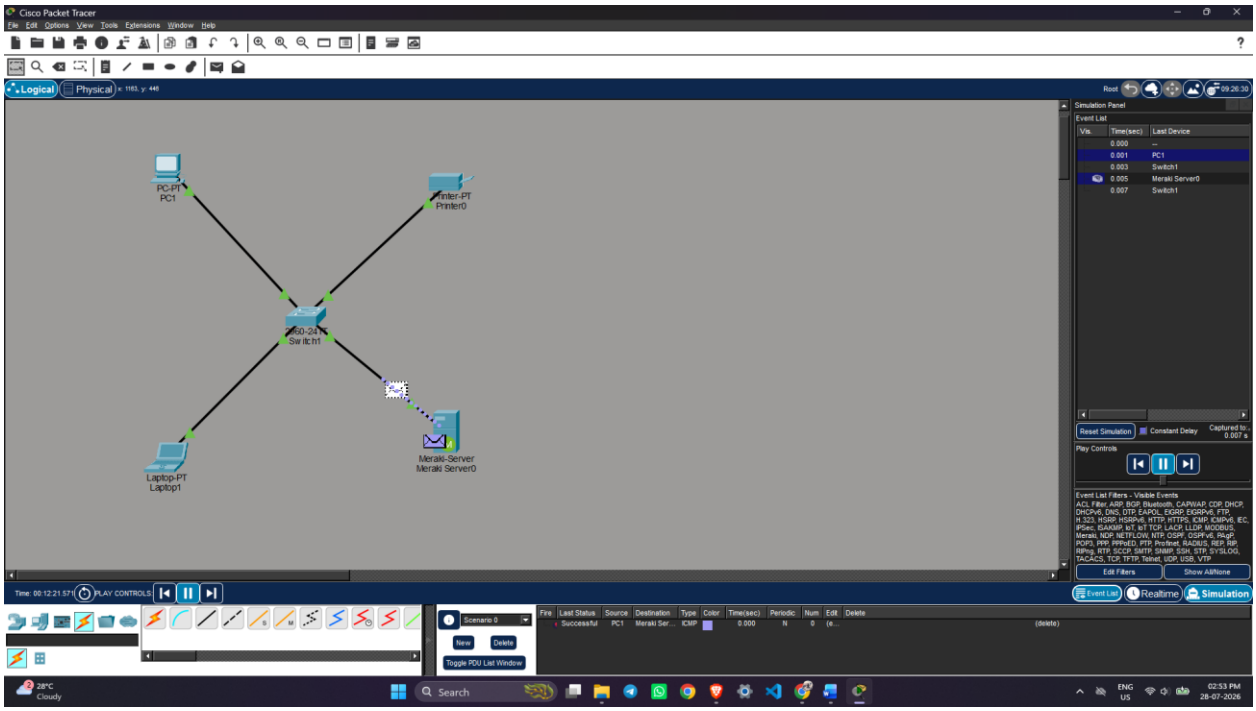

C:\Users\shash>
```

b) LAN Configuration




Cisco Packet Tracer
PC-PT PC1
Printer-PT Printer0
Switch1
Meraki-Server Meraki Server0
Laptop-PT Laptop1


Cisco Packet Tracer
PC-PT PC1
Printer-PT Printer0
Switch1
Meraki-Server Meraki Server0
Laptop-PT Laptop1


Cisco Packet Tracer
Logical
Physical
PC-PT
PC1
Printer-PT
Printer0
Switch1
Laptop-PT
Laptop1
Meraki-Server
Meraki Server0
Simulation Panel
Event List
Reset Simulation
Constant Delay
Play Controls
Edit Filters
Show All/None
Event List
Realtime
Simulation
Scenario 0
New
Delete
Toggle PDU List Window
Search

Result:

a. The function of various commands (some network-related, some general purpose) in the Command Prompt is executed and learnt about Cisco Packet Tracer, and how to simulate basic networks.
b. A simple LAN network using a switch and Hub is created and executed in cisco packet tracer.

References:

1. https://www.netacad.com/courses/packet-tracer

2. https://skillsforall.com/course/getting-started-cisco-packet-tracer

3. https://www.cloudflare.com/learning/network-layer/what-is-a-lan/

4. https://www.tutorialspoint.com/what-are-switches-in-computer-network

Exp No: 2	Configure Mesh, Ring, Bus, Star and Hybrid Topologies using Cisco Packet Tracer
01.08.2026	

Aim:

To configure and simulate Mesh, Ring, Bus, Star, and Hybrid network topologies using Cisco Packet Tracer and analyze their communication characteristics.

Objectives:

- To create different network topologies in Cisco Packet Tracer.
- To configure IP addresses for all network devices.
- To simulate data transmission in each topology.
- To compare the behavior and communication of different network topologies.

Simulation Tool:

Cisco Packet Tracer

Theory:

A network topology defines the physical or logical arrangement of devices in a network. Different topologies are used depending on network size, cost, reliability, and performance requirements.

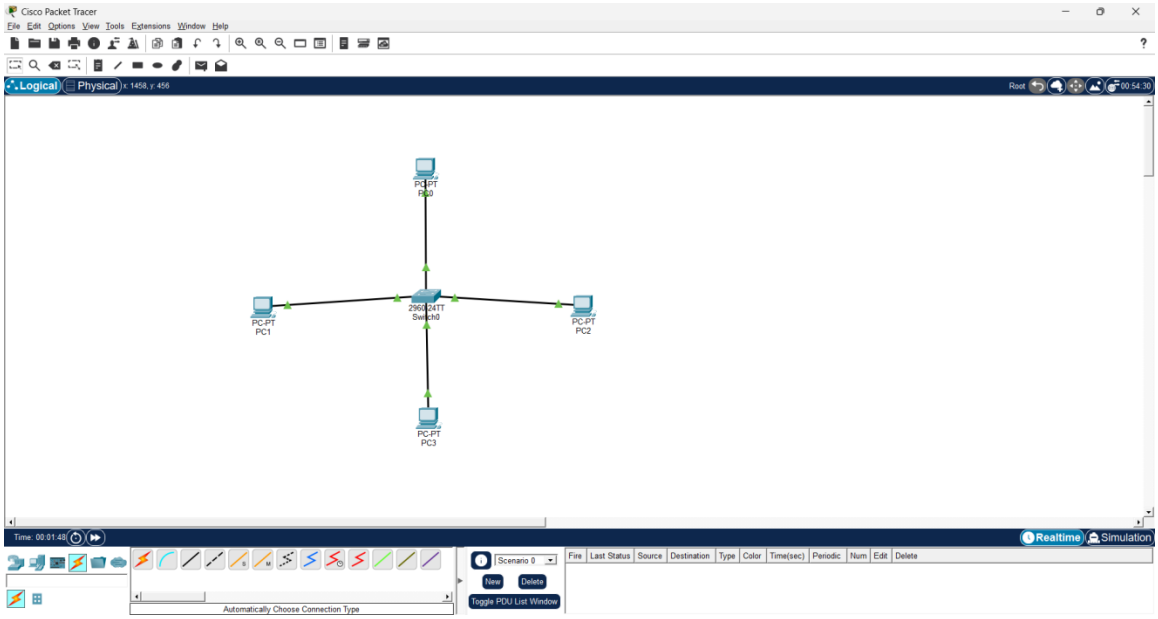
- **Bus Topology:** All devices are connected to a single communication cable called the bus. Data travels along the cable, making it simple and inexpensive, but failure of the main cable can disrupt the entire network.
- **Star Topology:** Every device is connected to a central hub or switch. It is easy to install, manage, and troubleshoot. Failure of one cable affects only one device, but failure of the central device affects the entire network.
- **Ring Topology:** Devices are connected in a circular path where data travels around the ring. It provides orderly communication but can be affected if one connection fails.
- **Mesh Topology:** Every device is connected directly to every other device. It offers high reliability and fault tolerance but requires more cables and higher installation cost.
- **Hybrid Topology:** A combination of two or more network topologies. It provides flexibility, scalability, and improved performance by combining the advantages of different topologies.

Each topology has its own advantages and limitations regarding scalability, fault tolerance, maintenance, and cost. The choice depends on the requirements of the network.

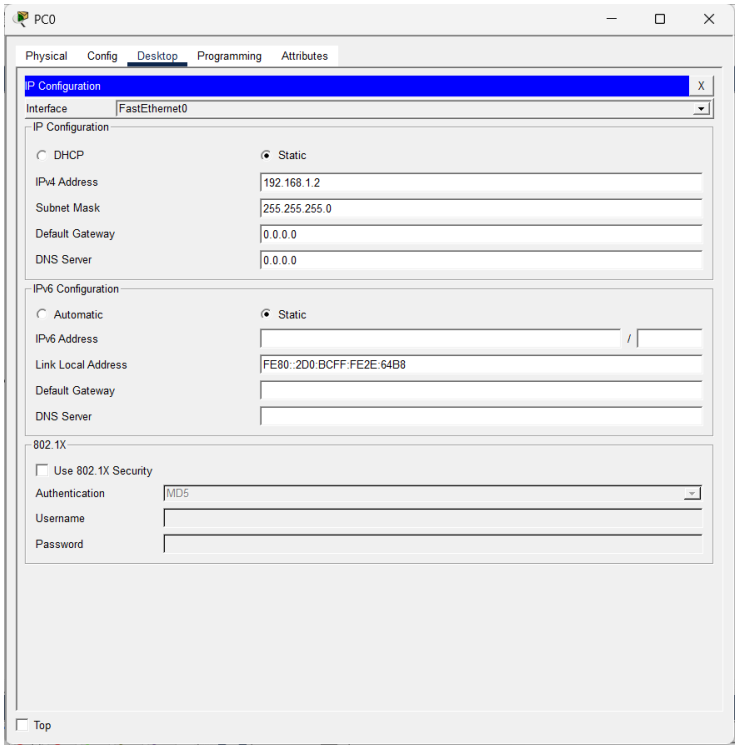
Output: ***(Simulation Screen Shots)***

- **STAR Topology:**

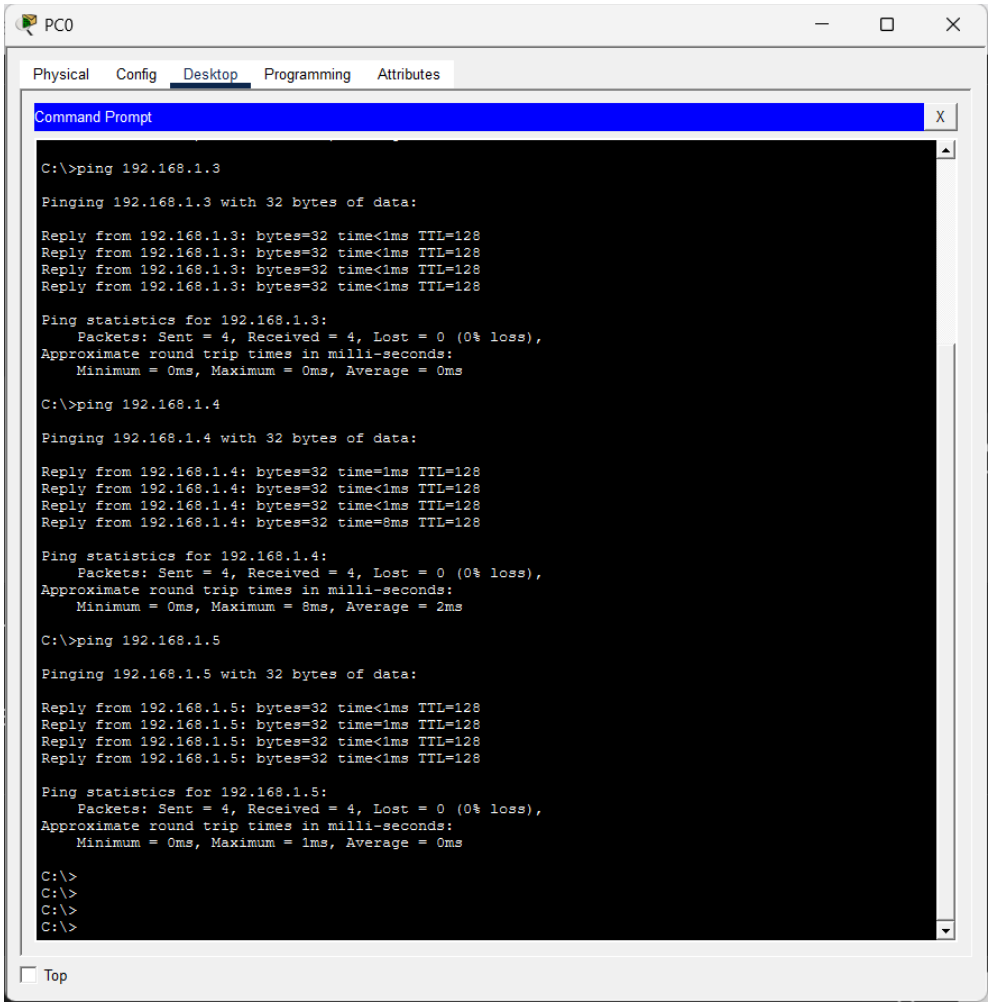
Basic structure:



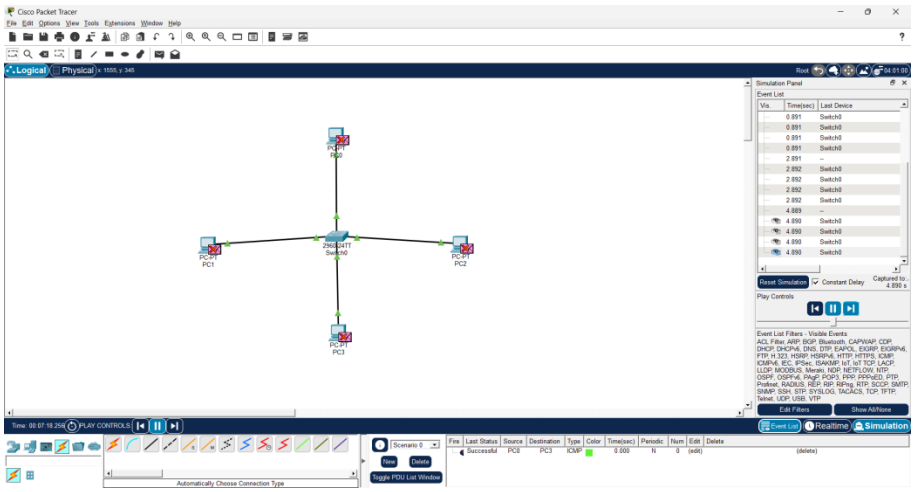
IP Congifuration:



PING Status:

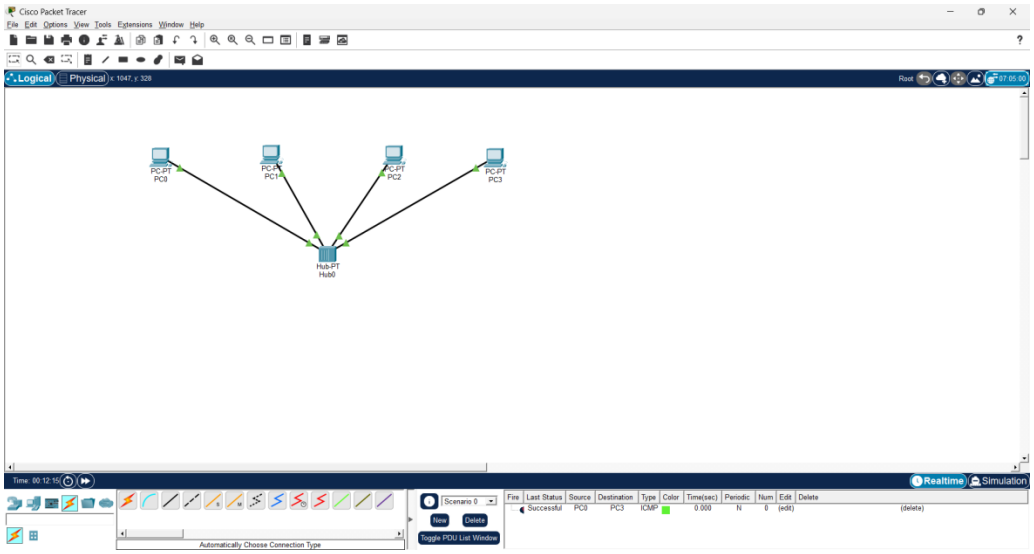


Simulation:

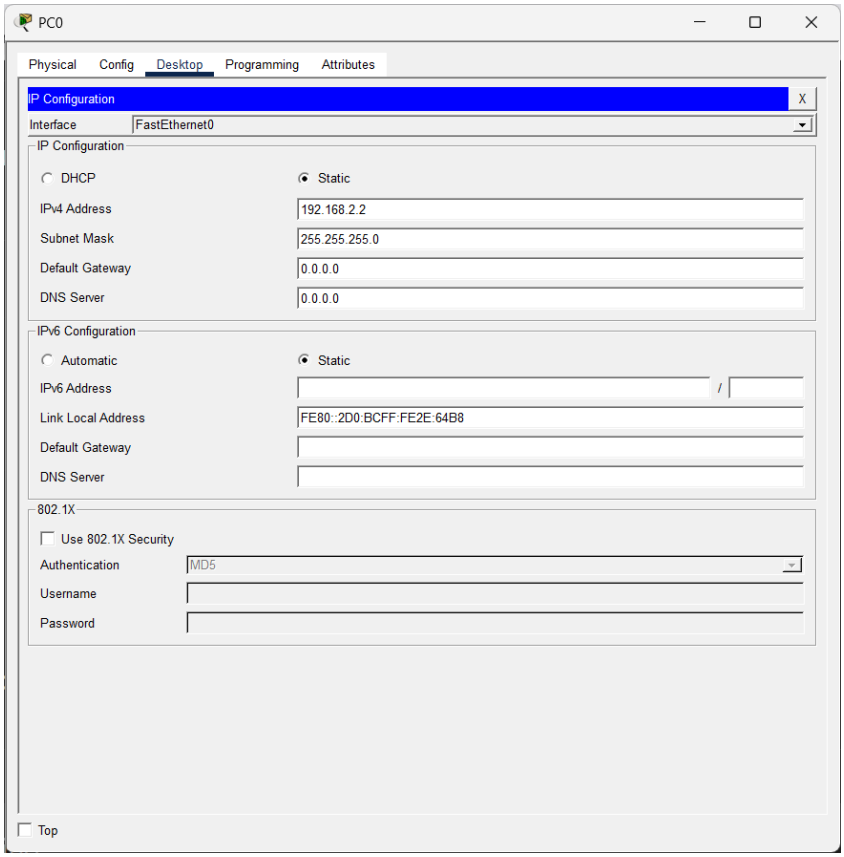


- **BUS Topology:**

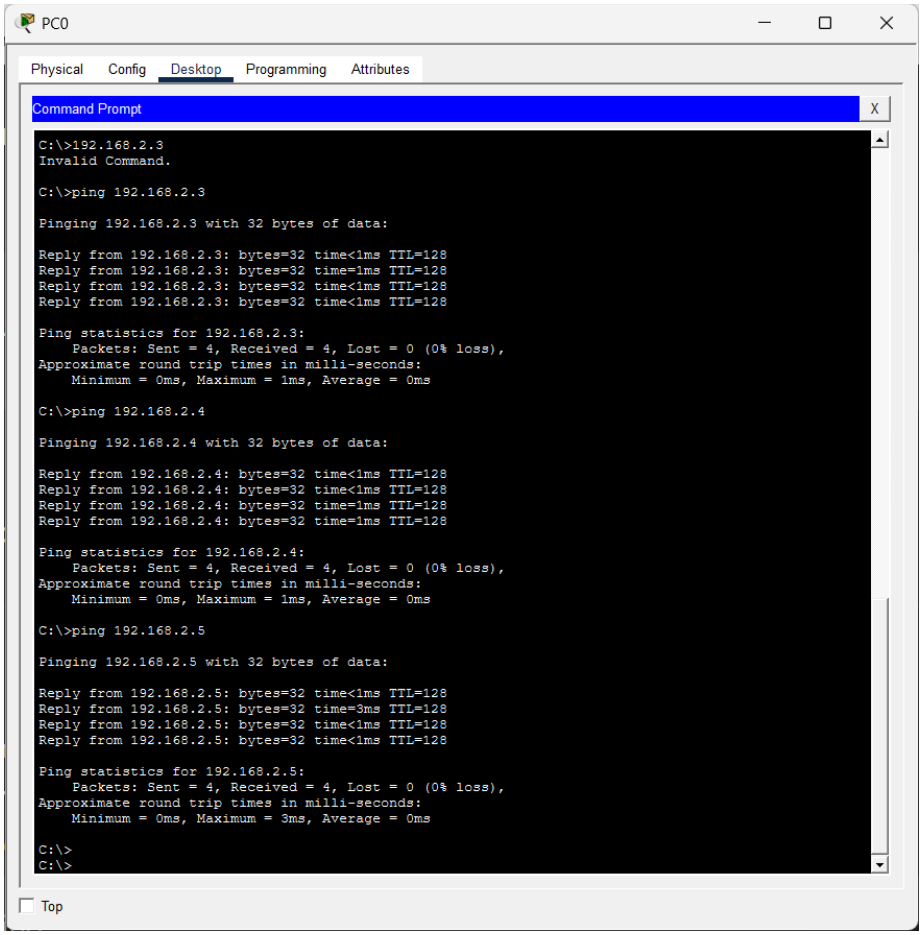
Basic Structure:



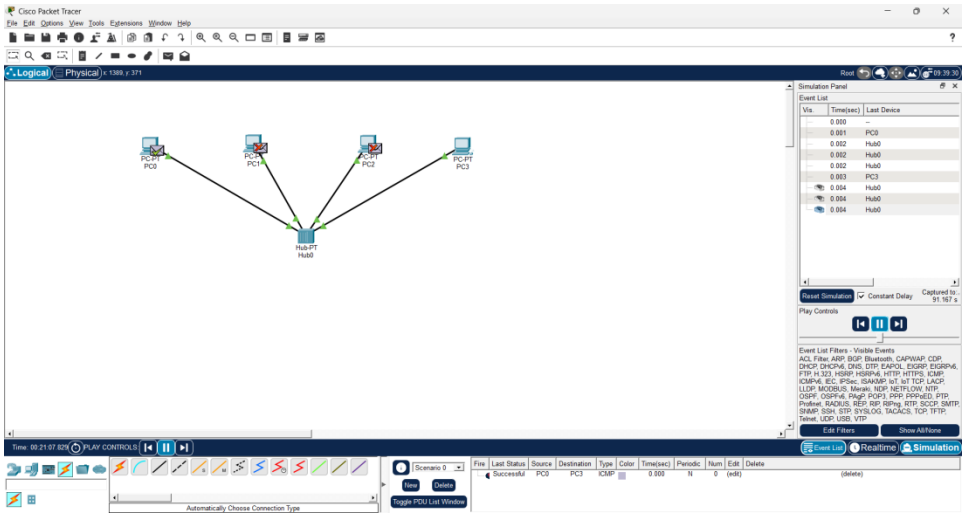
IP Configuration:



Ping Status:

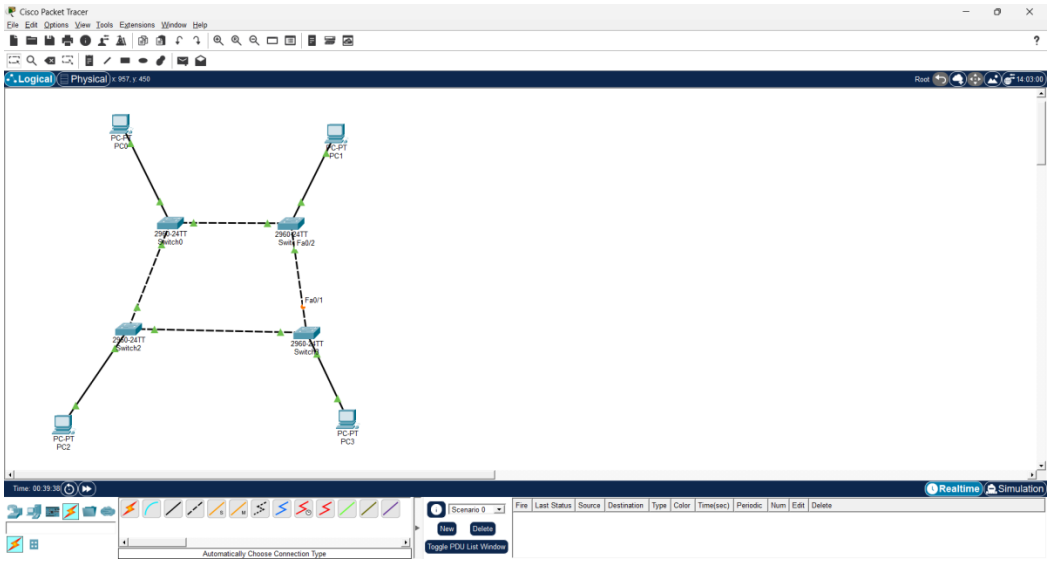


Simulation:

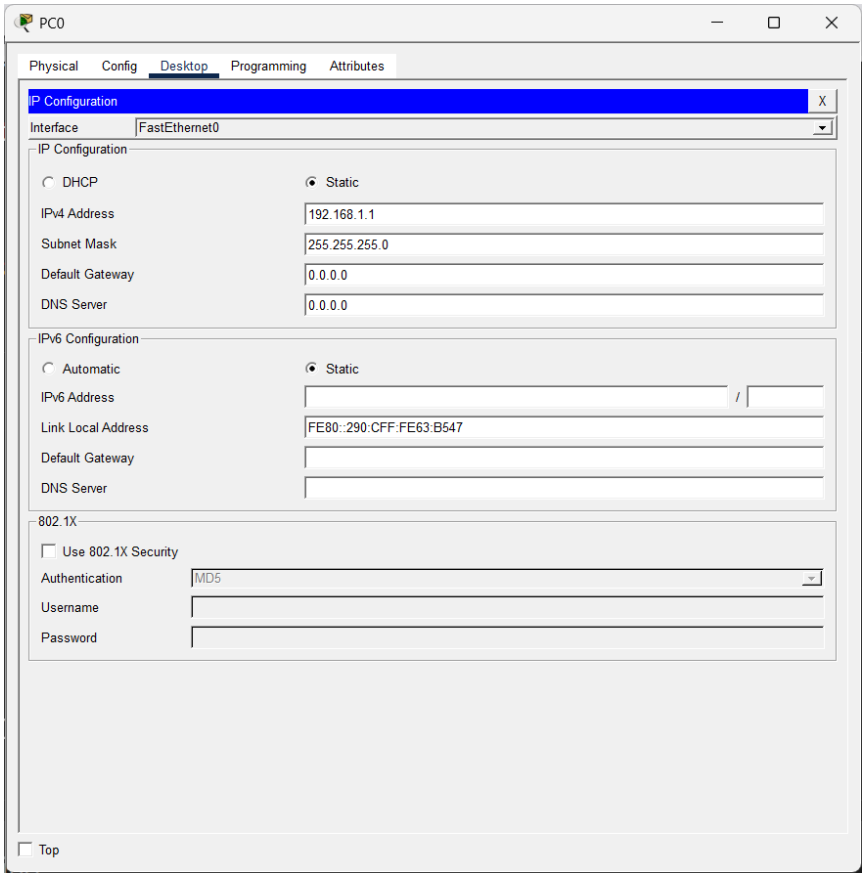


Ring Topology:

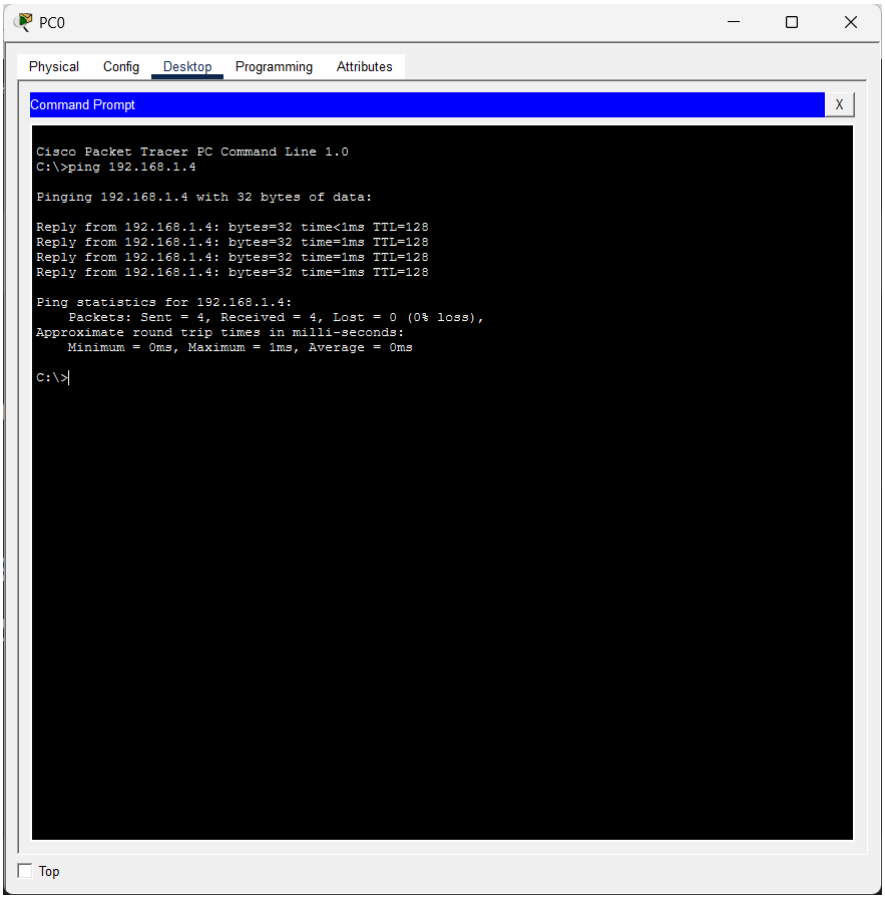
Basic structure:



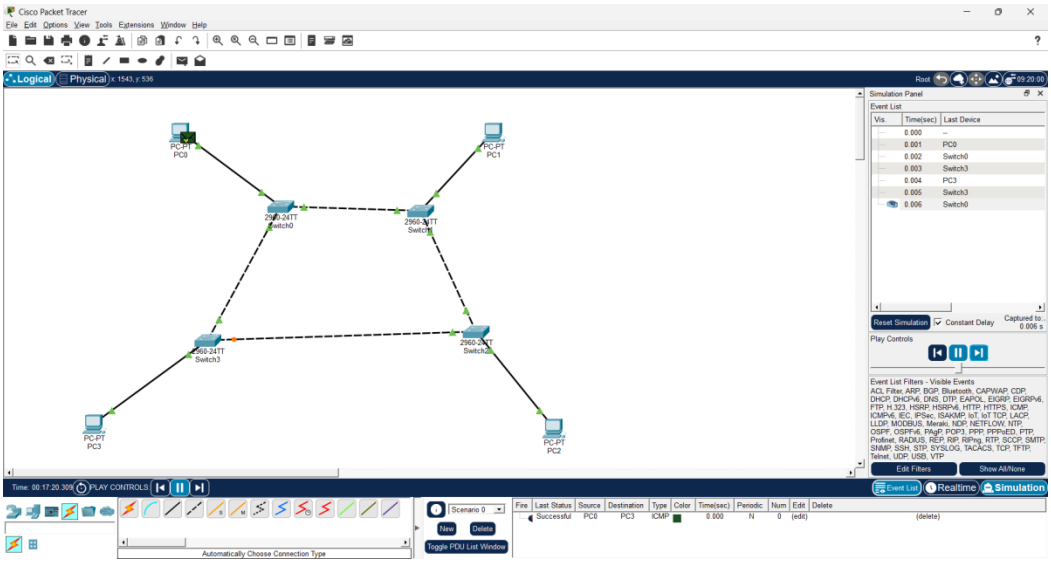
IP Configuration:



Ping Status:

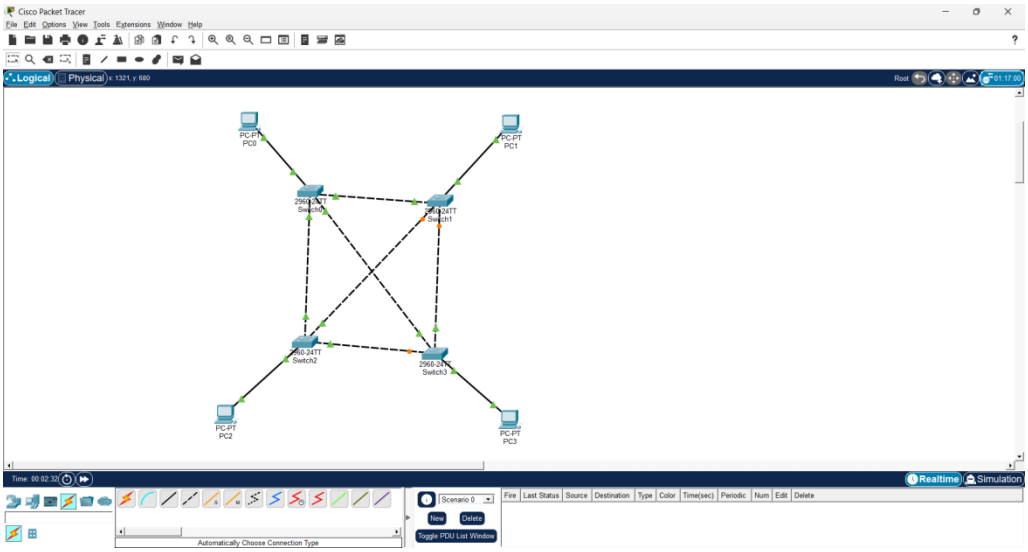


Simulation:

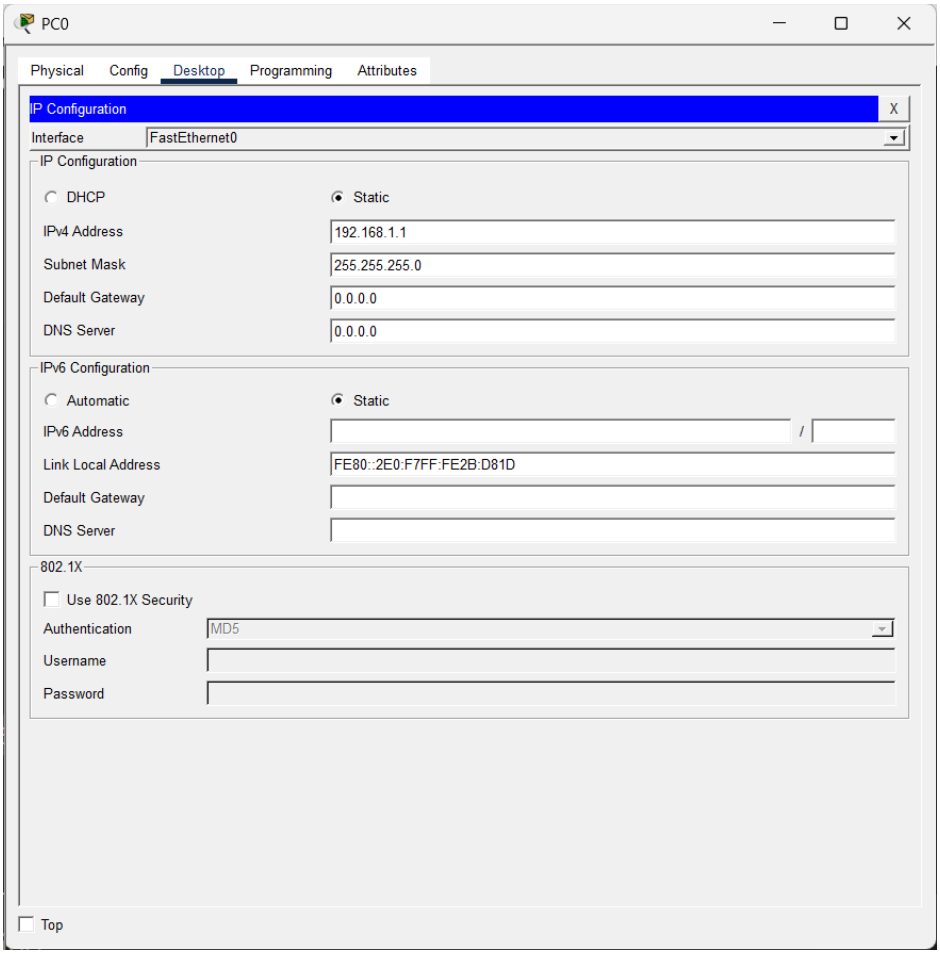


Mesh Topology:

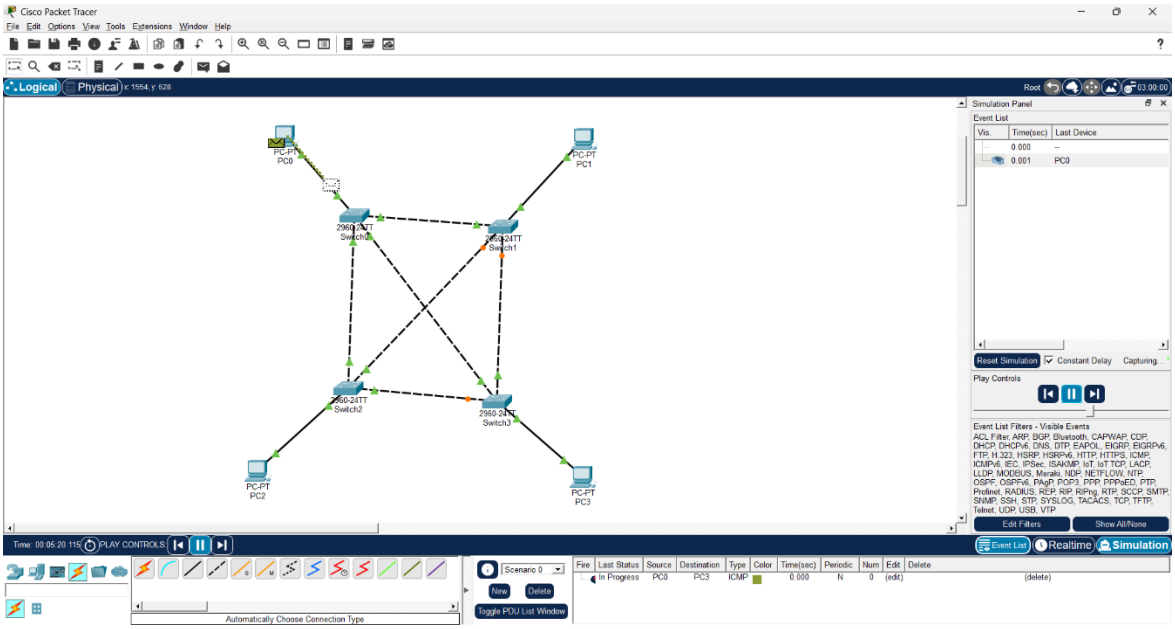
Basic structure:

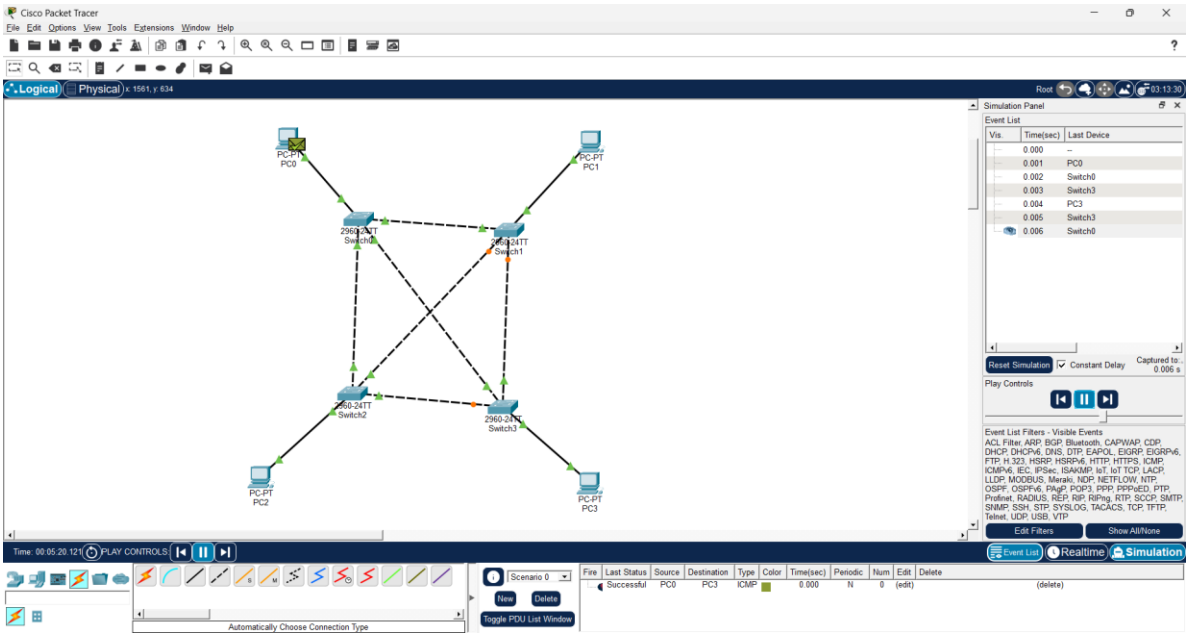


IP Configuration:



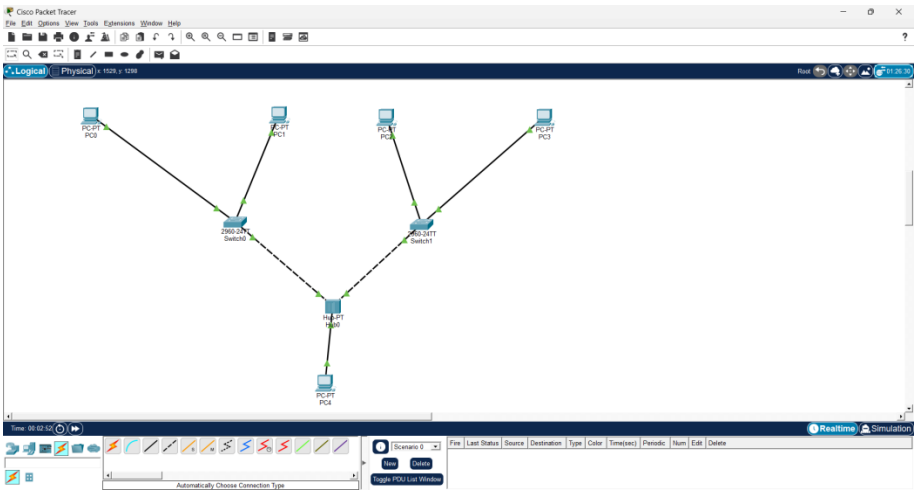
Simulation:


Cisco Packet Tracer
File Edit Options View Tools Extensions Window Help
Logical Physical
PC-PT PC0
PC-PT PC1
2960-24TT Switch0
2960-24TT Switch1
2960-24TT Switch2
2960-24TT Switch3
PC-PT PC2
PC-PT PC3
Simulation Panel
Event List
Vis. Time(sec) Last Device
0.000 --
0.001 PC0
Reset Simulation
Constant Delay
Play Controls
Event List Filters - Visible Events
Edit Filters
Show All/None
Event List
Realtime
Simulation
Fire Last Status Source Destination Type Color Time(sec) Periodic Num Edit Delete
In Progress PC0 PC3 ICMP 0.000 N 0 (edit) (delete)
Scenario 0
New
Delete
Toggle PDU List Window
Automatically Choose Connection Type

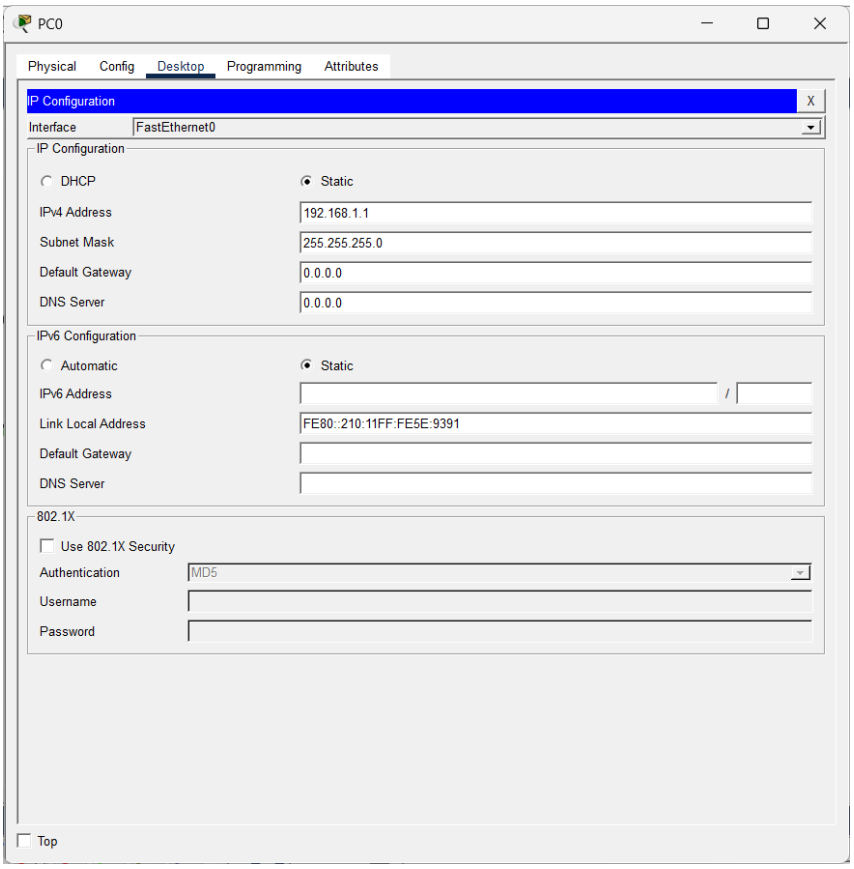

Cisco Packet Tracer
File Edit Options View Tools Extensions Window Help
Logical Physical
PC-PT PC0
PC-PT PC1
2960-24TT Switch0
2960-24TT Switch1
2960-24TT Switch2
2960-24TT Switch3
PC-PT PC2
PC-PT PC3
Simulation Panel
Event List
Vis. Time(sec) Last Device
0.000 --
0.001 PC0
0.002 Switch0
0.003 Switch3
0.004 PC3
0.005 Switch3
0.006 Switch0
Reset Simulation
Constant Delay
Play Controls
Event List Filters - Visible Events
Edit Filters
Show All/None
Event List
Realtime
Simulation
Fire Last Status Source Destination Type Color Time(sec) Periodic Num Edit Delete
Successful PC0 PC3 ICMP 0.000 N 0 (edit) (delete)
Scenario 0
New
Delete
Toggle PDU List Window
Automatically Choose Connection Type

Hybrid Topology:

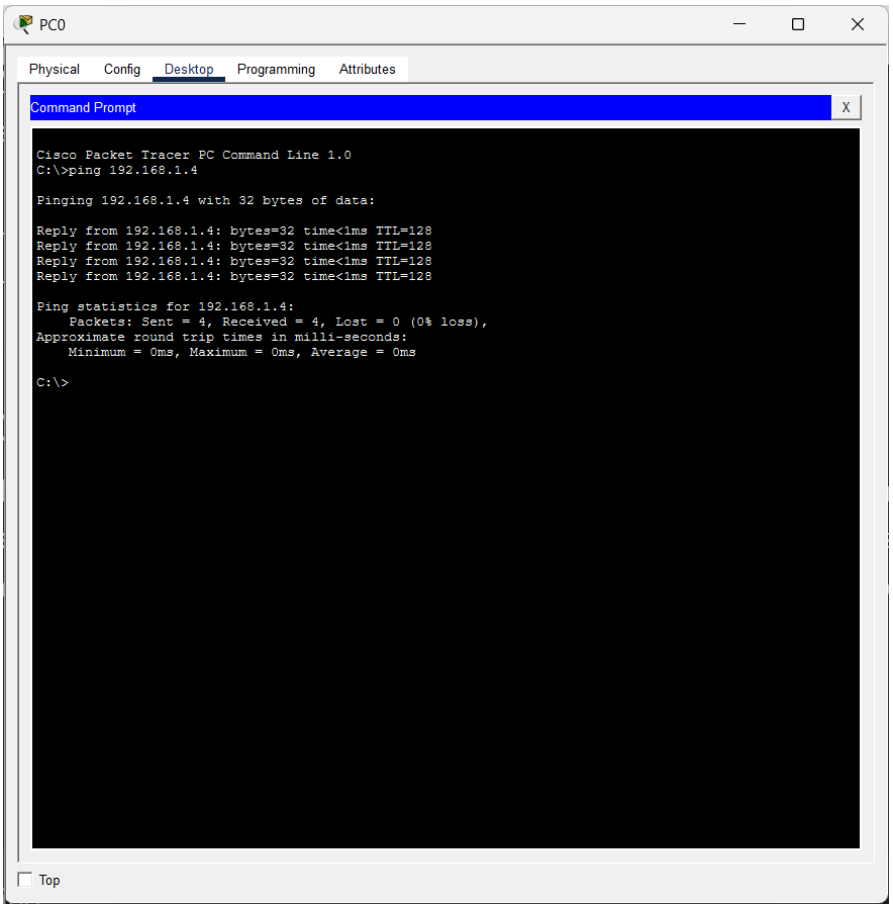
Basic Structure:



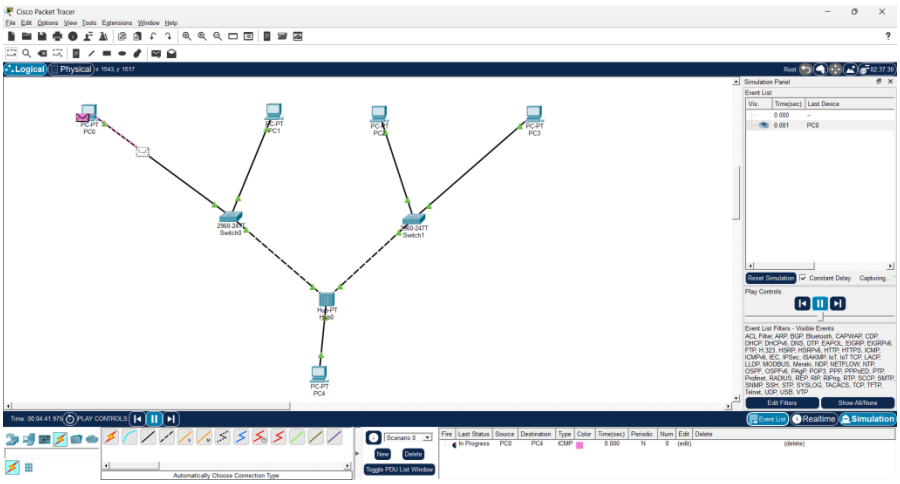
IP configuration:

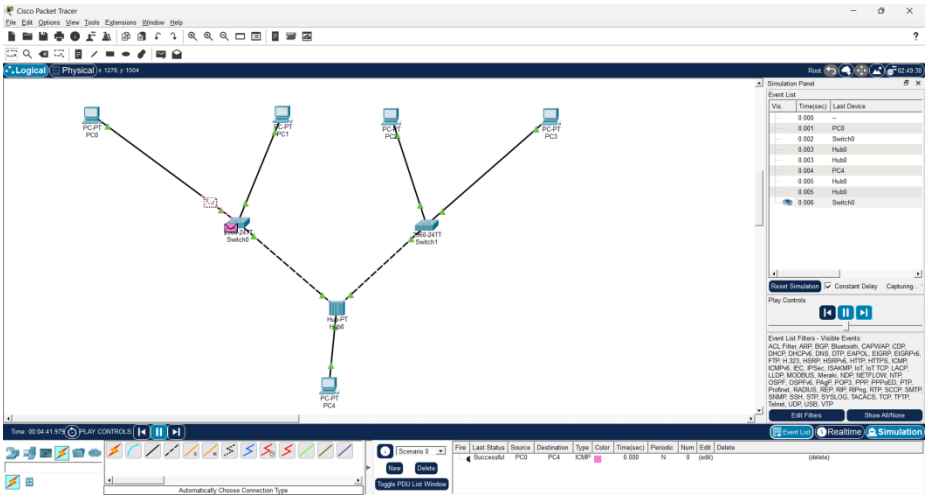


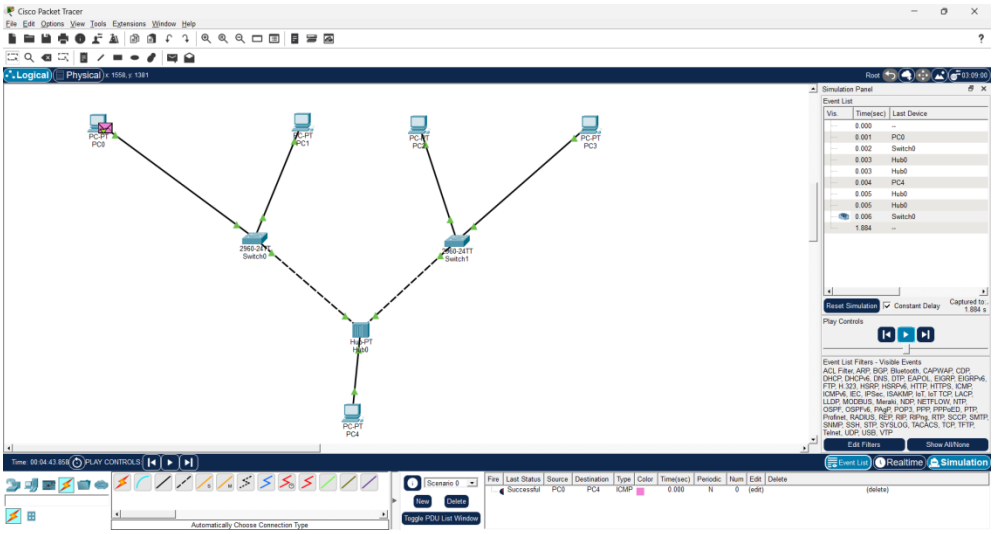
Ping Status:



Simulation:







Result

Successfully configured inter-LAN communication using a router and verified communication between different LANs. The working of repeaters and bridges was also demonstrated successfully using Cisco Packet Tracer, confirming proper signal transmission and network segmentation.

References:

- https://www.netacad.com/courses/packet-tracer
- https://skillsforall.com/course/getting-started-cisco-packet-tracer
- https://www.cloudflare.com/learning/network-layer/what-is-a-lan/

Exp No: 3	Inter-LAN Communication using Router Configuration and Working of Repeaters and Bridges
08.08.2026	

Aim:

- To connect two Local Area Networks (LANs) and enable communication between them using a router.
- To demonstrate the working of repeaters and bridges using Cisco Packet Tracer.

Objectives

- To configure a router for communication between two different LANs.
- To assign IP addresses and default gateways to all network devices.
- To understand the routing process between different networks.
- To study the working principle and applications of repeaters and bridges.
- To verify successful communication using simulation in Cisco Packet Tracer.

Simulation Tool:

- **Cisco Packet Tracer**

Theory:

A **router** is a Layer-3 networking device that connects two or more different networks and forwards data packets between them based on IP addresses. It examines the destination IP address of each packet and uses its routing table to determine the best path for forwarding the packet. Routers enable communication between different LANs and WANs while efficiently managing network traffic.

A **repeater** is a physical layer device that regenerates and amplifies weak or degraded signals before retransmitting them. It is mainly used to extend the communication distance of a network without changing the data being transmitted.

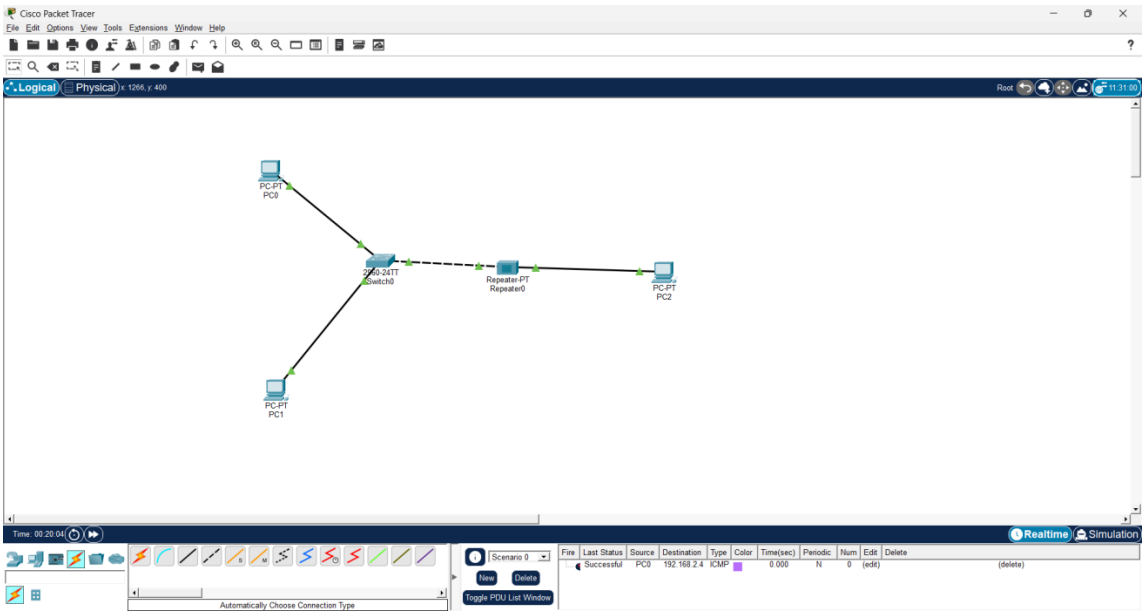
A **bridge** is a data link layer (Layer-2) device that connects two or more LAN segments. It filters and forwards frames using MAC addresses, reducing unnecessary network traffic and improving network performance.

In this practical, two separate LANs are connected through a router to establish inter-LAN communication. The working of repeaters and bridges is also demonstrated to understand their role in extending network coverage and connecting network segments efficiently.

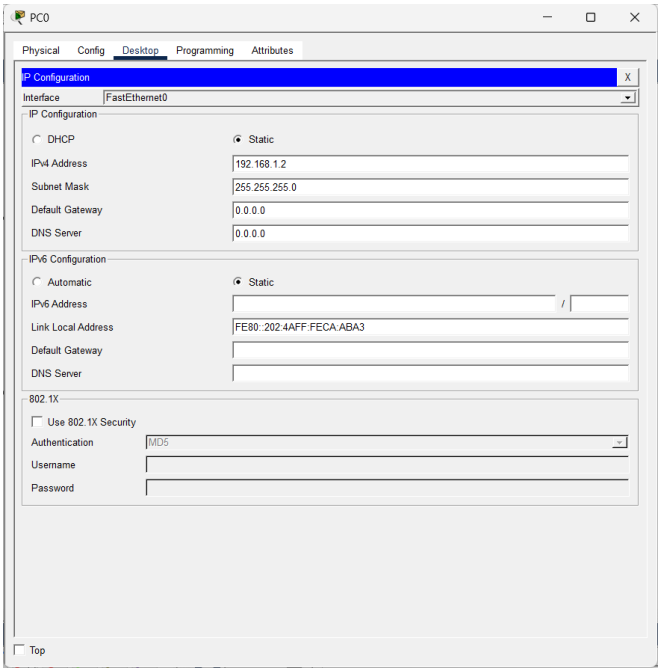
Output:

- **Repeater:**

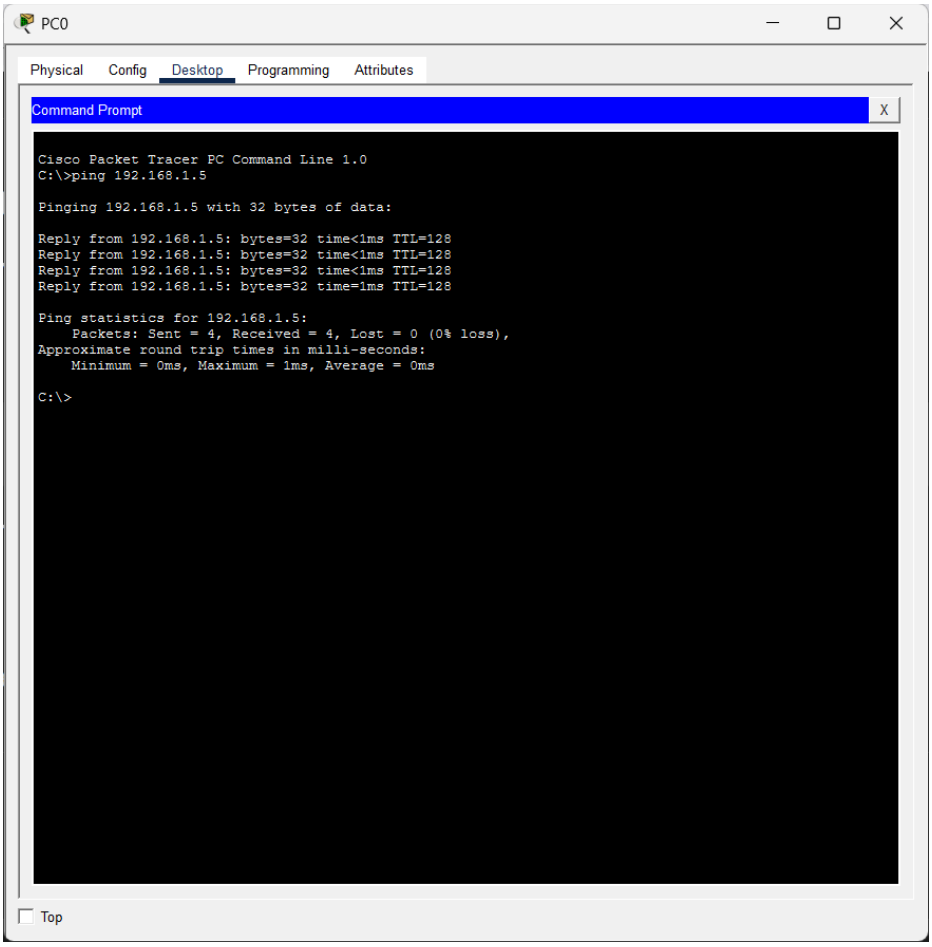
Basic Structure:



IP Configuration:

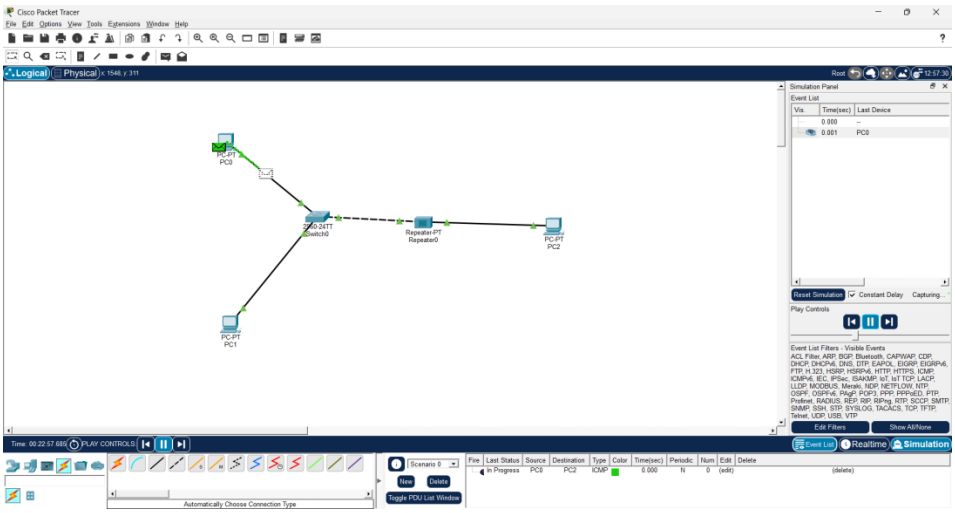


Ping Status :

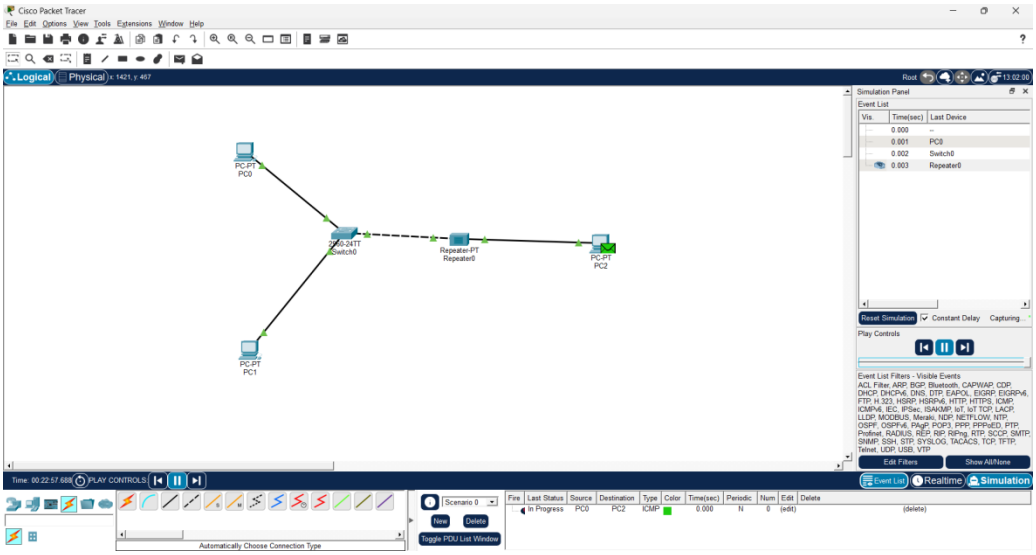


Simulation:

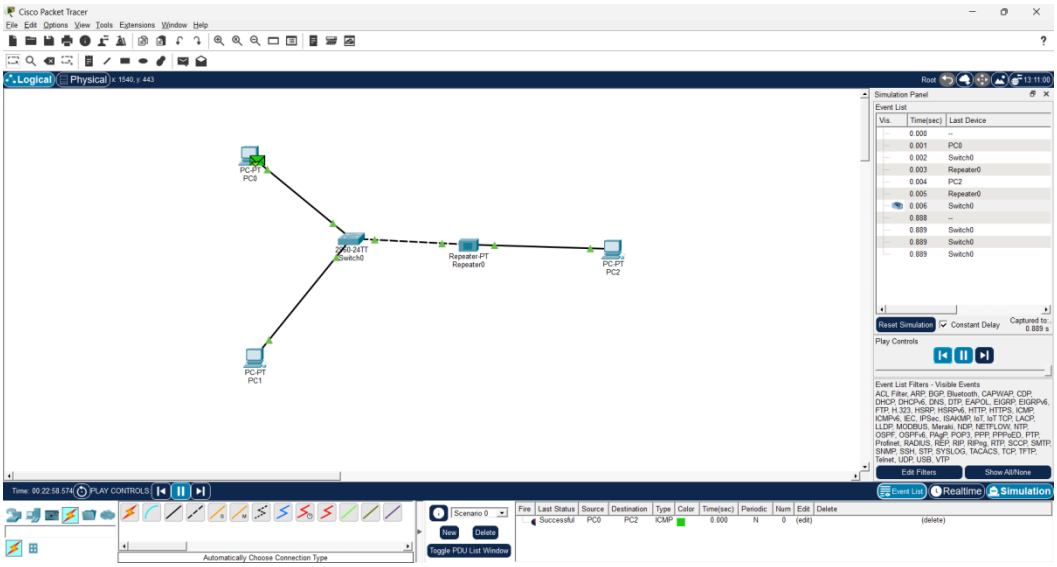
Initial Stage



In progress

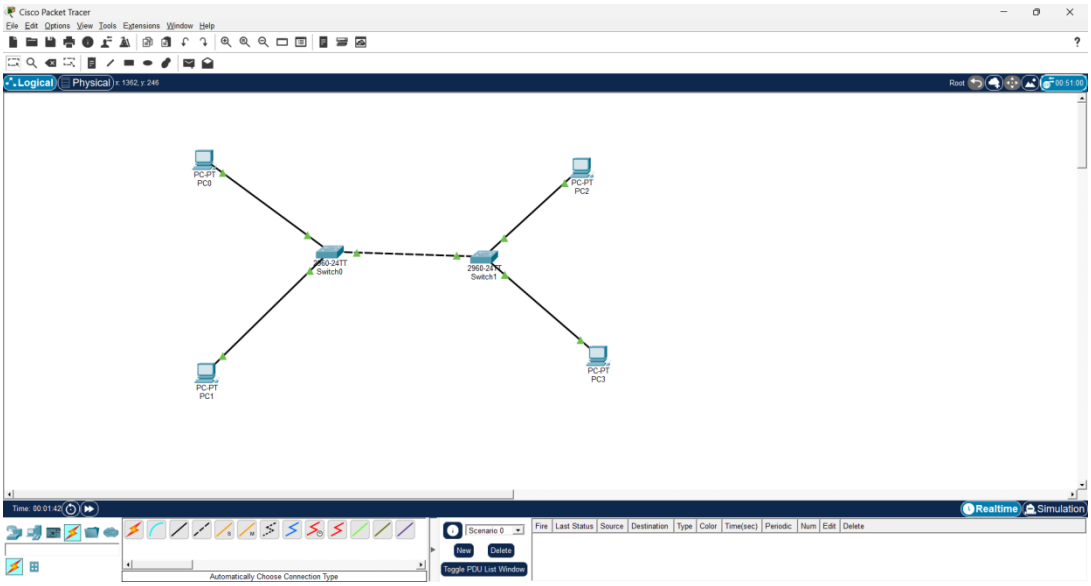


Successful

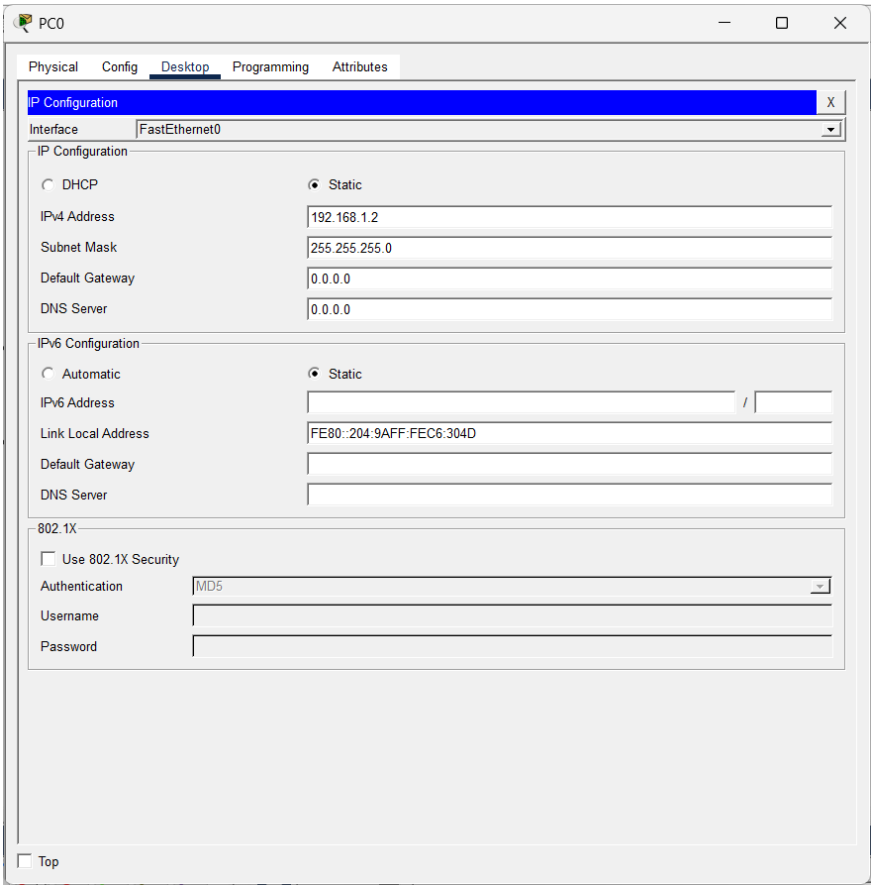


- **Bridge:**

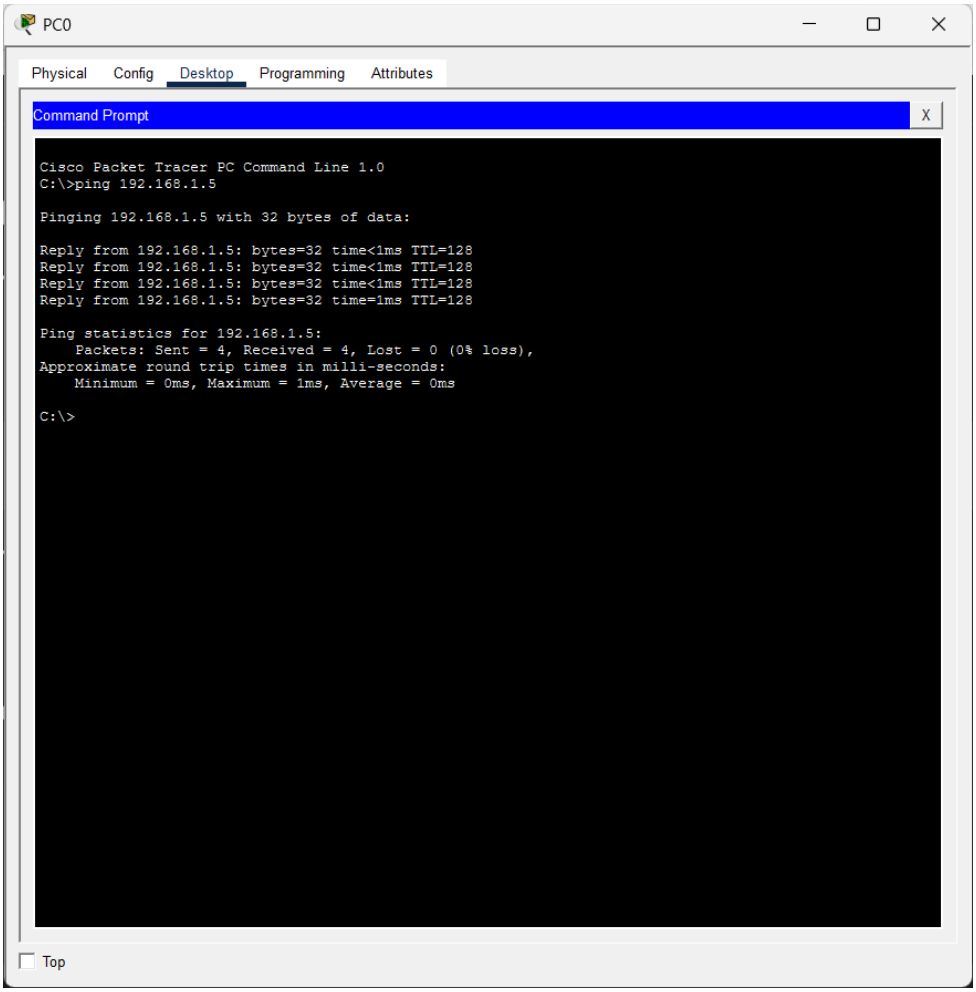
Basic Structure:



IP configuration:

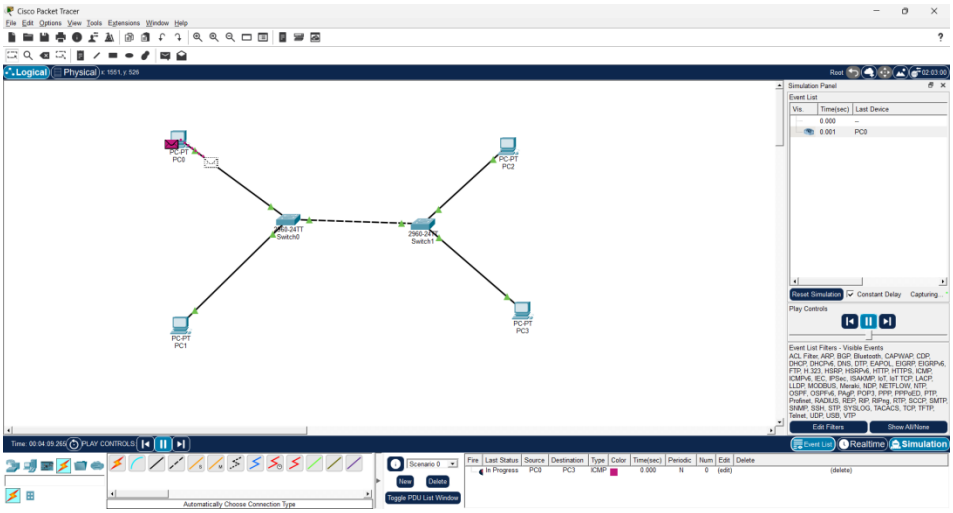


Ping Status:

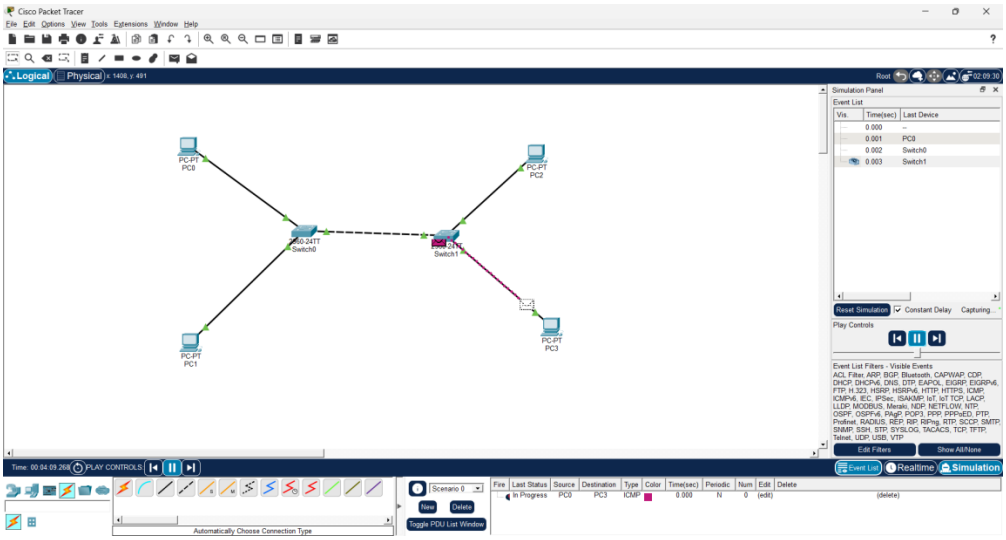

PC0
Physical
Config
Desktop
Programming
Attributes
Command Prompt
X
Cisco Packet Tracer PC Command Line 1.0
C:\>ping 192.168.1.5
Pinging 192.168.1.5 with 32 bytes of data:
Reply from 192.168.1.5: bytes=32 time<1ms TTL=128
Reply from 192.168.1.5: bytes=32 time<1ms TTL=128
Reply from 192.168.1.5: bytes=32 time<1ms TTL=128
Reply from 192.168.1.5: bytes=32 time=1ms TTL=128
Ping statistics for 192.168.1.5:
Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
Minimum = 0ms, Maximum = 1ms, Average = 0ms
C:\>
Top

Simulation:

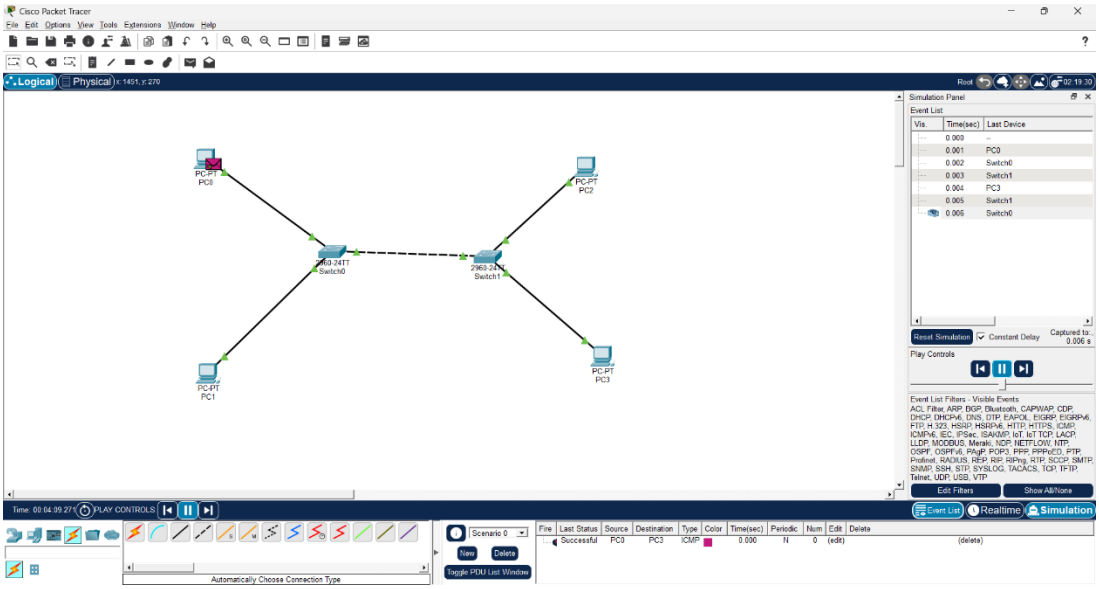
Initial Stage:



In Progress:



Successful:



Result

Successfully configured two LANs and established communication between them using a router in Cisco Packet Tracer. The working of repeaters and bridges was also demonstrated successfully, and packet transmission was verified using Simulation Mode, confirming proper network connectivity and communication.

Exp No: 4	Error Correcting Techniques using Hamming Code and Cyclic Redundancy Check
22.08.2026	

Aim:

- To implement an error detecting and correcting technique using Hamming Code in Python.
- To implement an error detecting technique using Cyclic Redundancy Check (CRC) in Python.

Objective:

- To understand the concept of error detection and correction in data communication.
- To calculate the number of redundant (parity) bits required for a given length of data using Hamming Code.
- To write a Python program that encodes data using Hamming Code, introduces a bit error, and detects and corrects it.
- To write a Python program that implements CRC for error detection using polynomial (modulo-2) division.
- To verify the correctness of both techniques using sample test data.

Simulation Tool:

Python (IDLE / VS Code / Google Colab)

Theory:

Error detection and correction techniques are used at the Data Link Layer to ensure reliable transmission of data over a communication channel that may introduce noise or interference, causing bits to be altered during transmission.

Hamming Code is an error detecting and single-bit error correcting code developed by R. W. Hamming. It works by adding redundant (parity) bits to the original data at positions that are powers of two (1, 2, 4, 8, ...). Each parity bit is computed over a specific subset of the data bits, chosen based on the binary representation of the bit positions it covers. At the receiver, the same parity checks are recomputed; if all checks pass, no error has occurred. If one or more checks fail, the sum of the failed parity positions directly gives the position of the erroneous bit, which can then be flipped to correct it. Hamming Code can detect up to two-bit errors and correct single-bit errors.

Cyclic Redundancy Check (CRC) is an error-detecting code commonly used in digital networks and storage devices to detect accidental changes to raw data. The sender and receiver agree on a fixed generator polynomial (key). The sender appends (n-1) zero bits to the data (where n is the length of the generator) and performs binary (modulo-2, XOR-based) division of this augmented data by the generator polynomial. The remainder obtained is the CRC checksum, which is appended to the original data to form the transmitted codeword. At the receiver's end, the received codeword is divided by the same generator polynomial; if the remainder is zero, the data is assumed to be error free, otherwise an error is detected.

Procedure - Algorithm:

a. Hamming Code (Generation): Determine the number of parity bits r required, such that $2^r >= m + r + 1$, where m is the number of data bits.

- Place the data bits and parity bits into a codeword of length (m + r), with parity bits occupying positions that are powers of two.
- Calculate each parity bit by XOR-ing all bit positions whose binary representation includes that parity bit's position.
- At the receiver, recompute all parity checks on the received codeword.
- Add up the positions of the failed checks to get the position of the error bit (0 indicates no error); flip that bit to correct it.

a. CRC (Generation & Checking): Choose a generator polynomial (key) known to both sender and receiver.

- Append (n-1) zeros to the data, where n is the length of the key.
- Perform modulo-2 (XOR) division of the augmented data by the key; the remainder is the CRC.
- Append the CRC to the original data to form the transmitted codeword.
- At the receiver, divide the received codeword by the same key; a non-zero remainder indicates a transmission error.

Program:

```
# ============================================================
# Practical 4: Error Correcting/Detecting Techniques
# Part A - Hamming Code (Error Detection & Correction)
# Part B - Cyclic Redundancy Check, CRC (Error Detection)
# ============================================================


# ---------------- PART A: HAMMING CODE ----------------

def num_parity_bits(m):
    """Find minimum redundant (parity) bits r needed for m data bits."""
    r = 0
    while (2 ** r) < (m + r + 1):
        r += 1
    return r


def is_power_of_two(n):
    return n != 0 and (n & (n - 1)) == 0


def generate_hamming_code(data):
    """Generate the Hamming codeword for a given data bit string."""
    m = len(data)
    r = num_parity_bits(m)
    total = m + r

    # 1-indexed array; parity positions are powers of 2
    code = [0] * (total + 1)
    j = 0
    for i in range(1, total + 1):
        if not is_power_of_two(i):
            code[i] = int(data[j])
            j += 1

    for i in range(r):
        pos = 2 ** i
        parity = 0
        for k in range(1, total + 1):
            if k & pos:
                parity ^= code[k]
        code[pos] = parity

    return ''.join(str(b) for b in code[1:])
```

```
def detect_and_correct(received):
    """Detect (and correct, if possible) a single-bit error."""
    total = len(received)
    code = [0] + [int(b) for b in received]     # 1-indexed

    r = num_parity_bits(total)   # enough parity positions to cover 'total' bits
    error_pos = 0
    for i in range(r):
        pos = 2 ** i
        if pos > total:
            continue
        parity = 0
        for k in range(1, total + 1):
            if k & pos:
                parity ^= code[k]
        if parity != 0:
            error_pos += pos

    if error_pos:
        print(f"  -> Error detected at bit position {error_pos}")
        code[error_pos] ^= 1
        print("  -> Error corrected.")
    else:
        print("  -> No error detected.")

    return ''.join(str(b) for b in code[1:])


def hamming_demo():
    print("=" * 55)
    print("PART A: HAMMING CODE")
    print("=" * 55)

    data = "1011"                       # 4-bit data to be sent
    codeword = generate_hamming_code(data)
    print(f"Original data           : {data}")
    print(f"Generated Hamming code  : {codeword}")

    # introduce a single-bit error for testing
    error_bit_index = 2                 # 0-indexed position to flip
    received = list(codeword)
    received[error_bit_index] = '1' if received[error_bit_index] == '0' else '0'
    received = ''.join(received)
    print(f"Received code (with error): {received}")

    corrected = detect_and_correct(received)
    print(f"Corrected code          : {corrected}")
    print(f"Correction successful?  : {corrected == codeword}")
```

```
# ---------------- PART B: CYCLIC REDUNDANCY CHECK (CRC) ----------------

def xor(a, b):
    """Bitwise XOR of two equal-length binary strings, dropping the
    leading bit (as used inside long-division)."""
    return ''.join('0' if a[i] == b[i] else '1' for i in range(1, len(b)))


def mod2_divide(dividend, divisor):
    """Perform modulo-2 (XOR) division and return the remainder."""
    pick = len(divisor)
    tmp = dividend[:pick]

    while pick < len(dividend):
        if tmp[0] == '1':
            tmp = xor(divisor, tmp) + dividend[pick]
        else:
            tmp = xor('0' * pick, tmp) + dividend[pick]
        pick += 1

    if tmp[0] == '1':
        tmp = xor(divisor, tmp)
    else:
        tmp = xor('0' * pick, tmp)

    return tmp


def crc_encode(data, key):
    appended = data + '0' * (len(key) - 1)
    remainder = mod2_divide(appended, key)
    return data + remainder


def crc_check(received, key):
    remainder = mod2_divide(received, key)
    return remainder, remainder.count('1') == 0


def crc_demo():
    print()
    print("=" * 55)
    print("PART B: CYCLIC REDUNDANCY CHECK (CRC)")
    print("=" * 55)

    data = "11010011101100"     # message bits
    key = "1011"                # generator polynomial (CRC-3 example)

    codeword = crc_encode(data, key)
    print(f"Original data      : {data}")
```

```
    print(f"Generator (key)    : {key}")
    print(f"Transmitted codeword: {codeword}")

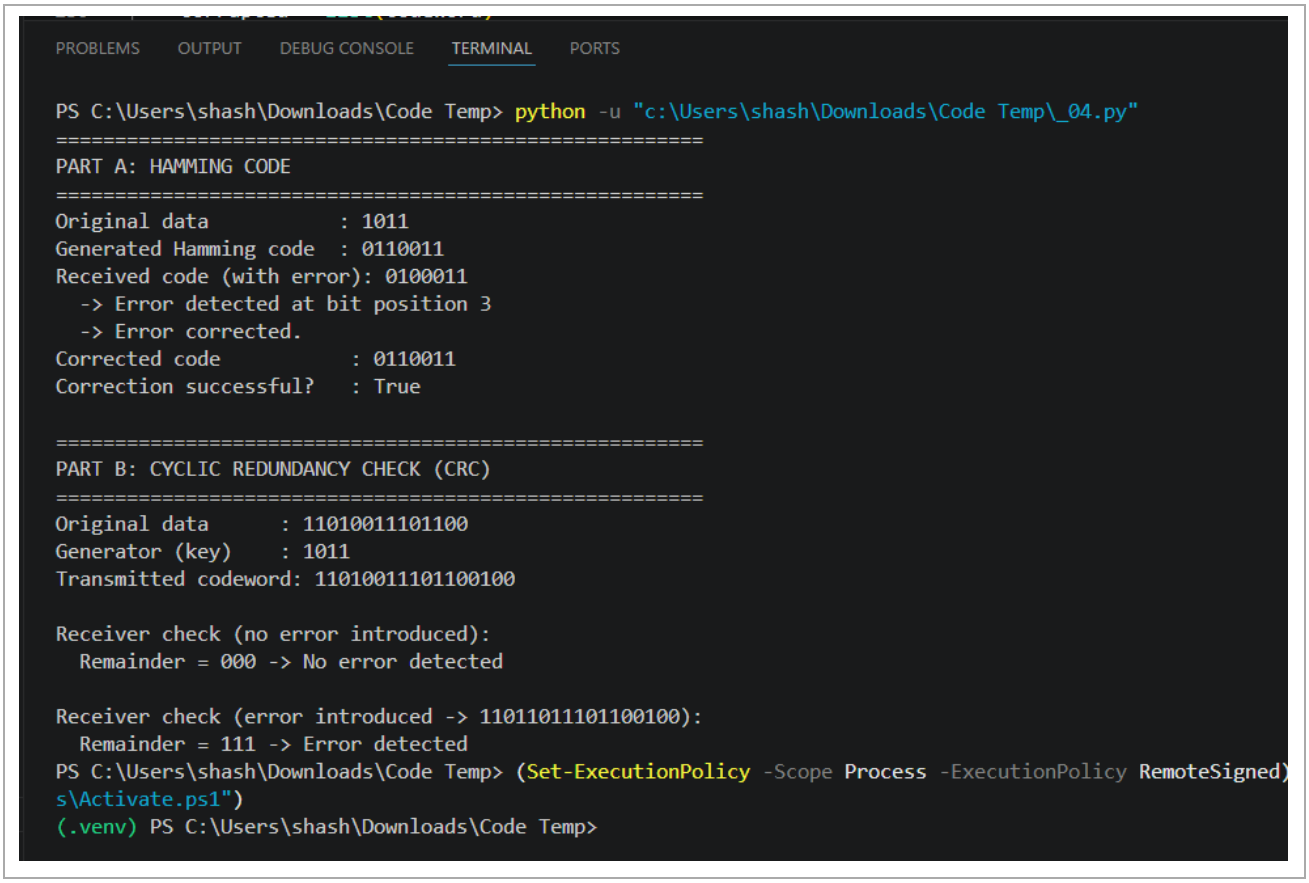
    # Case 1: no error introduced
    remainder, ok = crc_check(codeword, key)
    print(f"\nReceiver check (no error introduced):")
    print(f"  Remainder = {remainder} -> {'No error detected' if ok else 'Error detected'}")

    # Case 2: introduce an error
    corrupted = list(codeword)
    corrupted[4] = '1' if corrupted[4] == '0' else '0'
    corrupted = ''.join(corrupted)
    remainder, ok = crc_check(corrupted, key)
    print(f"\nReceiver check (error introduced -> {corrupted}):")
    print(f"  Remainder = {remainder} -> {'No error detected' if ok else 'Error detected'}")


if __name__ == "__main__":
    hamming_demo()
    crc_demo()
```

Output: (Simulation Screenshots)

Hamming Code & CRC- Output



Result:

- The Hamming Code technique was successfully implemented in Python; a single-bit error introduced in the transmitted codeword was correctly detected and corrected at the receiver.
- The Cyclic Redundancy Check (CRC) technique was successfully implemented in Python; the receiver correctly identified codewords with and without transmission errors.

References:

- https://www.geeksforgeeks.org/hamming-code-in-computer-network/
- https://www.geeksforgeeks.org/computer-networks/error-detection-in-computer-networks/
- https://www.tutorialspoint.com/cyclic-redundancy-check-crc
- https://www.javatpoint.com/hamming-code

Exp No: 5	Implementation of Data Link Layer (DLL) Protocols - Stop and Wait ARQ, Go-Back-N ARQ and Selective Repeat ARQ
24.08.2026	

Aim:

To write and execute a Python program to implement and simulate various Data Link Layer (DLL) error-control protocols - Stop-and-Wait ARQ, Go-Back-N ARQ, and Selective Repeat ARQ - and to understand their mechanisms for achieving reliable data transmission.

Objective:

- To understand the basic principles of Automatic Repeat reQuest (ARQ) protocols used at the Data Link Layer.
- To implement the Stop-and-Wait ARQ protocol using Python.
- To implement the Go-Back-N ARQ protocol using Python.
- To implement the Selective Repeat ARQ protocol using Python.
- To simulate frame transmission, acknowledgements (ACKs), frame loss, and retransmission.

Simulation Tool:

Python (IDLE / VS Code / Google Colab)

Theory:

Automatic Repeat reQuest (ARQ) refers to a set of error-control protocols used at the Data Link Layer to achieve reliable data transmission over an unreliable channel, by using acknowledgements and timeouts to detect and recover from lost or corrupted frames.

Stop-and-Wait ARQ is the simplest form of ARQ, in which the sender transmits a single frame and then waits for an acknowledgement (ACK) from the receiver before sending the next frame. If the ACK is not received within a timeout period (because the frame or the ACK was lost), the sender retransmits the same frame. Although simple, this protocol results in poor channel utilization since only one frame can be in transit at a time.

Go-Back-N ARQ is a sliding-window protocol in which the sender can transmit up to N frames (the window size) without waiting for individual acknowledgements. The receiver only accepts frames in order and sends a cumulative acknowledgement for the last correctly received in-order frame. If a frame is lost or an error occurs, the receiver discards all subsequent out-of-order frames, and the sender must go back

and retransmit all frames starting from the lost one, even if some of them were received correctly.

Selective Repeat ARQ improves upon Go-Back-N by allowing the receiver to accept and buffer out-of-order frames that arrive correctly, and individually acknowledge each frame. Only the specific frame(s) that were lost or corrupted are retransmitted by the sender, rather than the entire window. This makes more efficient use of the available bandwidth, at the cost of additional buffering and bookkeeping complexity at both the sender and the receiver.

Procedure - Algorithm:

1. **Stop-and-Wait ARQ:**
   Sender transmits a frame and starts a timer.
   - If the receiver receives the frame correctly, it sends an ACK.
   - If the sender receives the ACK before the timer expires, it sends the next frame; otherwise, it retransmits the same frame.
   - Repeat until all frames have been successfully delivered and acknowledged.
2. **Go-Back-N ARQ:**
   Sender maintains a window of size N and transmits all frames within the window without waiting for individual ACKs.
   - Receiver accepts frames only in order and sends a cumulative ACK for the last in-order frame received.
   - If a frame is lost, the receiver discards later out-of-order frames until the missing one is received.
   - On timeout, the sender retransmits all frames starting from the lost/unacknowledged frame.
3. **Selective Repeat ARQ:**
   Sender maintains a window of size N and transmits all frames within the window.
   - Receiver accepts and buffers correctly received out-of-order frames and sends an individual ACK for each frame received.
   - If a specific frame is lost, only that frame is retransmitted by the sender after a timeout.
   - The window slides forward as frames are acknowledged, in order.

Program:

```
# ============================================================
# Practical 5: Data Link Layer (DLL) ARQ Protocols
#   1. Stop-and-Wait ARQ
#   2. Go-Back-N ARQ
#   3. Selective Repeat ARQ
# ============================================================

import random

random.seed(7)          # fixed seed so the run is reproducible for the record
FRAME_LOSS_PROB = 0.3    # probability a frame or ACK is lost in transit


def frame_lost():
    return random.random() < FRAME_LOSS_PROB


# ---------------- 1. STOP-AND-WAIT ARQ ----------------

def stop_and_wait(total_frames):
    print("\n" + "=" * 55)
    print("STOP-AND-WAIT ARQ")
    print("=" * 55)

    frame = 0
    while frame < total_frames:
        print(f"Sender  : sending frame {frame}")
        if frame_lost():
            print(f"          !! frame {frame} lost in transit")
            print(f"Sender  : timeout, retransmitting frame {frame}")
            continue

        print(f"Receiver: frame {frame} received correctly")

        if frame_lost():
            print("          !! ACK lost in transit")
            print(f"Sender  : timeout, retransmitting frame {frame}")
            continue

        print(f"Receiver: sending ACK{frame}")
        print(f"Sender  : ACK{frame} received, moving to next frame")
        frame += 1

    print("All frames delivered successfully (Stop-and-Wait).")
```

```
# ---------------- 2. GO-BACK-N ARQ ----------------

def go_back_n(total_frames, window_size=4):
    print("\n" + "=" * 55)
    print(f"GO-BACK-N ARQ  (window size = {window_size})")
    print("=" * 55)

    base = 0
    next_seq = 0
    expected = 0

    while base < total_frames:
        # send every frame currently within the window
        while next_seq < base + window_size and next_seq < total_frames:
            print(f"Sender  : sending frame {next_seq}")
            next_seq += 1

        # receiver processes frames in order, sender waits for cumulative ACK
        while base < next_seq:
            frame = base
            if frame != expected:
                base += 1
                continue

            if frame_lost():
                print(f"          !! frame {frame} lost -> Go-Back-N: "
                      f"resend frames {frame} to {next_seq - 1}")
                next_seq = frame          # roll back and resend the window
                break

            print(f"Receiver: frame {frame} received in order, sending ACK{frame}")
            expected += 1
            base += 1
        else:
            continue
        continue

    print("All frames delivered successfully (Go-Back-N).")
```

```
# ---------------- 3. SELECTIVE REPEAT ARQ ----------------

def selective_repeat(total_frames, window_size=4):
    print("\n" + "=" * 55)
    print(f"SELECTIVE REPEAT ARQ  (window size = {window_size})")
    print("=" * 55)

    acked = [False] * total_frames
    base = 0

    while base < total_frames:
        window = [f for f in range(base, min(base + window_size, total_frames))
                  if not acked[f]]

        for frame in window:
            print(f"Sender  : sending frame {frame}")
            if frame_lost():
                print(f"          !! frame {frame} lost in transit")
                continue

            print(f"Receiver: frame {frame} received, buffering/delivering it")

            if frame_lost():
                print(f"          !! ACK{frame} lost in transit")
                continue

            print(f"Receiver: sending ACK{frame}")
            acked[frame] = True
            print(f"Sender  : ACK{frame} received")

        while base < total_frames and acked[base]:
            base += 1

        if window and not any(acked[f] for f in window):
            print(f"Sender  : no progress this round, retransmitting unacked "
                  f"frames {window}")

    print("All frames delivered successfully (Selective Repeat).")


if __name__ == "__main__":
    NUM_FRAMES = 6

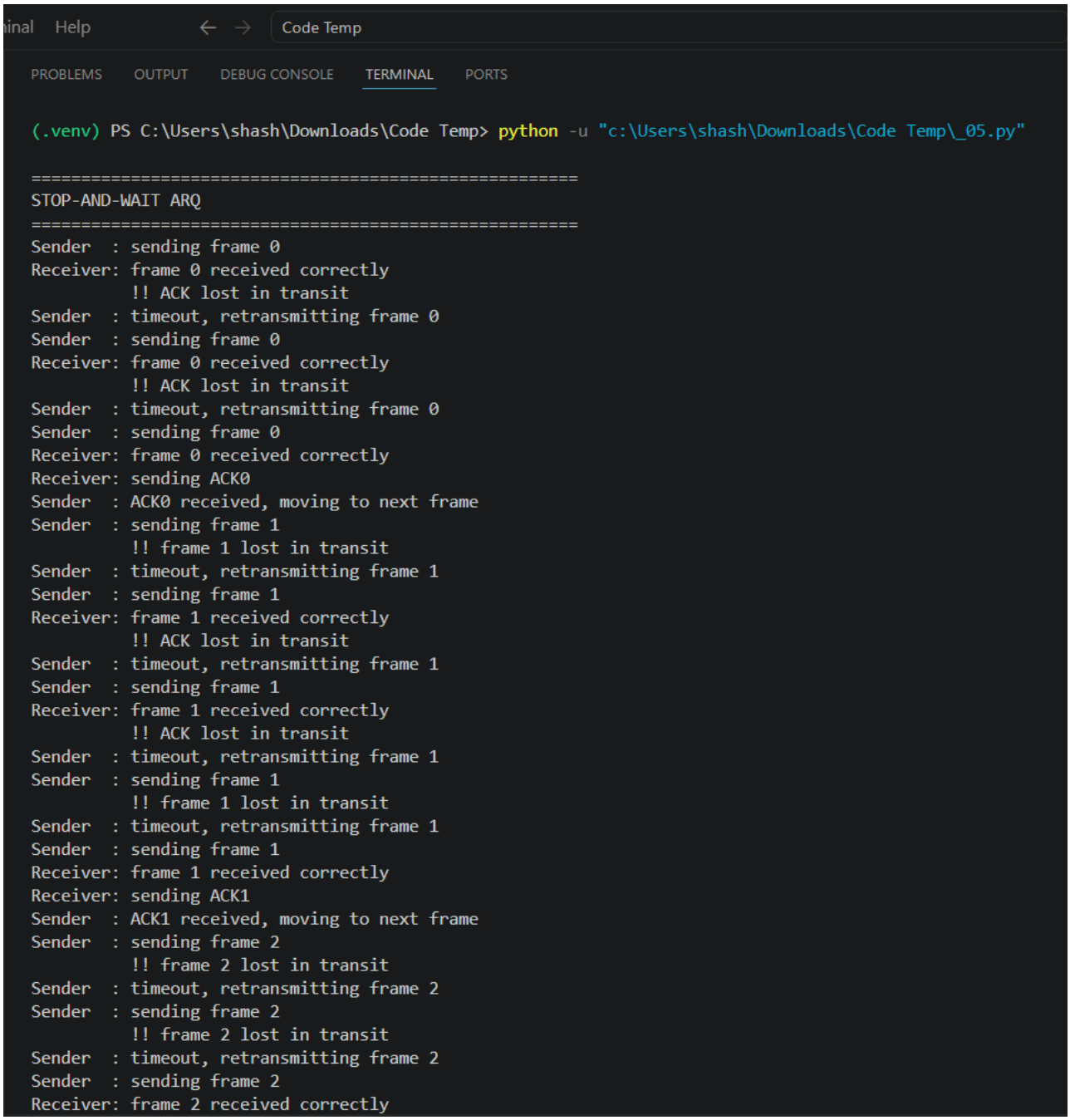
    stop_and_wait(NUM_FRAMES)

    random.seed(11)
    go_back_n(NUM_FRAMES, window_size=4)

    random.seed(3)
    selective_repeat(NUM_FRAMES, window_size=4)
```

Output: (Simulation Screenshots)

Stop-and-Wait ARQ – Output



```
Sender   : ACK1 received, moving to next frame
Sender   : sending frame 2
           !! frame 2 lost in transit
Sender   : timeout, retransmitting frame 2
Sender   : sending frame 2
           !! frame 2 lost in transit
Sender   : timeout, retransmitting frame 2
Sender   : sending frame 2
Receiver: frame 2 received correctly
Receiver: sending ACK2
Sender   : ACK2 received, moving to next frame
Sender   : sending frame 3
Receiver: frame 3 received correctly
Receiver: sending ACK3
Sender   : ACK3 received, moving to next frame
Sender   : sending frame 4
Receiver: frame 4 received correctly
           !! ACK lost in transit
Sender   : timeout, retransmitting frame 4
Sender   : sending frame 4
Receiver: frame 4 received correctly
           !! ACK lost in transit
Sender   : timeout, retransmitting frame 4
Sender   : sending frame 4
           !! frame 4 lost in transit
Sender   : timeout, retransmitting frame 4
Sender   : sending frame 4
           !! frame 4 lost in transit
Sender   : timeout, retransmitting frame 4
Sender   : sending frame 4
Receiver: frame 4 received correctly
Receiver: sending ACK4
Sender   : ACK4 received, moving to next frame
Sender   : sending frame 5
           !! frame 5 lost in transit
Sender   : timeout, retransmitting frame 5
Sender   : sending frame 5
Receiver: frame 5 received correctly
Receiver: sending ACK5
Sender   : ACK5 received, moving to next frame
All frames delivered successfully (Stop-and-Wait).

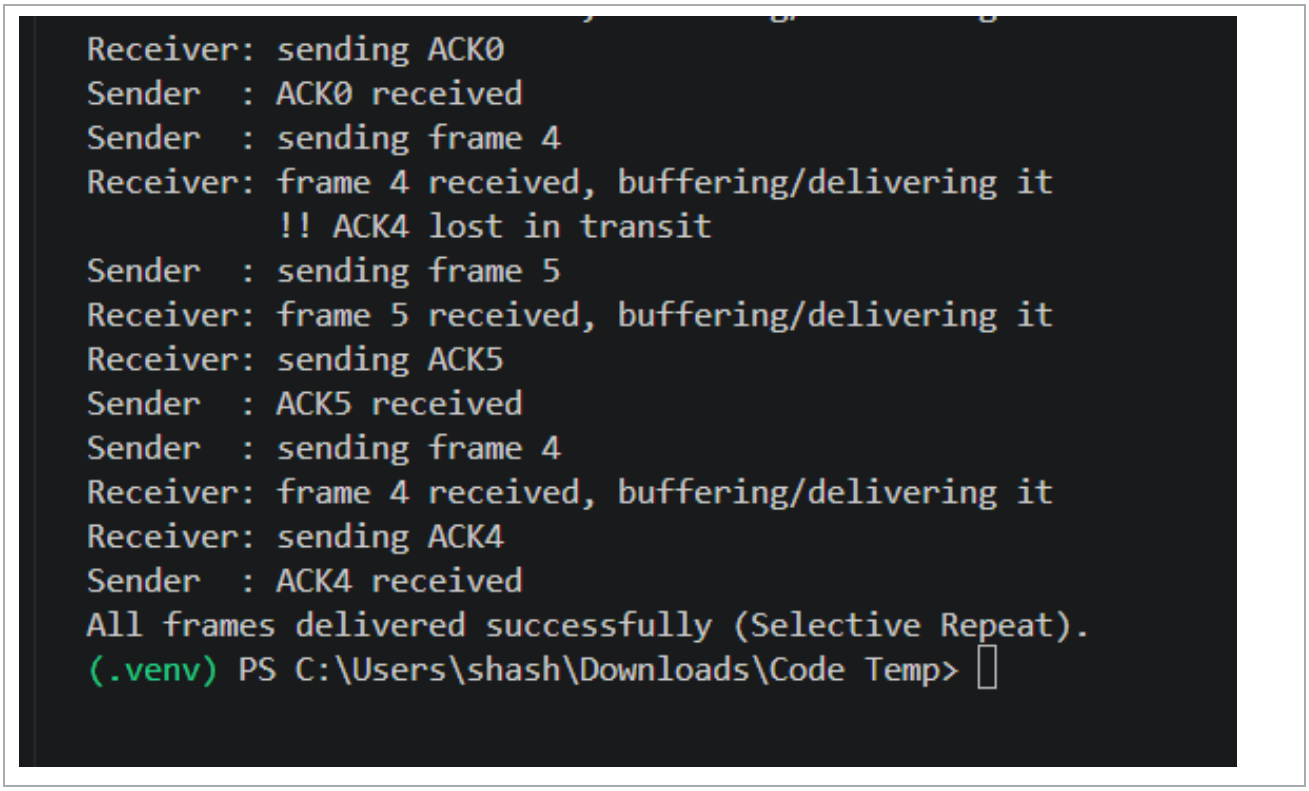
==================================================
```

Go-Back-N ARQ - Output

```
============================================================
GO-BACK-N ARQ  (window size = 4)
============================================================
Sender   : sending frame 0
Sender   : sending frame 1
Sender   : sending frame 2
Sender   : sending frame 3
Receiver: frame 0 received in order, sending ACK0
Receiver: frame 1 received in order, sending ACK1
Receiver: frame 2 received in order, sending ACK2
Receiver: frame 3 received in order, sending ACK3
Sender   : sending frame 4
Sender   : sending frame 5
Receiver: frame 4 received in order, sending ACK4
Receiver: frame 5 received in order, sending ACK5
All frames delivered successfully (Go-Back-N).
```

Selective Repeat ARQ – Output

```
============================================================
SELECTIVE REPEAT ARQ  (window size = 4)
============================================================
Sender   : sending frame 0
           !! frame 0 lost in transit
Sender   : sending frame 1
Receiver: frame 1 received, buffering/delivering it
Receiver: sending ACK1
Sender   : ACK1 received
Sender   : sending frame 2
Receiver: frame 2 received, buffering/delivering it
Receiver: sending ACK2
Sender   : ACK2 received
Sender   : sending frame 3
           !! frame 3 lost in transit
Sender   : sending frame 0
           !! frame 0 lost in transit
Sender   : sending frame 3
Receiver: frame 3 received, buffering/delivering it
           !! ACK3 lost in transit
Sender   : no progress this round, retransmitting unacked frames [0, 3]
Sender   : sending frame 0
           !! frame 0 lost in transit
Sender   : sending frame 3
Receiver: frame 3 received, buffering/delivering it
Receiver: sending ACK3
Sender   : ACK3 received
Sender   : sending frame 0
Receiver: frame 0 received, buffering/delivering it
Receiver: sending ACK0
Sender   : ACK0 received
Sender   : sending frame 4
Receiver: frame 4 received, buffering/delivering it
           !! ACK4 lost in transit
Sender   : sending frame 5
Receiver: frame 5 received, buffering/delivering it
Receiver: sending ACK5
Sender   : ACK5 received
Sender   : sending frame 4
Receiver: frame 4 received, buffering/delivering it
Receiver: sending ACK4
```



Result:

The Stop-and-Wait, Go-Back-N, and Selective Repeat ARQ protocols were successfully implemented and simulated in Python, demonstrating frame transmission, acknowledgement, frame loss, and retransmission behaviour for each protocol.

References:

- https://www.geeksforgeeks.org/computer-networks/stop-and-wait-arq/
- https://www.geeksforgeeks.org/computer-networks/sliding-window-protocol-set-2-receiver-side/
- https://www.tutorialspoint.com/what-is-selective-repeat-protocol
- https://www.javatpoint.com/computer-network-sliding-window-protocol

Exp No: 6	Basic IPv6 Addressing with Cisco Packet Tracer
05-09-2026	

Aim:

To demonstrate IPv6 Addressing using Cisco Packet Tracer.

Objective:

Configuring a router in order to enable communication between two IPv6 networks.

Simulation Tool:

Cisco Packet Tracer

Theory:

IPv6 (Internet Protocol version 6) is the most recent version of the Internet Protocol, designed to succeed IPv4. The most notable difference is its address space: IPv6 uses 128-bit addresses (written as eight groups of four hexadecimal digits, separated by colons, e.g., 2001:0db8:85a3:0000:0000:8a2e:0370:7334) compared to IPv4's 32-bit addresses. This allows for a practically inexhaustible number of unique addresses.

A Link-Local address is automatically assigned to every IPv6-enabled interface. These addresses (starting with FE80::/10) are used for communication within the local network segment (link) and are not routable beyond that link. They are essential for neighbour discovery and router discovery.

IPv6 routers must have IPv6 unicast routing enabled via the ipv6 unicast-routing command, and each interface must be assigned a global unicast address as well as a link-local address. Hosts can auto-configure their own addresses using Stateless Address Autoconfiguration (SLAAC) by combining the router-advertised prefix with their own interface identifier.

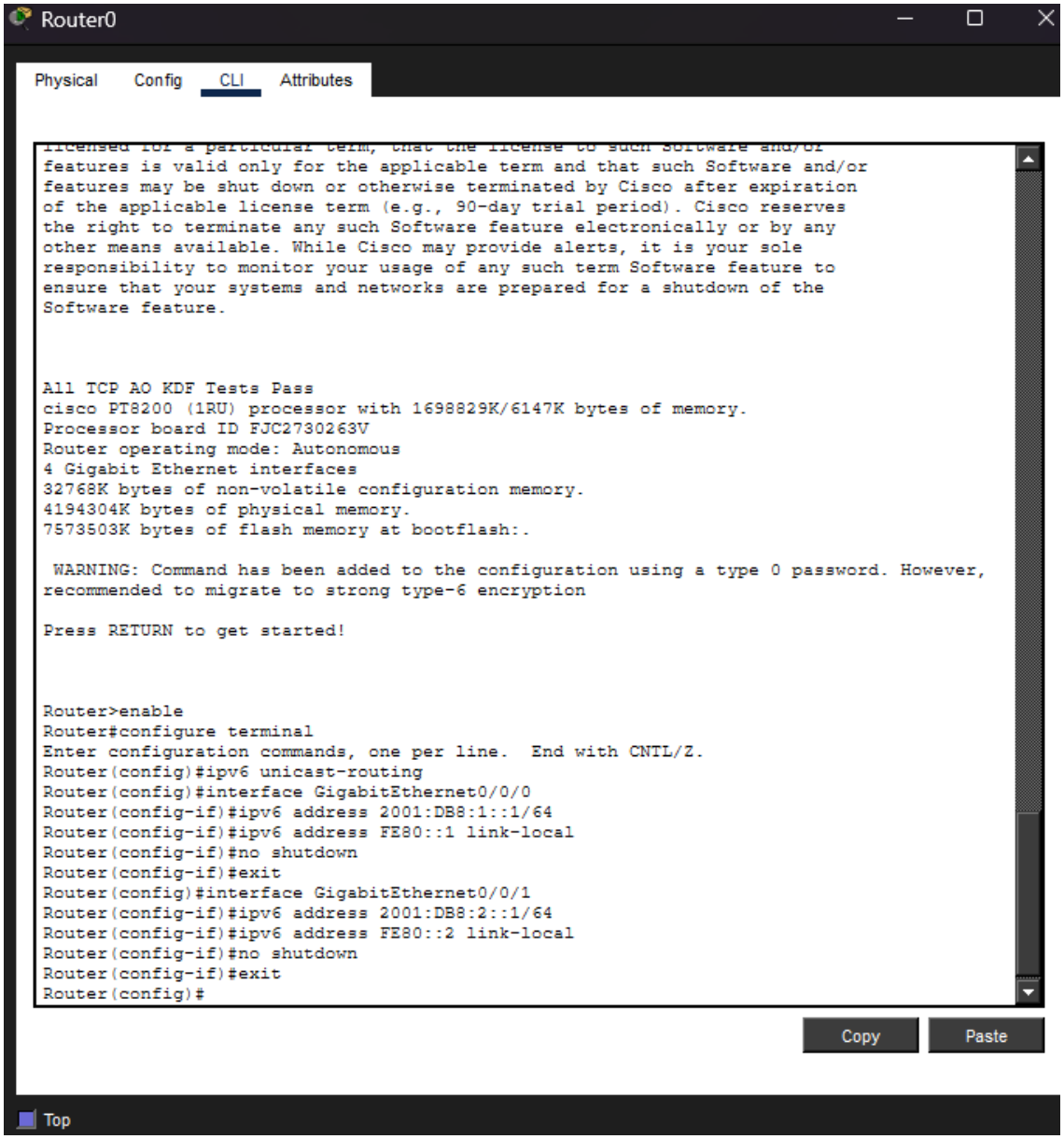
Procedure:

1. Start up Cisco Packet Tracer, and place a couple of end devices of your choice.
2. Form a LAN by connecting the devices with a switch; repeat to form a second LAN, then connect them both with a router.
3. Open the Command Line Interface (CLI) for the router and enable IPv6 addressing using: ipv6 unicast-routing
4. Set an IPv6 address for each router port and a separate link-local address using the ipv6 address command.

5. Configure an IP for each end device; use the auto-configuration option to let the device automatically determine the address by communicating with the router.
6. Test whether the connection works by pinging a device from one LAN to another using its IPv6 address.

Output: (Simulation Screenshots)

Router Configuration -



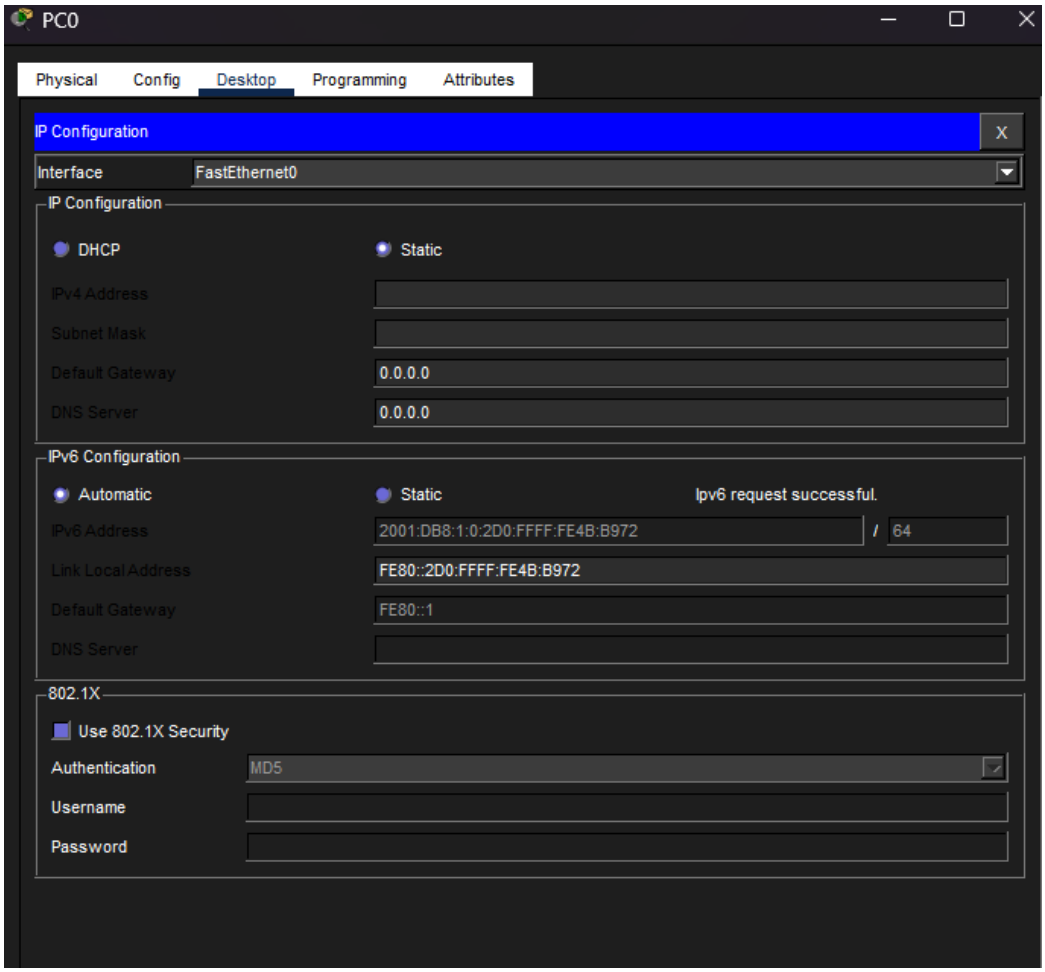
Router configuration finished -

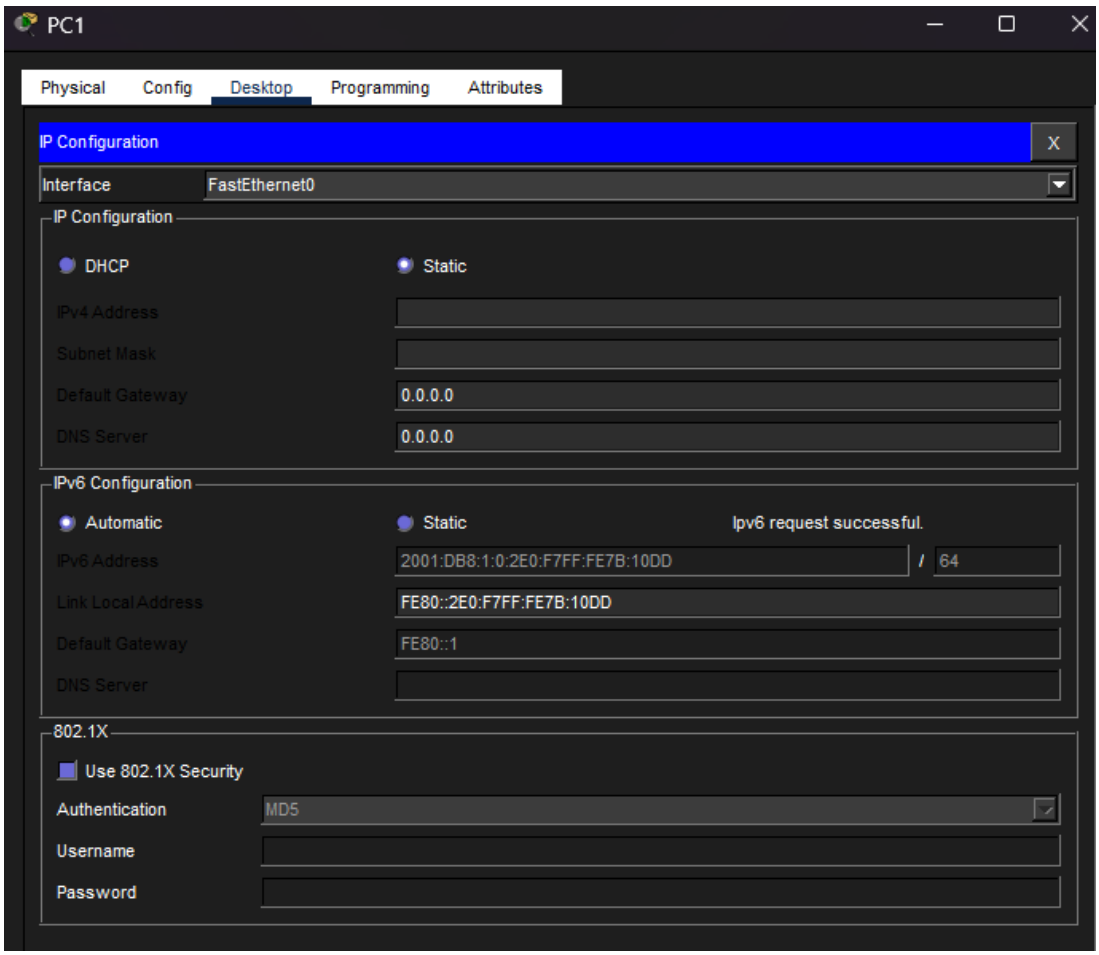
```
Router(config)#end
Router#
*Sep 18, 13:39:13.3939: SYS-5-CONFIG_I: Configured from console by console
```

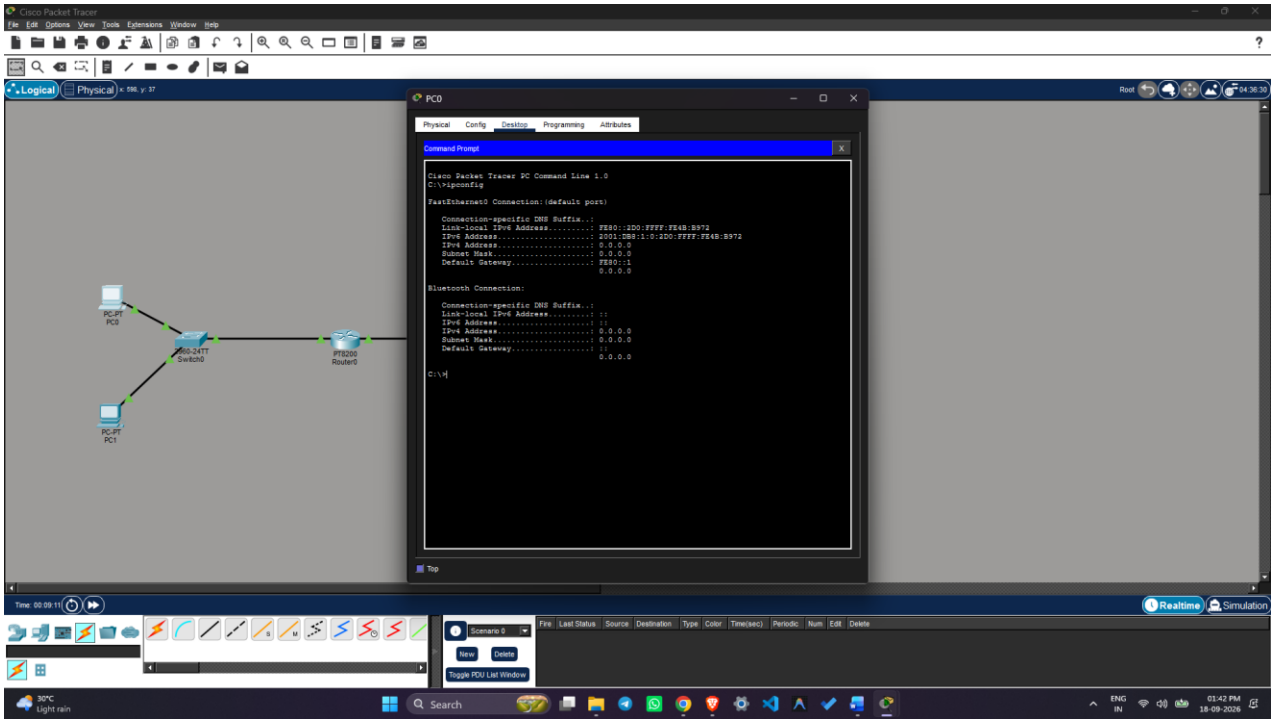
Router Status -

```
Router#
Router#
Router#
Router#
Router#
Router#show ipv6 interface brief
GigabitEthernet0/0/0       [up/up]
    FE80::1
    2001:DB8:1::1
GigabitEthernet0/0/1       [up/up]
    FE80::2
    2001:DB8:2::1
GigabitEthernet0/0/2       [up/down]
    unassigned
GigabitEthernet0/0/3       [up/down]
    unassigned
Vlan1                      [up/down]
    unassigned
Router#
```

PC IPv6 - Setup




PC1
Physical
Config
Desktop
Programming
Attributes
IP Configuration
X
Interface
FastEthernet0
IP Configuration
DHCP
Static
IPv4 Address
Subnet Mask
Default Gateway
0.0.0.0
DNS Server
0.0.0.0
IPv6 Configuration
Automatic
Static
Ipv6 request successful.
IPv6 Address
2001:DB8:1:0:2E0:F7FF:FE7B:10DD
/
64
Link Local Address
FE80::2E0:F7FF:FE7B:10DD
Default Gateway
FE80::1
DNS Server
802.1X
Use 802.1X Security
Authentication
MD5
Username
Password


Cisco Packet Tracer
Logical
Physical
PC0
Command Prompt
Cisco Packet Tracer PC Command Line 1.0
C:\>ipconfig
FastEthernet0 Connection:(default port)
Bluetooth Connection:
C:\>
Realtime
Simulation

Result:

IPv6 addressing was successfully demonstrated using Cisco Packet Tracer. Two separate LANs were configured with IPv6 addresses, the router was enabled for IPv6 unicast routing, and successful communication between the two LANs was verified using the ping command.

References:

- https://www.netacad.com/courses/packet-tracer
- https://skillsforall.com/course/getting-started-cisco-packet-tracer
- https://www.cisco.com/c/en/us/support/docs/ip/ip-version-6-ipv6/113328-ipv6-config-guide.html
- https://www.geeksforgeeks.org/internet-protocol-version-6-ipv6/

Exp No: 7	Implementation of ARP using Cisco Packet Tracer
05.09.2026	

Aim:

To demonstrate ARP (Address Resolution Protocol) using Cisco Packet Tracer.

Objective:

Configuring a setup involving switches, end devices and a server in order to demonstrate the working of the Address Resolution Protocol.

Simulation Tool:

Cisco Packet Tracer

Theory:

ARP (Address Resolution Protocol) is a communication protocol used for discovering the link-layer address (MAC address) associated with a given internet layer address (IP address). ARP operates within the Data Link Layer and Network Layer boundary and is fundamental to communication in an Ethernet LAN.

When a host wants to communicate with another host on the same local network, it needs the destination's MAC address. It first checks its own ARP cache (a table mapping IP addresses to MAC addresses). If the mapping is not found, the host broadcasts an ARP Request packet to all devices on the network, asking "Who has IP address X? Tell me your MAC address." The host with that IP address replies with an ARP Reply containing its MAC address. This mapping is then cached by the requester for future use.

The command 'arp -a' displays the current ARP table (cache) of a device, showing the known IP-to-MAC address mappings. Initially, if no communication has taken place, the table is empty. After pinging another device, an entry is added to the ARP table for that device's IP and MAC address pair.

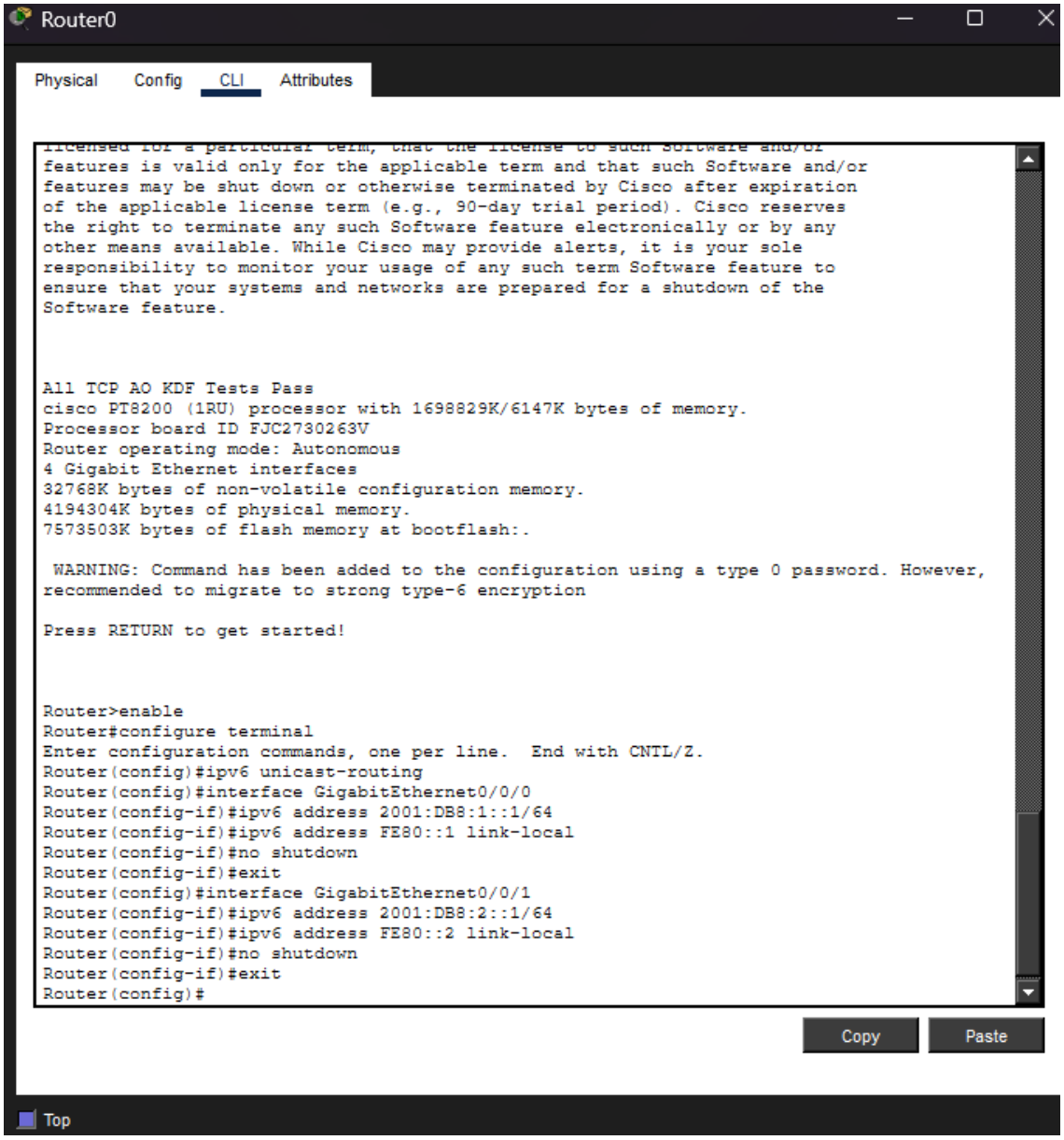
Procedure:

1. Start up Cisco Packet Tracer and place a few end devices of your choice.
2. Connect all of them using a switch to form a LAN.
3. For each device: double-click the device, go to the Desktop tab, click on IP Configuration and set an IP address.
4. Make sure to give them all similar IPs — if an address is p.q.r.s, keep the 'p', 'q', and 'r' values equal for all end devices.

5. Double-click any two of the devices, go to the Command Prompt, and type 'arp -a'. Observe the ARP table — it will show 'no entries found' initially.
6. From one device, ping another device in the LAN.
7. Now, ping the first device from that other device's Command Prompt.
8. Check the ARP tables of both devices again using 'arp -a' and observe the newly populated entries.

Output: (Simulation Screenshots)

Router Configuration -



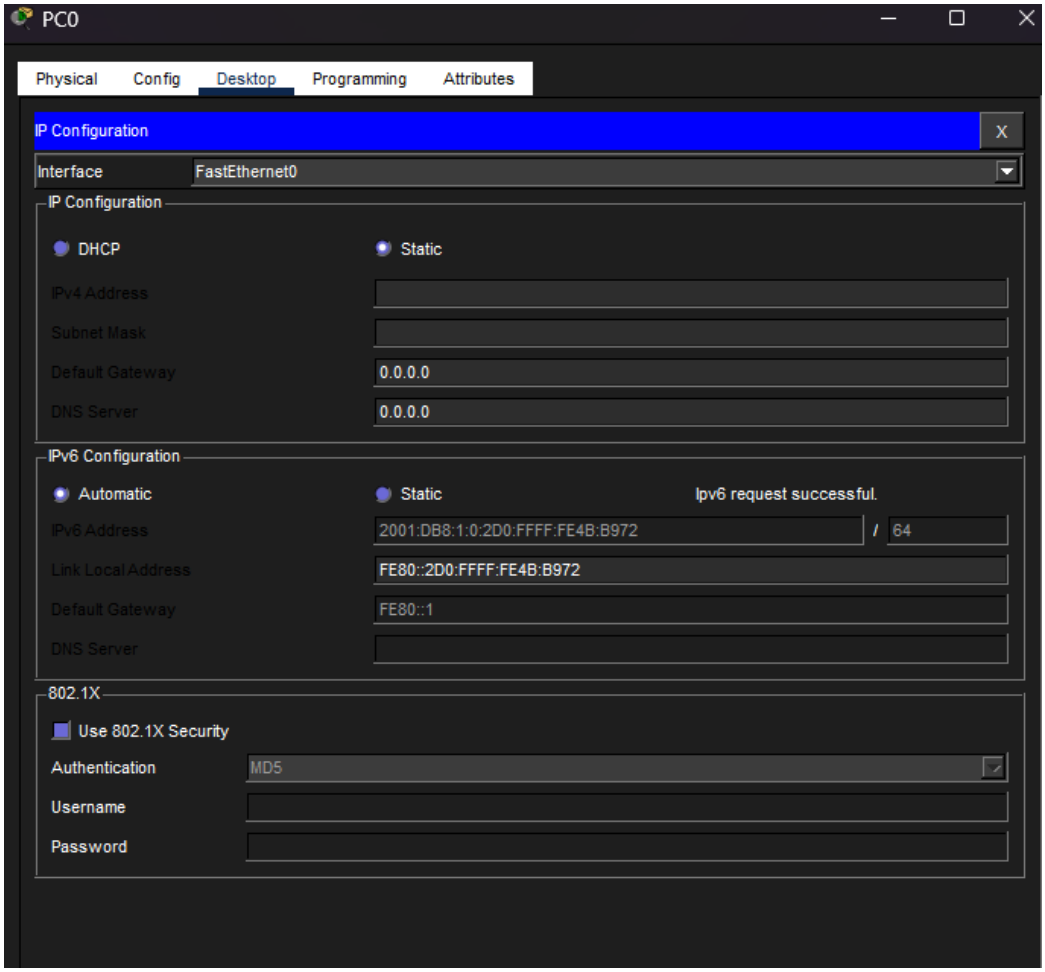
Router configuration finished -

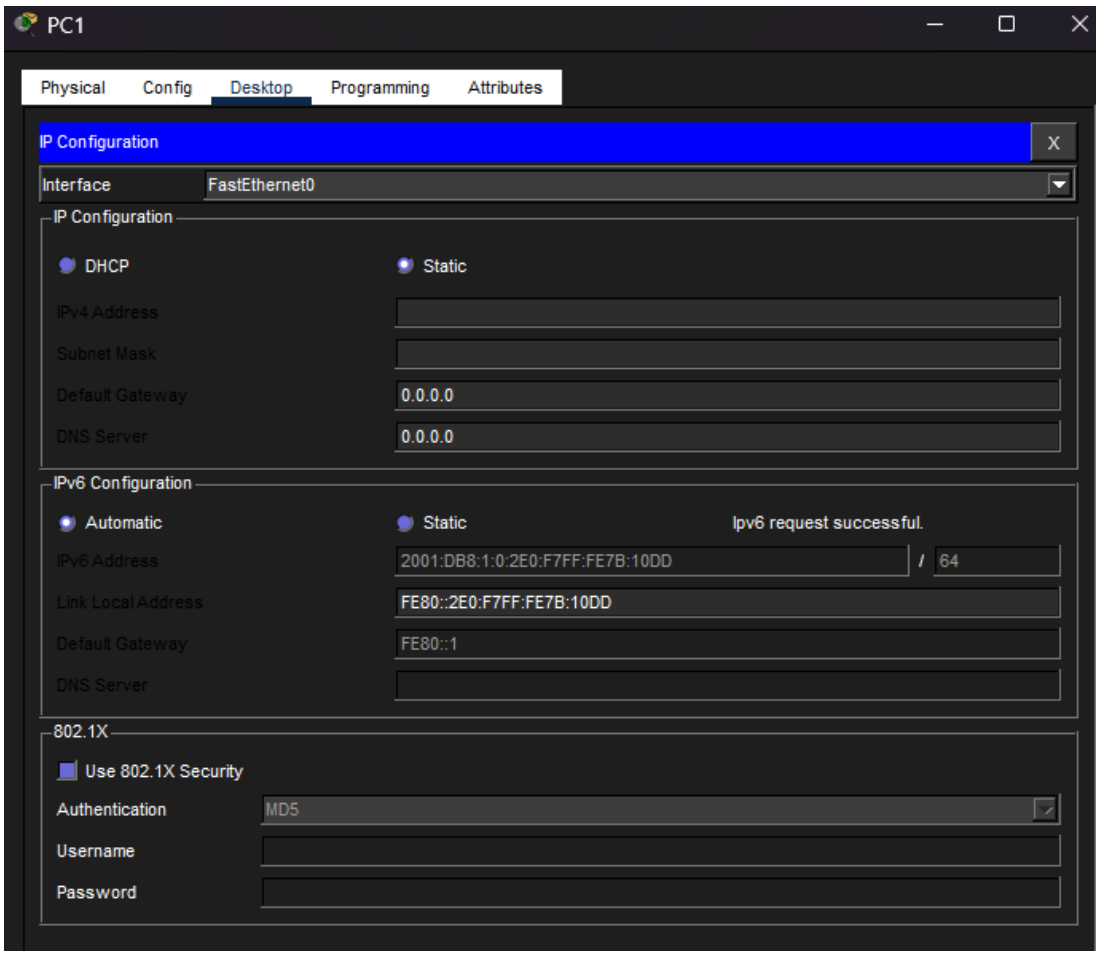
```
Router(config)#end
Router#
*Sep 18, 13:39:13.3939: SYS-5-CONFIG_I: Configured from console by console
```

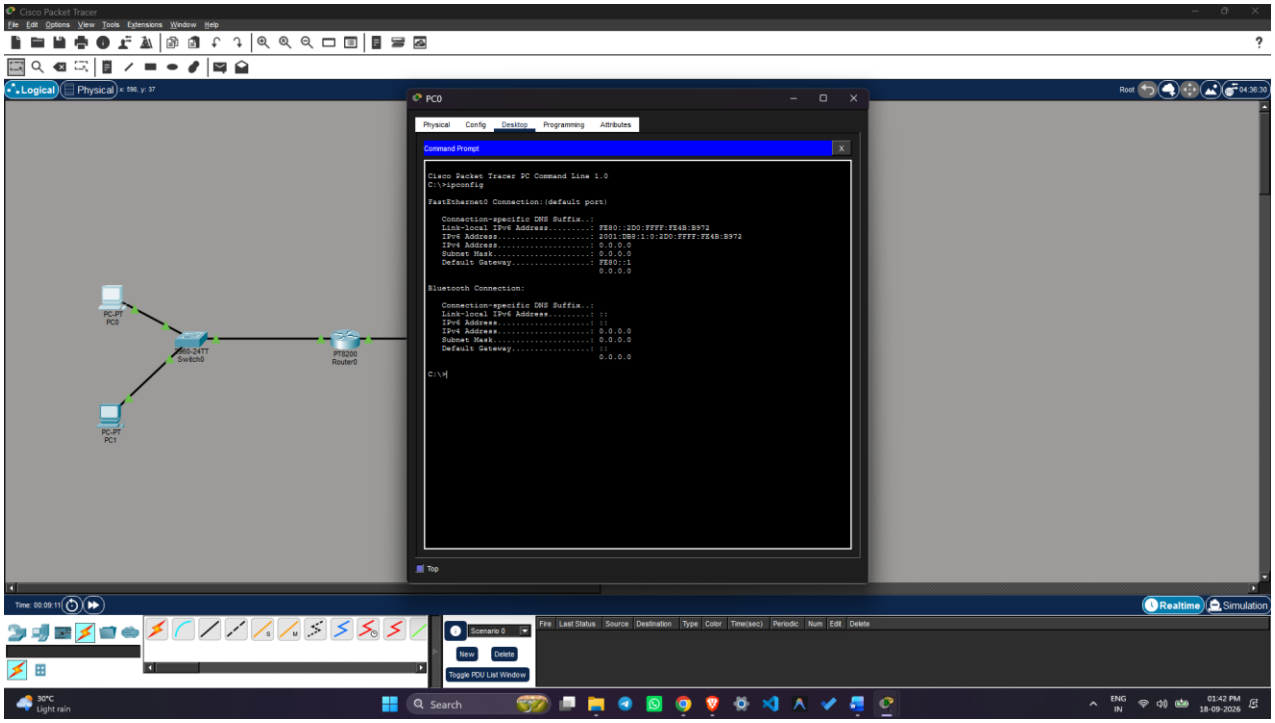
Router Status -

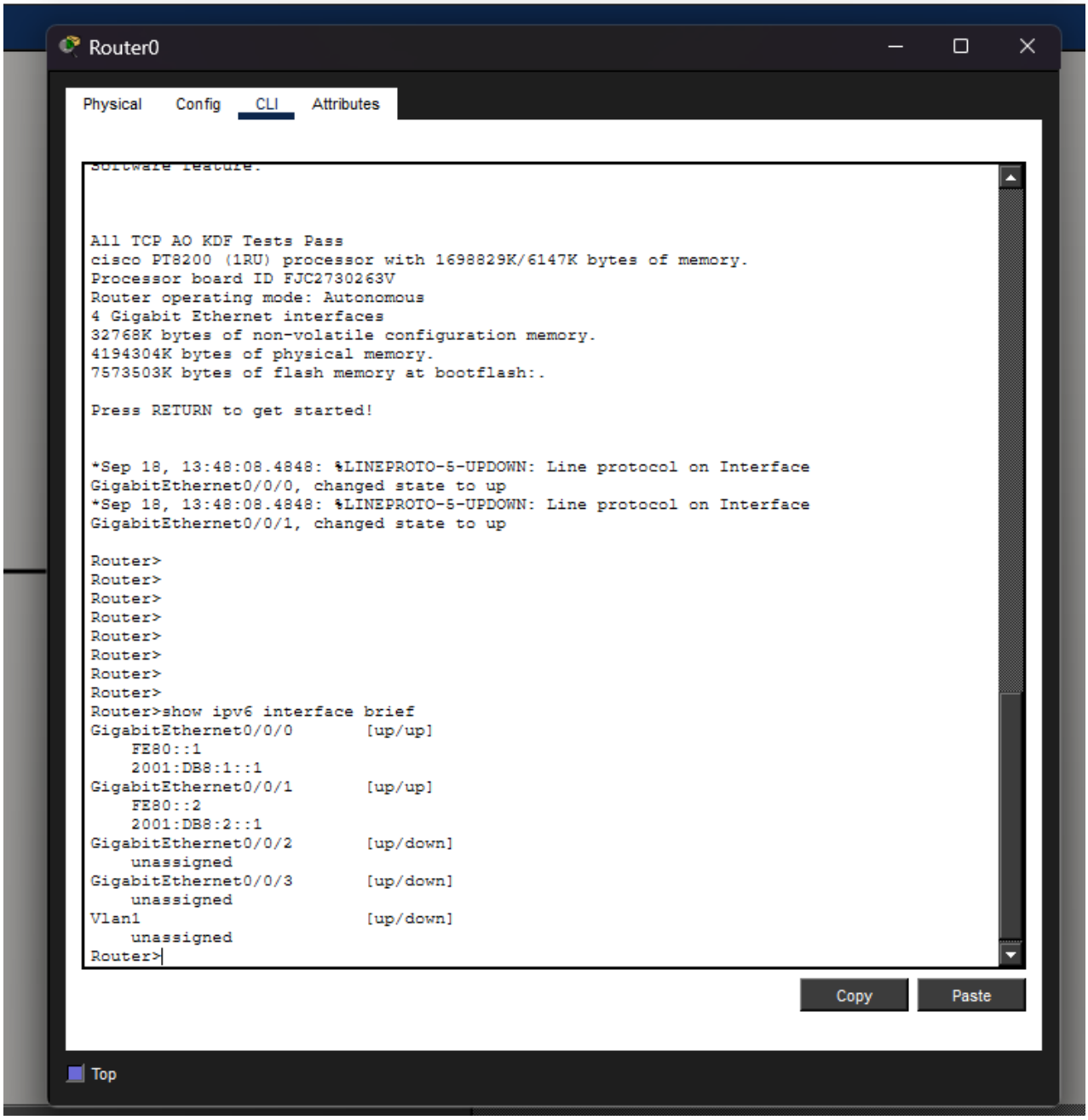
```
Router#
Router#
Router#
Router#
Router#
Router#show ipv6 interface brief
GigabitEthernet0/0/0       [up/up]
    FE80::1
    2001:DB8:1::1
GigabitEthernet0/0/1       [up/up]
    FE80::2
    2001:DB8:2::1
GigabitEthernet0/0/2       [up/down]
    unassigned
GigabitEthernet0/0/3       [up/down]
    unassigned
Vlan1                      [up/down]
    unassigned
Router#
```

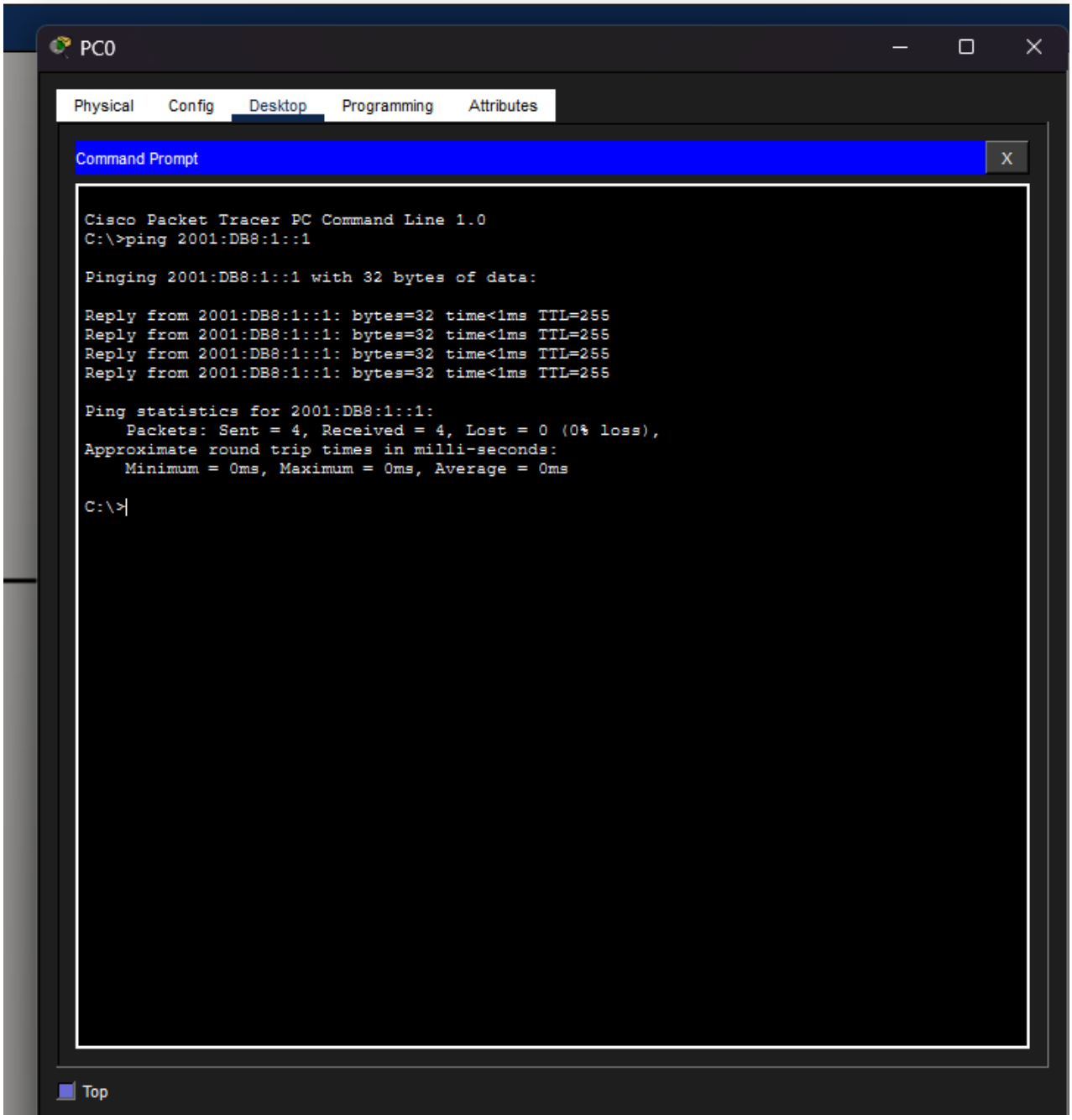
PC IPv6 - Setup

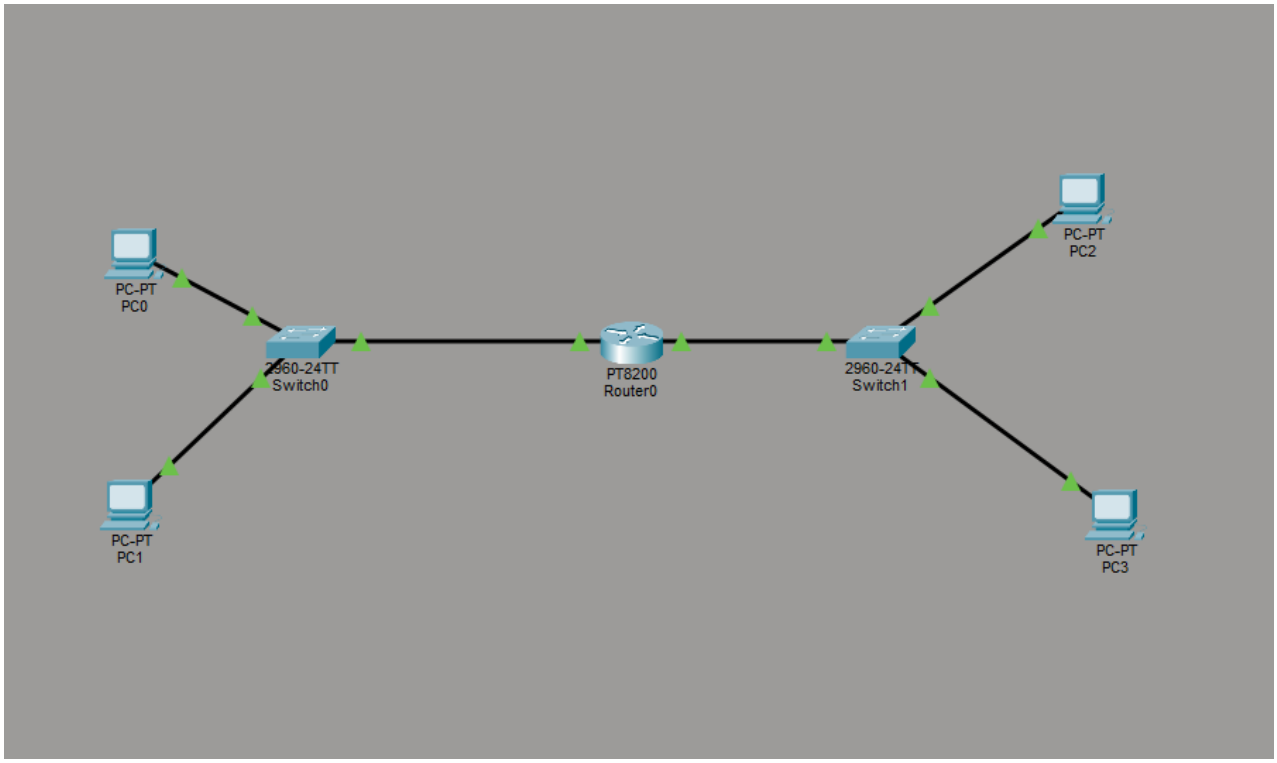



PC1
Physical
Config
Desktop
Programming
Attributes
IP Configuration
X
Interface
FastEthernet0
IP Configuration
DHCP
Static
IPv4 Address
Subnet Mask
Default Gateway
0.0.0.0
DNS Server
0.0.0.0
IPv6 Configuration
Automatic
Static
Ipv6 request successful.
IPv6 Address
2001:DB8:1:0:2E0:F7FF:FE7B:10DD
/
64
Link Local Address
FE80::2E0:F7FF:FE7B:10DD
Default Gateway
FE80::1
DNS Server
802.1X
Use 802.1X Security
Authentication
MD5
Username
Password


Cisco Packet Tracer
Logical
Physical
PC0
Command Prompt
Cisco Packet Tracer PC Command Line 1.0
C:\>ipconfig
FastEthernet0 Connection:(default port)
Connection-specific DNS Suffix..:
Link-local IPv6 Address.....: FE80::2E0:FFFF:FE4B:B972
IPv6 Address.....: 2001:DB8:1:0:2E0:FFFF:FE4B:B972
IPv4 Address.....: 0.0.0.0
Subnet Mask.....: 0.0.0.0
Default Gateway.....: FE80::1
0.0.0.0
Bluetooth Connection:
Connection-specific DNS Suffix..:
Link-local IPv6 Address.....: ::
IPv6 Address.....: ::
IPv4 Address.....: 0.0.0.0
Subnet Mask.....: 0.0.0.0
Default Gateway.....: ::
0.0.0.0
C:\>
Top
Realtime
Simulation


Router0
Physical
Config
CLI
Attributes
Software feature.
All TCP AO KDF Tests Pass
cisco PT8200 (1RU) processor with 1698829K/6147K bytes of memory.
Processor board ID FJC2730263V
Router operating mode: Autonomous
4 Gigabit Ethernet interfaces
32768K bytes of non-volatile configuration memory.
4194304K bytes of physical memory.
7573503K bytes of flash memory at bootflash:.
Press RETURN to get started!
*Sep 18, 13:48:08.4848: %LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0/0, changed state to up
*Sep 18, 13:48:08.4848: %LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0/1, changed state to up
Router>
Router>
Router>
Router>
Router>
Router>
Router>
Router>
Router>show ipv6 interface brief
GigabitEthernet0/0/0 [up/up]
FE80::1
2001:DB8:1::1
GigabitEthernet0/0/1 [up/up]
FE80::2
2001:DB8:2::1
GigabitEthernet0/0/2 [up/down]
unassigned
GigabitEthernet0/0/3 [up/down]
unassigned
Vlan1 [up/down]
unassigned
Router>
Copy
Paste
Top


PC0
Physical
Config
Desktop
Programming
Attributes
Command Prompt
X
Cisco Packet Tracer PC Command Line 1.0
C:\>ping 2001:DB8:1::1
Pinging 2001:DB8:1::1 with 32 bytes of data:
Reply from 2001:DB8:1::1: bytes=32 time<1ms TTL=255
Reply from 2001:DB8:1::1: bytes=32 time<1ms TTL=255
Reply from 2001:DB8:1::1: bytes=32 time<1ms TTL=255
Reply from 2001:DB8:1::1: bytes=32 time<1ms TTL=255
Ping statistics for 2001:DB8:1::1:
Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
Minimum = 0ms, Maximum = 0ms, Average = 0ms
C:\>
Top



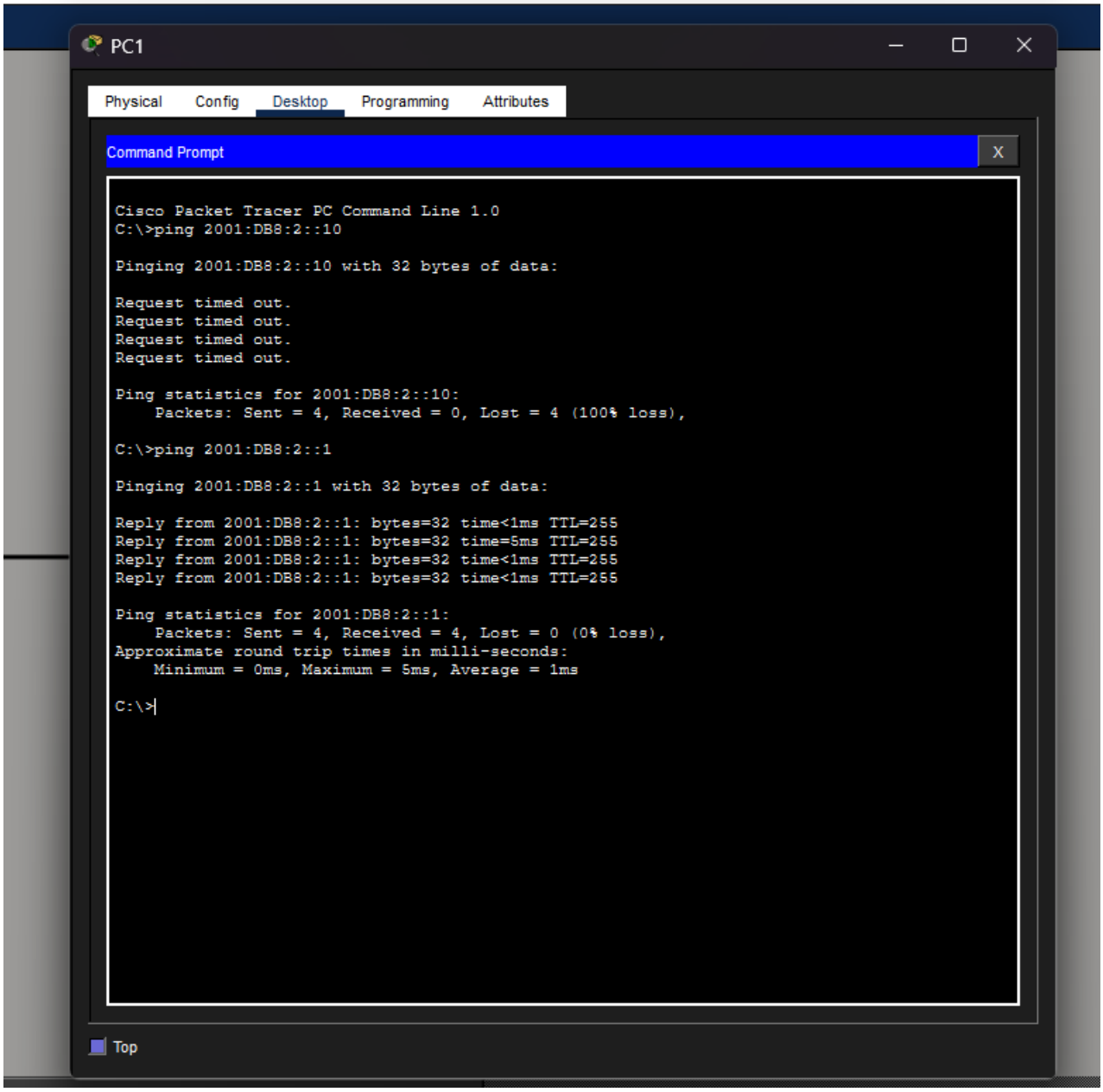

```
C:\>ping 2001:DB8:2::1

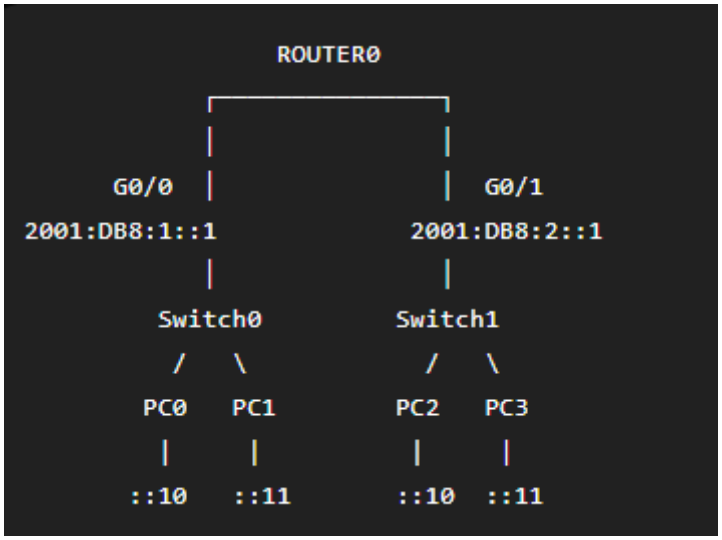
Pinging 2001:DB8:2::1 with 32 bytes of data:

Reply from 2001:DB8:2::1: bytes=32 time<1ms TTL=255
Reply from 2001:DB8:2::1: bytes=32 time=5ms TTL=255
Reply from 2001:DB8:2::1: bytes=32 time<1ms TTL=255
Reply from 2001:DB8:2::1: bytes=32 time<1ms TTL=255

Ping statistics for 2001:DB8:2::1:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 5ms, Average = 1ms

C:\>
```


PC1
Physical
Config
Desktop
Programming
Attributes
Command Prompt
X
Cisco Packet Tracer PC Command Line 1.0
C:\>ping 2001:DB8:2::10
Pinging 2001:DB8:2::10 with 32 bytes of data:
Request timed out.
Request timed out.
Request timed out.
Request timed out.
Ping statistics for 2001:DB8:2::10:
Packets: Sent = 4, Received = 0, Lost = 4 (100% loss),
C:\>ping 2001:DB8:2::1
Pinging 2001:DB8:2::1 with 32 bytes of data:
Reply from 2001:DB8:2::1: bytes=32 time<1ms TTL=255
Reply from 2001:DB8:2::1: bytes=32 time=5ms TTL=255
Reply from 2001:DB8:2::1: bytes=32 time<1ms TTL=255
Reply from 2001:DB8:2::1: bytes=32 time<1ms TTL=255
Ping statistics for 2001:DB8:2::1:
Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
Minimum = 0ms, Maximum = 5ms, Average = 1ms
C:\>
Top



Result:

The working of ARP (Address Resolution Protocol) was successfully demonstrated using Cisco Packet Tracer. The ARP tables of the devices were initially empty, and after performing ping operations between the devices, the corresponding IP-to-MAC address mappings were populated in the ARP tables, confirming successful ARP resolution.

References:

1. https://www.netacad.com/courses/packet-tracer

2. https://skillsforall.com/course/getting-started-cisco-packet-tracer

3. https://www.geeksforgeeks.org/how-address-resolution-protocol-arp-works/

4. https://www.tutorialspoint.com/address-resolution-in-arp

Exp No: 8.1	To configure a wireless network using a wireless router and end devices in Cisco Packet Tracer.
09.09.2026	

Aim

To configure a wireless network using Cisco Packet Tracer.

Objectives

- To create a wireless network using a wireless router.
- To connect multiple end devices using Wi-Fi.
- To verify connectivity using the ping command.

Tools:

- Cisco Packet Tracer
- Wireless router
- PCs or laptops with wireless modules

Theory

A wireless network allows devices such as PCs, laptops, and smartphones to communicate without physical cables using radio signals. In Cisco Packet Tracer, a wireless router acts as the central device that provides wireless connectivity and can automatically assign IP addresses to connected devices using DHCP.

The wireless network is identified by an SSID (Service Set Identifier). Security can be configured using a password to prevent unauthorized access. In this practical, the WPC300N wireless module is installed in the PCs so they can connect to the router wirelessly. Connectivity is verified using the ping command, which checks whether one device can communicate with another.

Procedure

1. Open Cisco Packet Tracer and drag a wireless router into the workspace.
2. Add a few end devices, such as PCs or laptops.
3. Double-click the router and open the GUI tab.
4. Configure the wireless network:
   - Set the SSID (network name).
   - Select WEP security.
   - Set a wireless password.

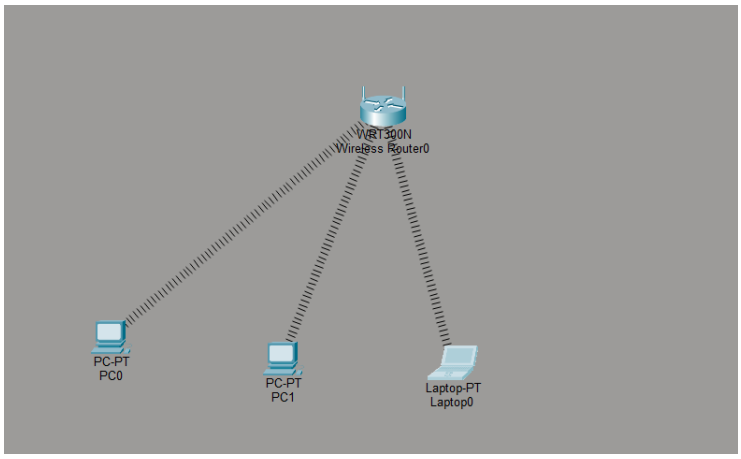
5. For each end device:

   - Open the Physical tab.
   - Turn off the device.
   - Replace the existing module with the WPC300N wireless module.
   - Turn the device back on.

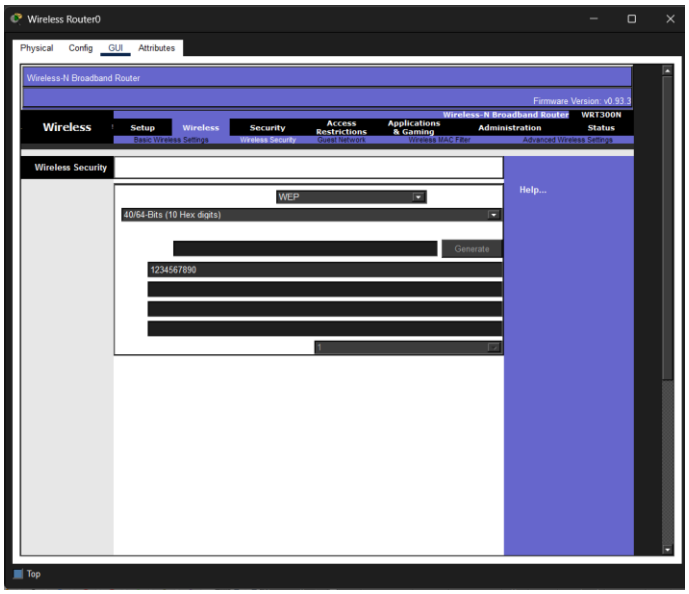
6. Open the PC's Desktop → PC Wireless → Connect tab.
7. Select the configured SSID and enter the password.
8. Repeat for all devices.
9. Open Command Prompt on one PC and ping another PC's IP address.

Output: (Simulation Screenshots):

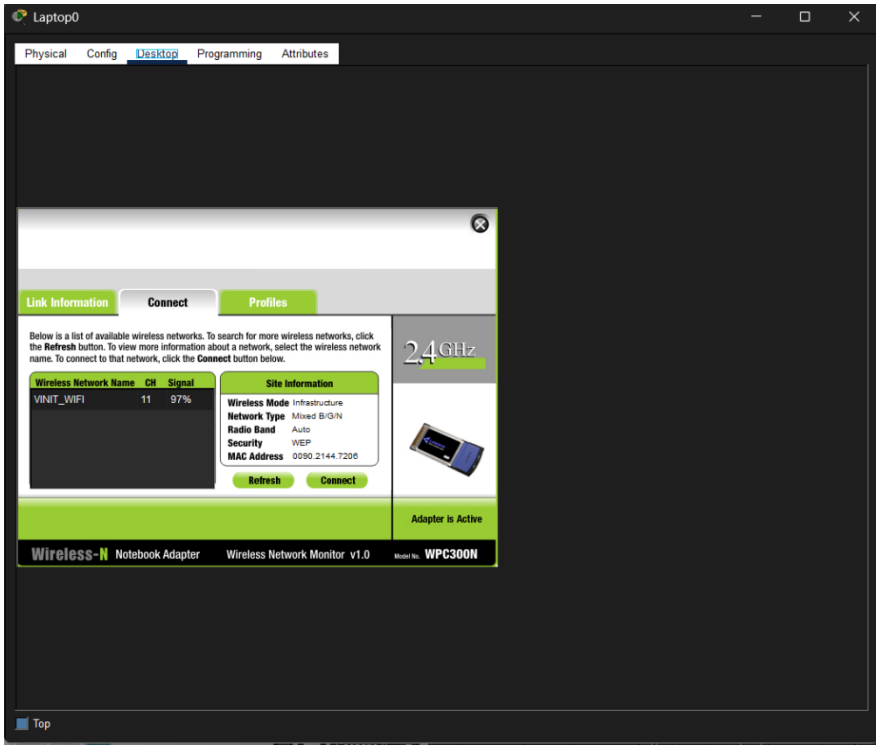
Complete Wireless Topology

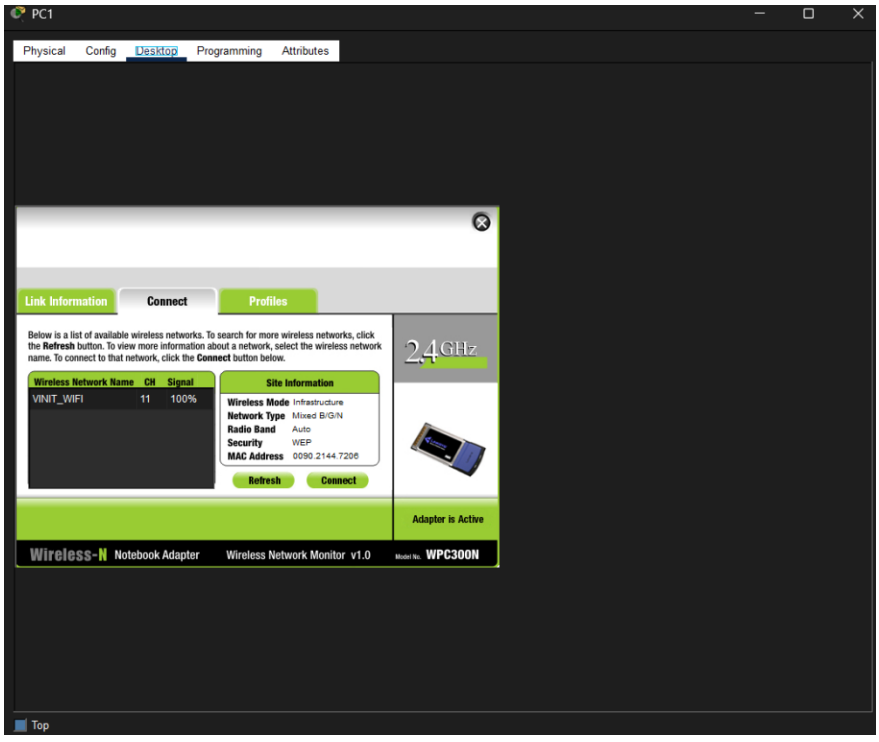


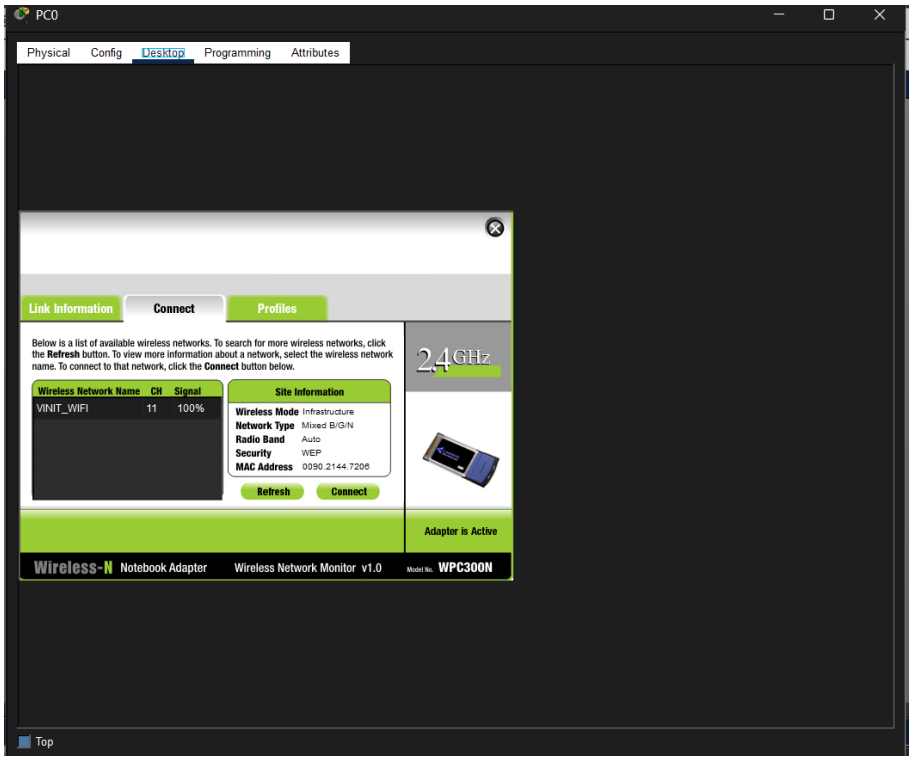
Wireless Router Configuration



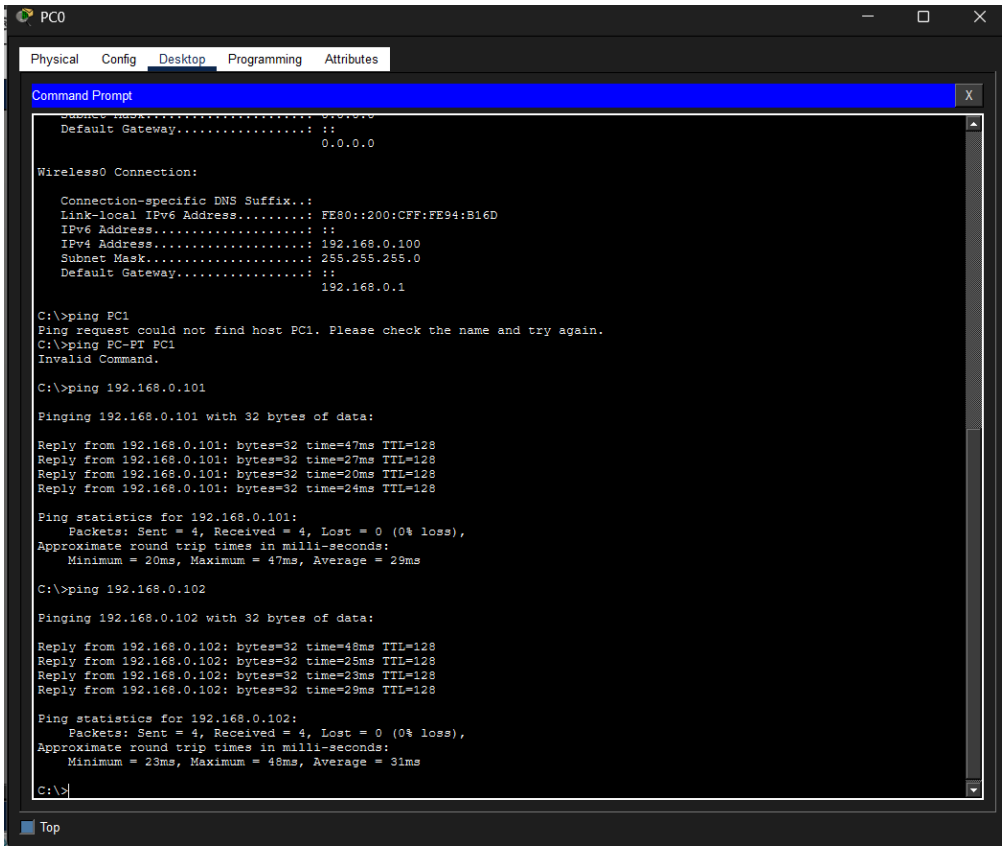
PC Wireless Connection






PC0
Physical
Config
Desktop
Programming
Attributes
Link Information
Connect
Profiles
Below is a list of available wireless networks. To search for more wireless networks, click the Refresh button. To view more information about a network, select the wireless network name. To connect to that network, click the Connect button below.
2.4GHz
Wireless Network Name
CH
Signal
VINIT_WIFI
11
100%
Site Information
Wireless Mode Infrastructure
Network Type Mixed B/G/N
Radio Band Auto
Security WEP
MAC Address 0090.2144.7206
Refresh
Connect
Adapter is Active
Wireless-N Notebook Adapter
Wireless Network Monitor v1.0
Model No. WPC300N
Top

Successful Ping



Result:

The wireless network was successfully configured in Cisco Packet Tracer using a wireless router and end devices. The devices connected through Wi-Fi, obtained IP addresses using DHCP, and connectivity was verified using the ping command.

References:

1. https://www.netacad.com/courses/packet-tracer
2. https://skillsforall.com/course/getting-started-cisco-packet-tracer
3. https://www.geeksforgeeks.org/how-address-resolution-protocol-arp-works/
4. https://www.tutorialspoint.com/address-resolution-in-arp

Exp No: 8.2	To implement an IoT setup in Cisco Packet Tracer.
09.09.2026	

Aim:

To implement an IoT setup in Cisco Packet Tracer.

Objective:

Configuring IoT network with smart devices.

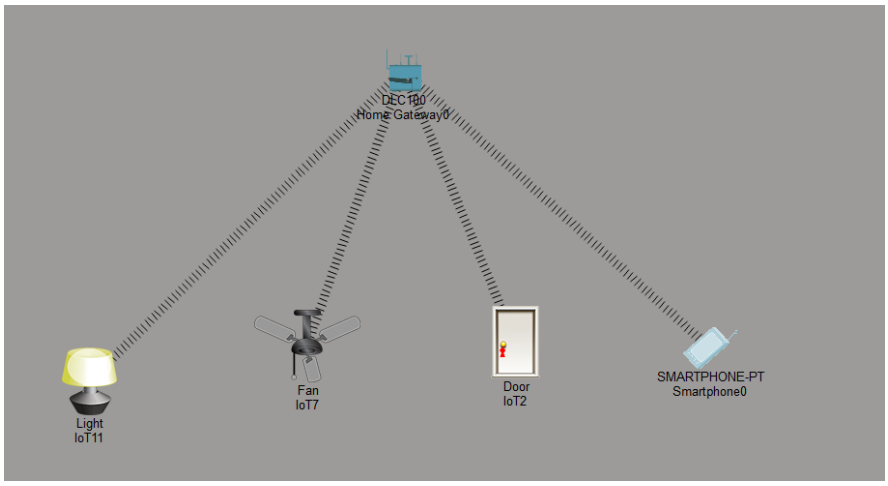
Simulation Tool: Cisco Packet Tracer

Procedure: IoT

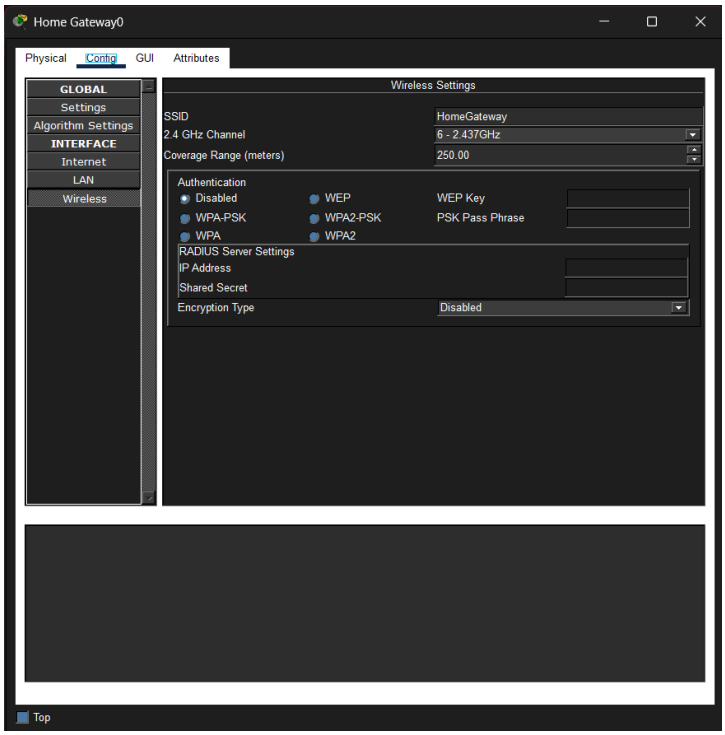
1. Start Up: Cisco Packet Tracer, and select a few Home devices.
2. Select a Home Gateway (Wireless Network Device) and drag it onto the workspace.
3. Go to 'Wireless' in its Settings after double clicking it, and copy its SSID.
4. Double click each device, go to 'Config' and select Home Gateway as the IoT server.
5. Finally, select a Smartphone, double click, go to 'Config' and paste the SSID.
6. You will now see all devices connected to the Gateway. Go to IoT Monitor in Desktop if you'd like to manipulate your chosen end devices through your Smartphone.

Output: (Simulation Screenshots):

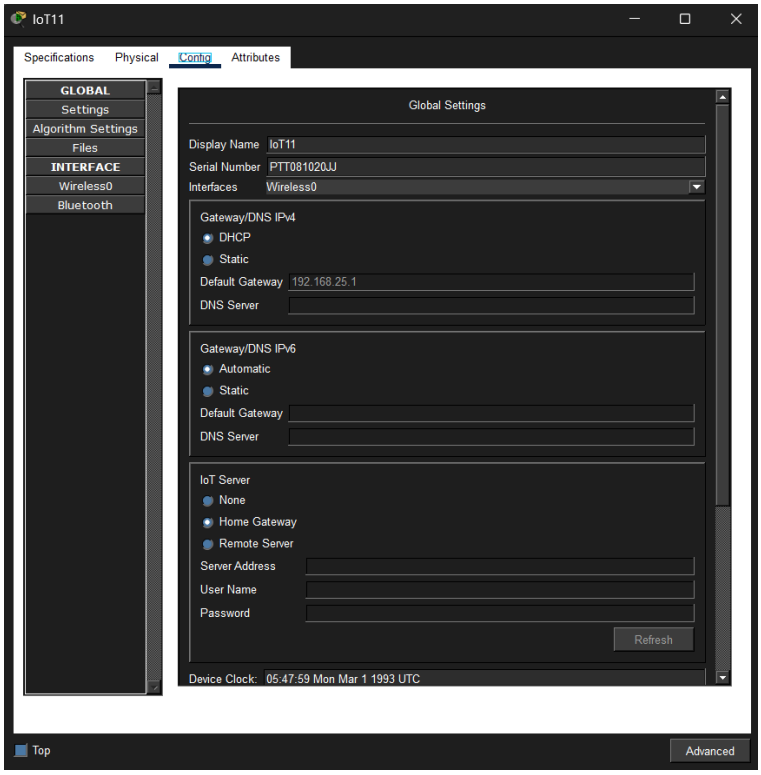
Complete IoT Topology



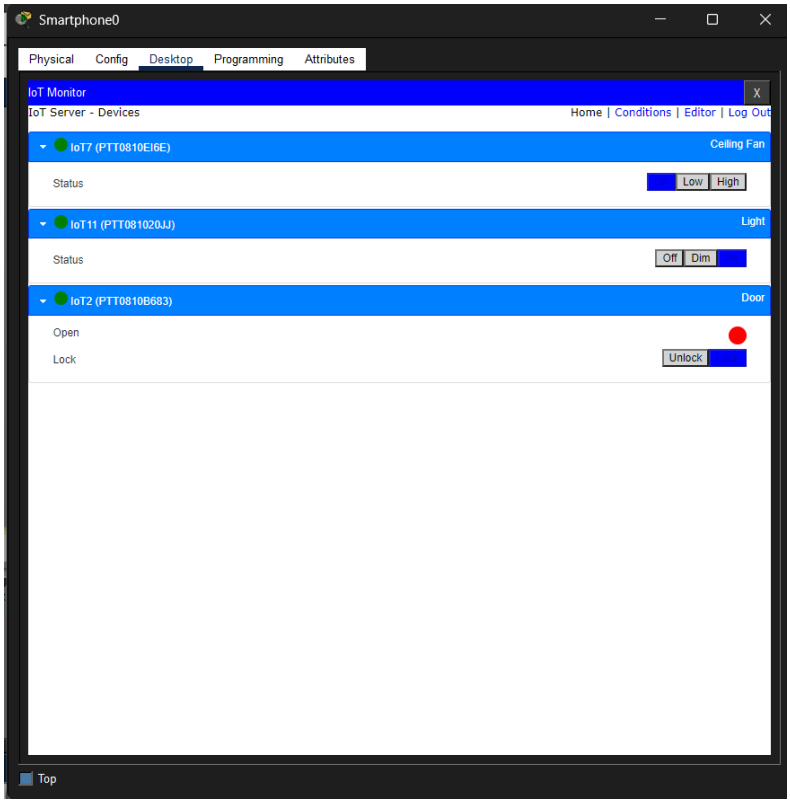
Home Gateway Configuration



IoT Devices Connected



Smartphone IoT Monitor



Result:

The IoT network was successfully configured in Cisco Packet Tracer using a Home Gateway, smart devices, and a smartphone to establish wireless communication and monitor/control the connected devices.

References:

1. https://www.netacad.com/courses/packet-tracer
2. https://skillsforall.com/course/getting-started-cisco-packet-tracer
3. https://www.geeksforgeeks.org/how-address-resolution-protocol-arp-works/
4. https://www.tutorialspoint.com/address-resolution-in-arp

Exp No: 9	Develop a TCP client-server program
18.09.2026	

Aim

To implement a TCP client-server program in Python where the client connects to a server and receives the server's current date and time.

Objective

To implement TCP-based client-server communication for retrieving and displaying the server's current date and time at the client side.

Simulation Tool

Python 3.x And VS Code / Python IDLE

Theory

TCP (Transmission Control Protocol) is a connection-oriented transport layer protocol that provides reliable communication between two devices over a network. In a client-server communication model, the **client** requests a service and the **server** listens for incoming requests and provides the required response.

Socket programming provides an interface for communication between processes over a network. A TCP server creates a socket, binds it to an IP address and port number, listens for incoming connections, and accepts a client connection. The client creates a socket and connects to the server using its IP address and port number.

In this experiment, the client represents a web browser and the server represents a web server. After establishing a TCP connection, the server obtains its **current system date and time** and sends this information to the client. The client receives the data and displays it.

The basic communication process is:

Client → Establish TCP Connection → Server → Send Current Date & Time → Client Displays Date & Time

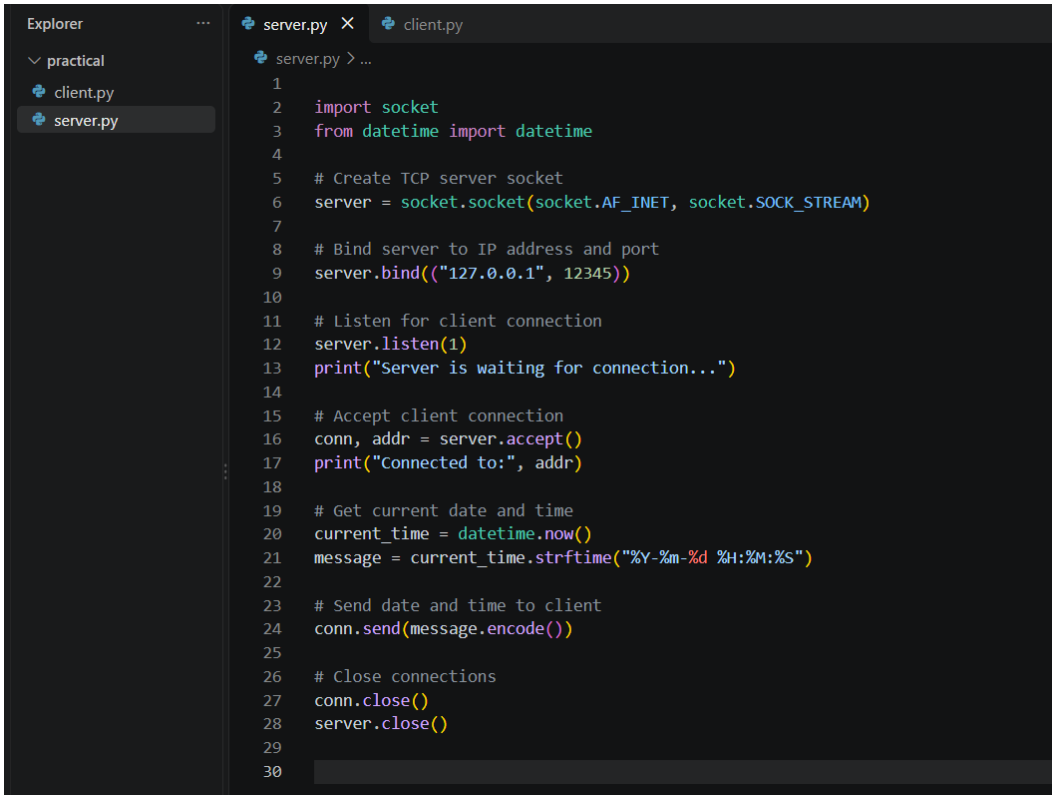
TCP ensures that the data is delivered reliably and in the correct order. The Python socket module can be used to implement this client-server communication.

Procedure:

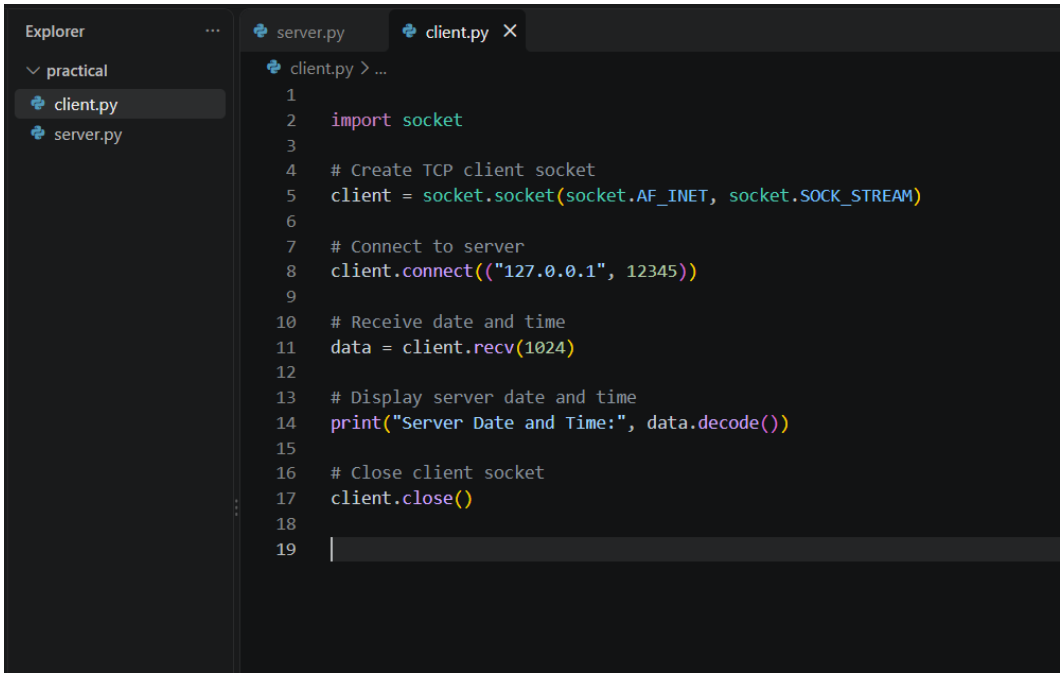
1. Open VS Code.
2. Create a Python file named server.py and write the TCP server program using the socket and datetime modules.
3. Create another Python file named client.py and write the TCP client program using the socket module.
4. In server.py, create a TCP socket using:
5. IP address: 127.0.0.1
6. Port number: 12345
7. Run the server program first. The server waits for a client connection.
8. Open a second terminal and run client.py.
9. The client connects to the server using the same IP address and port number.
10. Once the connection is established, the server obtains the current date and time using the datetime module.
11. The server sends the date and time to the client through the TCP connection.
12. The client receives the data and displays the server's current date and time on the terminal.
13. Close the client and server sockets after communication is completed.
14. Verify the successful TCP communication by checking the server connection message and the date/time displayed by the client.

Output: (Simulation Screenshots):

Server Program



Client Program



Server Output

```
PS C:\Shashi Singh\College\2. Computer Networks\Lab Records\Code> python server.py
Server is waiting for a connection...
Connected to client: ('127.0.0.1', 52551)
Client: Hello from Client!
PS C:\Shashi Singh\College\2. Computer Networks\Lab Records\Code>
```

Client Output

```
PS C:\Shashi Singh\College\2. Computer Networks\Lab Records\Code> python client.py
Server: Hello from Server!
PS C:\Shashi Singh\College\2. Computer Networks\Lab Records\Code>
```

Result:

The TCP client-server program was successfully implemented in Python. The client connected to the server, received the server's current date and time, and displayed it on the terminal.

References:

1. https://www.netacad.com/courses/packet-tracer
2. https://skillsforall.com/course/getting-started-cisco-packet-tracer
3. https://www.geeksforgeeks.org/how-address-resolution-protocol-arp-works/
4. https://www.tutorialspoint.com/address-resolution-in-arp

Exp No: 10	Develop a TCP client-server program
19.09.2026	

Aim

To develop a client–server network communication program that establishes a connection between a client and a server and transfers the client's IP address to the server for display.

Objective

To establish a reliable connection between the client and server using socket programming.

Tools and Technologies Required

- **Programming Language:** Python
- **Networking Concept:** Client–Server Communication
- **Protocol:** TCP
- **Library:** Python socket library
- **IDE/Editor:** VS Code / PyCharm / IDLE
- **Operating System:** Windows/Linux
- **Terminal/Command Prompt**

Theory

A client-server network model is a communication architecture in which a client requests a service from a server, while the server listens for and responds to client requests. Communication between the two processes can be implemented using socket programming**.**

A socket acts as an endpoint for communication between two processes. In TCP-based communication, the server creates a socket, binds it to an IP address and port number, listens for connections, and accepts a client connection. The client creates its own socket and establishes a connection with the server.

An IP address (Internet Protocol address) is used to identify a device or network interface on a network. During a client-server connection, the server can obtain the client's IP address from the connection information provided by the socket.

In this experiment, the client establishes a connection with the server. Once the connection is established, the client's IP address is transferred to or obtained by the server and displayed on the server side**.** This demonstrates how socket programming can be used to establish communication and obtain addressing information between networked processes.

The basic communication process is:

Client → Establish TCP Connection → Server → Obtain Client IP Address → Display IP Address

Python provides the socket module for implementing this type of network communication. The socket connection information can be used by the server to identify the network address of the connected client.

Procedure

1. Install Python on the system and verify the installation using:

   python --version
2. Create a new folder for the practical.
3. Create two Python files:
   - server.py
   - client.py
4. In server.py, import the socket module and create a TCP socket.
5. Assign a port number to the server and bind the socket to the server address.
6. Make the server listen for incoming client connections.
7. In client.py, create a TCP socket and specify the server IP address and port number.
8. Establish a connection from the client to the server.
9. Determine the client's IP address using Python's socket functions.
10. Send the client's IP address from the client to the server.
11. Receive the IP address at the server using the recv() function.
12. Display the received client IP address on the server terminal.
13. Close the client and server sockets after communication is completed.
14. Run the **server first** and then execute the **client** program.

Python Code

server.py client.py

server.py > ...

```python
import socket

# Create socket
server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# Server IP and port
HOST = '0.0.0.0'
PORT = 5000

# Bind socket to address
server_socket.bind((HOST, PORT))

# Listen for client connection
server_socket.listen(1)

print("Server is waiting for a client...")

# Accept connection
client_socket, client_address = server_socket.accept()

print("Client connected!")
print("Client IP Address:", client_address[0])

# Receive IP address sent by client
client_ip = client_socket.recv(1024).decode()

print("IP Address received from client:", client_ip)

# Close connection
client_socket.close()
server_socket.close()
```

server.py client.py

client.py > ...

```python
import socket

# Create socket
client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# Server address
SERVER_HOST = '127.0.0.1'
SERVER_PORT = 5000

# Connect to server
client_socket.connect((SERVER_HOST, SERVER_PORT))

# Get client's IP address
client_ip = socket.gethostbyname(socket.gethostname())

print("Client IP Address:", client_ip)

# Send IP address to server
client_socket.send(client_ip.encode())

print("IP address sent to server successfully.")

# Close connection
client_socket.close()
```

Output

Server Output

```
PS C:\Shashi Singh\College\2. Computer Networks\Lab Records\Code> python server.py
Server is waiting for a client...
Client connected!
Client IP Address: 127.0.0.1
IP Address received from client: 172.25.189.179
PS C:\Shashi Singh\College\2. Computer Networks\Lab Records\Code>
```

Client Output

```
PS C:\Shashi Singh\College\2. Computer Networks\Lab Records\Code> python client.py
Client IP Address: 172.25.189.179
IP address sent to server successfully.
PS C:\Shashi Singh\College\2. Computer Networks\Lab Records\Code>
```

Result

The client–server communication was successfully established using Python socket programming. The client successfully transferred its IP address to the server, and the server received and displayed the IP address.

References

- **Computer Networks Lab Manual**, Practical 10 – Client–Server Communication using Socket Programming, VIT Bhopal University.
- **Python Documentation**, *socket — Low-level networking interface*, Python Software Foundation.
- **Behrouz A. Forouzan**, *Data Communications and Networking*, McGraw-Hill.
- **Andrew S. Tanenbaum and David J. Wetherall**, *Computer Networks*, Pearson.